



UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Análise Comparativa de LLMs Open-Source para a Conversão de Normas de Arquitetura, Engenharia e Construção em Representações RASE

Proposta de Dissertação de Mestrado

Eike Natan Sousa Brito



São Cristóvão – Sergipe

2026

UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Eike Natan Sousa Brito

**Análise Comparativa de LLMs Open-Source para a Conversão
de Normas de Arquitetura, Engenharia e Construção em
Representações RASE**

Proposta de Dissertação de Mestrado apresentada ao Programa de Pós-Graduação em Ciência da Computação da Universidade Federal de Sergipe como requisito parcial para a obtenção do título de mestre em Ciência da Computação.

Orientador(a): Andre Britto de Carvalho

São Cristóvão – Sergipe

2026

Resumo

A verificação da conformidade de projetos com normas técnicas ainda é majoritariamente manual: converter cada norma para um formato computável, como o RASE (*Requirement, Applicability, Selection, Exception*), exige trabalho especializado regra a regra, o que se torna um gargalo para a automação na metodologia BIM. Esta dissertação avalia comparativamente seis LLMs *open-source* (Llama, Dolphin, Gemma, Mistral, Alpaca e Qwen) na conversão automática de normas técnicas brasileiras da AEC para o formato RASE, usando exclusivamente Engenharia de Prompt. A tarefa foi decomposta em três níveis sucessivos de normalização (N1: segmentação atômica das sentenças normativas; N2: identificação dos operadores RASE; N3: estruturação semântica em JSON) e avaliada em cinco experimentos (EN1, EN2, EN1N2, EN3 e EN1N2N3) sobre um *dataset* de 79 normas anotadas da NBR 9050, com treze métricas distribuídas em cinco famílias (lexicais, semânticas, de distância, voltadas para geração e de classificação). Os modelos foram servidos localmente via Ollama, sob protocolo idêntico de *dataset*, *prompts* e *hardware*. O Llama obteve a melhor média global de similaridade (0,730), liderando os experimentos do nível N3 (EN3 e EN1N2N3); o Dolphin (0,725) entregou a melhor relação custo-benefício, cerca de noventa vezes mais rápido no *pipeline* encadeado, e dominou o EN1. As métricas baseadas em *embeddings* (SBERT, BERTimbau, Multilingual e BERTScore) mostraram-se as mais adequadas para validar transformações normativas, por tolerarem reformulações que preservam o sentido. Os operadores Seleção e Exceção colapsaram em N2 (F1 próxima de zero) para todos os modelos, apontando a necessidade de *prompts* mais especializados. Os resultados indicam que LLMs *open-source*, somados a uma decomposição cuidadosa da tarefa, são uma alternativa viável e acessível para automatizar a conversão de normas da AEC em representações computáveis, abrindo caminho para a verificação automática de conformidade em projetos baseados em BIM.

Palavras-chave: Aprendizado de Máquina, Processamento de Linguagem Natural, LLM, Engenharia de Prompt, BIM, RASE.

Abstract

Checking the compliance of engineering designs against technical standards is still largely manual: converting each standard into a computable format, such as RASE (*Requirement, Applicability, Selection, Exception*), requires rule-by-rule expert effort, which becomes a bottleneck for automation within the BIM methodology. This dissertation comparatively evaluates six open-source LLMs (Llama, Dolphin, Gemma, Mistral, Alpaca, and Qwen) on the automatic conversion of Brazilian AEC technical standards to the RASE format, using Prompt Engineering exclusively. The task was decomposed into three successive normalization levels (N1: atomic segmentation of normative sentences; N2: RASE operator identification; N3: semantic JSON structuring) and assessed over five experiments (EN1, EN2, EN1N2, EN3, and EN1N2N3) on a dataset of 79 annotated NBR 9050 standards, with thirteen metrics across five families (lexical, semantic, distance-based, generation-oriented, and classification). The models were served locally through Ollama, under an identical dataset, prompt, and hardware protocol. Llama achieved the best global similarity average (0.730), leading the N3-level experiments (EN3 and EN1N2N3); Dolphin (0.725) delivered the best cost-benefit ratio, running about ninety times faster on the chained pipeline, and dominated EN1. Embedding-based metrics (SBERT, BERTimbau, Multilingual, and BERTScore) proved the most suitable for validating normative transformations, as they tolerate meaning-preserving rephrasings. The Selection and Exception operators collapsed in N2 (F1 close to zero) for every model, signalling the need for more specialized prompts. The results indicate that open-source LLMs, combined with a careful task decomposition, are a viable and accessible alternative for automating the conversion of AEC standards into computable representations, paving the way for automatic compliance checking in BIM-based projects.

Keywords: Machine Learning, Natural Language Processing, LLM, Prompt Engineering, BIM, RASE.

Lista de ilustrações

Figura 01 – Representação do BIM - Fonte: HOCH	17
Figura 02 – Utilização da metodologia RASE na NBR 9050	22
Figura 03 – Matriz de confusão.	24
Figura 04 – Exemplo da transformação de palavras em <i>word embeddings</i>	27
Figura 05 – Exemplo da transformação de <i>word embeddings</i> para <i>sentence embeddings</i>	28
Figura 06 – Rede Neural MLP.	31
Figura 07 – Estrutura de um neurônio no MLP.	32
Figura 08 – Exemplo do deslocamento do gráfico através da mudança da constante bias.	33
Figura 09 – Tipos de camadas e conexões de nodos.	36
Figura 10 – Exemplo de um modelo seq2seq para tradução do inglês para português.	39
Figura 11 – Exemplo de um modelo com o mecanismo de atenção para tradução do inglês para português.	40
Figura 12 – Exemplo do cálculo de uma camada de <i>self-attention</i>	42
Figura 13 – Representação da pilha (<i>stack</i>) do <i>transformer</i>	43
Figura 14 – Camadas do <i>encoder</i> e <i>decoder</i> do <i>transformer</i>	44
Figura 15 – Pipeline de conversão de normas técnicas para o formato RASE.	53
Figura 16 – Estrutura dos cinco experimentos: EN1, EN2 e EN3 (isolados) e EN1N2 e EN1N2N3 (encadeados).	58
Figura 17 – Comparativo modelo × métrica em EN1.	65
Figura 18 – Comparativo modelo × métrica em EN2.	66
Figura 19 – Comparativo modelo × métrica em EN1N2.	67

Lista de tabelas

Tabela 01 – Exemplos de normas brasileiras relevantes à verificação automática em BIM.	18
Tabela 02 – Modelos LLM <i>open-source</i> avaliados, com origem e <i>tag</i> servida via Ollama (definidos em <code>config/models.py</code>).	56
Tabela 03 – Médias por métrica nos cinco experimentos, agregadas sobre os seis modelos.	64
Tabela 04 – Resultados por modelo em EN1 (similaridade e classificação).	65
Tabela 05 – Resultados por modelo em EN2 (similaridade e classificação macro por operador).	66
Tabela 06 – Resultados por modelo em EN1N2 (similaridade).	67
Tabela 07 – Resultados por modelo em EN3 (similaridade e classificação macro por campo).	68
Tabela 08 – Resultados por modelo em EN1N2N3 (similaridade sobre o JSON estruturado).	68
Tabela 09 – Média global por modelo (nove métricas de similaridade × cinco experimentos).	69
Tabela 10 – Tempo total de geração por modelo em N1.	71
Tabela 11 – Tempo total de geração por modelo em N2.	71
Tabela 12 – Tempo total de geração por modelo em N1N2.	71
Tabela 13 – Tempo total de geração por modelo em N3 (EN3 isolado).	72
Tabela 14 – Tempo total de geração por modelo em N1N2N3 (<i>pipeline</i> completo). . . .	72

Lista de códigos

Lista de abreviaturas e siglas

DCOMP	Departamento de Computação
UFS	Universidade Federal de Sergipe
BIM	Building Information Modeling ou Modelagem da Informação da Construção
AEC	Architecture, Engineering and Construction ou Arquitetura, Engenharia e Construção
IA	Artificial Intelligence ou Inteligência Artificial
IAG	Generative AI ou IA Generativa
AM	Machine Learning ou Aprendizado de Máquina
PLN	Natural Language Processing ou Processamento de Linguagem Natural
RNA	Artificial Neural Networks ou Redes Neurais Artificiais
MLP	Multi-Layer Perceptron ou Perceptron Multi-Camadas
RNN	Recurrent Neural Networks ou Redes Neurais Recorrentes
LSTM	Long Short-Term Memory ou Memória de Longo e Curto Prazo
BiLSTM	Bidirectional Long Short-Term Memory ou Memória de Longo e Curto Prazo Bidirecional
LLM	Large Language Model ou Modelo de Linguagem de Grande Porte
EP	Prompt Engineering ou Engenharia de Prompt
BERT	Bidirectional Encoder Representations from Transformers
SBERT	Sentence-BERT
BERTimbau	BERT Pré-treinado para o Português Brasileiro
BERTScore	Métrica de Similaridade Baseada em <i>Embeddings</i> do BERT
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
TF-IDF	Term Frequency–Inverse Document Frequency
FuzzyWuzzy	Biblioteca de Similaridade Lexical Baseada em Distância de Levenshtein

FastText	Modelo de Word Embeddings Baseado em Subpalavras
NILC	Núcleo Interinstitucional de Linguística Computacional
WMD	Word Mover's Distance ou Distância do Movimentador de Palavras
JSON	JavaScript Object Notation
NBR	Norma Brasileira
RASE	Requirement, Applicability, Selection, Exception
N1	Nível 1 de normalização: Segmentação Atômica
N2	Nível 2 de normalização: Identificação dos Operadores RASE
N3	Nível 3 de normalização: Estruturação Semântica em JSON
EN1	Experimento de avaliação isolada do nível N1
EN2	Experimento de avaliação isolada do nível N2
EN1N2	Experimento do pipeline encadeado $N1 \rightarrow N2$
ACC	Accuracy ou Acurácia
PRE	Precision ou Precisão
REC	Recall ou Revocação
F1	F1-score ou Medida F1
MSE	Mean Squared Error ou Erro Quadrático Médio
RMSE	Root Mean Squared Error ou Raiz do Erro Quadrático Médio
MAE	Mean Absolute Error ou Erro Médio Absoluto

Lista de símbolos

θ	Bias
w	Peso
u	Ponto de ativação
q	Query
k	Key
v	Value
W^q	Cabeça de atenção da Query
Q^k	Cabeça de atenção da Key
W^v	Cabeça de atenção do Value

Sumário

1	Introdução	12
1.1	Objetivos	14
1.2	Organização do Documento	14
2	Fundamentação Teórica	16
2.1	<i>Building Information Modeling</i> (BIM)	16
2.2	Normas de Engenharia	18
2.2.1	Normas Regulatórias Brasileiras na AEC	18
2.3	Metodologia RASE	19
2.4	Aprendizado de Máquina	21
2.5	Processamento de Linguagem Natural (PLN)	25
2.6	Métricas de Similaridade Textual	29
2.7	Redes Neurais Artificiais	30
2.8	Memória de Longo e Curto Prazo (LSTM)	37
2.9	<i>Transformers</i>	38
2.10	Modelos de Linguagem de Grande Porte (LLMs)	44
2.10.1	Outros LLMs <i>open-source</i> avaliados	48
2.11	Trabalhos Relacionados	49
2.11.1	Análise dos Trabalhos Relacionados	50
3	Metodologia	52
3.1	Visão Geral	52
3.2	Necessidade de Normalização em Três Níveis	53
3.2.1	N1: Segmentação Atômica	53
3.2.2	N2: Identificação dos Operadores RASE	53
3.2.3	N3: Estruturação Semântica em JSON	54
3.3	Engenharia de Prompt	54
3.3.1	<i>Prompt Direct</i>	54
3.3.2	<i>Prompt Chain-of-Thought</i>	55
3.3.3	<i>Prompt Few-Shot</i>	55
3.4	Modelos LLM Avaliados	55
3.5	<i>Dataset</i>	56
3.6	<i>Pipeline</i> de Execução e Experimentos	57
3.6.1	Experimento EN1	59
3.6.2	Experimento EN2	59
3.6.3	Experimento EN1N2	59

3.6.4	Experimento EN3	59
3.6.5	Experimento EN1N2N3	59
3.7	Métricas de Validação	59
3.7.1	Similaridade Lexical	60
3.7.2	Similaridade Semântica	60
3.7.3	Distância Semântica (WMD)	60
3.7.4	Métricas Voltadas para Geração	60
3.7.5	Métricas de Classificação	60
3.8	Ambiente Computacional	61
4	Resultados	63
4.1	Visão Geral dos Experimentos	63
4.2	Resultados por Experimento	64
4.2.1	EN1: Segmentação N1	64
4.2.2	EN2: Identificação dos Operadores RASE	65
4.2.3	EN1N2: Pipeline Encadeado	66
4.2.4	EN3: Estruturação Semântica em JSON com N2 de referência	67
4.2.5	EN1N2N3: Pipeline Encadeado Completo	68
4.3	Resultados por Modelo	69
4.4	Análise por Métrica	70
4.5	Tempo de Execução	71
4.6	Discussão Integrada	72
5	Conclusão	74
5.1	Síntese dos Resultados	74
5.2	Contribuições	75
5.3	Limitações	75
5.4	Trabalhos Futuros	76
	Referências	78
	Apêndices	85
	APÊNDICE A Prompts utilizados na Engenharia de Prompt	86
A.1	N1 — <i>Prompt Direct</i>	86
A.2	N2 — <i>Chain-of-Thought</i>	87
A.3	N3 — <i>Few-Shot</i>	88

A.3.1	Aplicabilidade	88
A.3.2	Requisito	91
A.3.3	Seleção	93
A.3.4	Exceção	94

1

Introdução

Projetar e fiscalizar obras na engenharia civil brasileira ainda envolve uma quantidade enorme de retrabalho. Parte desse desperdício vem da forma como os projetos são representados: o desenho 2D, padrão histórico do setor, esconde colisões e omite informações que só aparecem no canteiro. A migração para modelos tridimensionais (e, mais especificamente, para a metodologia *Building Information Modeling* ou BIM) vem como resposta a esse problema ([EASTMAN; TEICHOLZ; SACKS, 2011](#)).

O BIM organiza o planejamento e a execução da obra em torno de um modelo 3D detalhado. A metodologia divide-se em dimensões que vão do 3D ao 7D: 3D para o modelo geométrico em si; 4D para a programação temporal; 5D para custos; 6D para sustentabilidade; e 7D para o gerenciamento da operação pós-construção ([EASTMAN; TEICHOLZ; SACKS, 2011](#)). BIM não é um *software*, e sim um conceito que se apoia em sistemas informatizados como o CAD. Quem implementa esse conceito são ferramentas como o Revit, da Autodesk, e o ArchiCAD, da Graphisoft.

Modelar em BIM permite visualizar interferências antes da obra começar, o que reduz o retrabalho em campo. As aplicações típicas vão da análise de projetos ao orçamento, passando pelo planejamento, pela coordenação modular e pela gestão do ciclo de vida ([EASTMAN; TEICHOLZ; SACKS, 2011](#)).

Quanto maior a adoção do BIM, maior o volume de dados gerado por cada projeto atraindo naturalmente a atenção de cientistas de dados, profissionais cuja função é transformar grandes volumes de informação em valor para as organizações ([DAVENPORT; PATIL, 2012](#)). Dentro desse repertório, o Aprendizado de Máquina (AM) é a frente mais promissora.

O AM é a subárea da Inteligência Artificial (IA) que estuda como construir algoritmos capazes de aprender padrões diretamente dos dados, sem programação explícita das regras ([TAN; STEINBACH; KUMAR, 2006](#)). A partir dos padrões aprendidos, esses modelos fazem previsões e respondem a situações novas. Na Engenharia Civil, a IA tem automatizado etapas que antes

dependiam de inspeção humana (desde a detecção de patologias em estruturas até a previsão de prazos de obra) (RANE; CHOUDHARY; RANE, 2023; LIU; ZHANG; LI, 2022).

Três exemplos recentes ilustram esse movimento. Liu, Zhang e Li (2022) combinou BIM, AM e regras de *design* para otimizar o processo de montagem da construção, mostrando ganhos em eficiência e custo. Alawadhi e Yan (2023) treinou Redes Neurais Artificiais sobre dados BIM para reconhecer objetos de construção em fotos: as imagens sintéticas geradas a partir do modelo produziram segmentação semântica eficaz em fotografias reais, abrindo caminho para o uso de dados sintéticos em problemas arquitetônicos. Em outra frente, Saka et al. (2024) alimentou um modelo com 2.280 projetos de demolição para prever a circularidade de resíduos: o sistema estima as quantidades de material reciclável e destina os resíduos para o aterro adequado, apoiando o planejamento.

O BIM, contudo, não resolve sozinho a gestão de normas regulatórias. Cada região tem seu próprio conjunto de regras, muitas delas interpretativas, o que torna a automação um problema em aberto (DIMYADI; AMOR, 2013; SOLIHIN; EASTMAN, 2015).

É aí que entra o Processamento de Linguagem Natural (PLN), área que permite a máquina processar texto humano em tarefas como tradução, agentes de conversação e respostas automatizadas. Dentro do PLN, a IA Generativa (IAG) é a frente que mais cresceu nos últimos anos: seus modelos geram novos dados (texto, imagem, áudio) a partir do treinamento. ChatGPT, da OpenAI (OPENAI et al., 2024), e Llama 3, da Meta (AI@META, 2024), são exemplos conhecidos. Ambos pertencem à categoria dos Modelos de Linguagem de Grande Porte (*Large Language Models* ou LLMs) (ZHAO et al., 2023).

Os LLMs são sistemas de IA voltados a entender e produzir linguagem humana em escala. O treinamento envolve volumes massivos de texto, o que permite ao modelo capturar nuances linguísticas e contextuais. Quase todos os LLMs atuais usam alguma variação de arquitetura *transformer*, escolhida por sua eficiência no processamento de sequências longas (ZHAO et al., 2023).

A adaptação desses modelos a domínios específicos pode ser feita por diferentes caminhos. O mais leve e acessível deles é a *Engenharia de Prompt* (EP): em vez de retreinar pesos ou montar uma infraestrutura de recuperação, ajusta-se apenas a instrução enviada ao modelo.

No setor AEC, os LLMs podem auxiliar tanto na leitura das normas quanto na sua conversão para formatos processáveis por máquina (FUCHS et al., 2024). Só que a literatura ainda tem lacunas: uma delas é justamente a interpretação de leis, códigos e padrões regulatórios, cuja complexidade semântica historicamente desafiou os métodos de IA anteriores (FUCHS et al., 2024).

Para enfrentar esse problema, Hjelseth e Nisbet (2011) propôs a metodologia RASE, que reescreve normas regulatórias como dados computáveis. Cada trecho normativo é decomposto em quatro atributos: **Requisito** (o que precisa ser atendido), **Aplicabilidade** (onde a regra se

aplica), Seleção (opções para cumpri-la) e Exceção (casos em que a regra não vale). Com isso, *softwares* BIM passam a interpretar a norma de forma automática e verificar a conformidade do projeto sem intervenção manual (HJELSETH; NISBET, 2011).

Uma vez convertida para RASE, a norma viabiliza ferramentas integradas ao Revit capazes de validar elemento a elemento do projeto (SANTOS; SAMPAIO, 2021). O gargalo está antes disso: a conversão em si ainda é feita à mão, regra por regra. E como um modelo BIM pode ter centenas ou milhares de elementos, o custo desse trabalho manual cresce rápido.

1.1 Objetivos

Objetivo geral

Avaliar comparativamente o uso de seis Modelos de Linguagem de Grande Porte (LLMs) *open-source* (Llama, Dolphin, Gemma, Mistral, Alpaca e Qwen) na conversão automática de normas técnicas brasileiras da AEC (em especial a NBR 9050) para o formato RASE, utilizando exclusivamente Engenharia de Prompt em três níveis sucessivos de normalização (N1, N2 e N3).

Objetivos específicos

- Definir e implementar três níveis de normalização sobre a metodologia RASE: **N1** (segmentação atômica de sentenças normativas); **N2** (identificação dos operadores RASE (Requisito, Aplicabilidade, Seleção e Exceção)); e **N3** (estruturação semântica das regras em formato JSON).
- Projetar prompts adequados a cada nível, empregando as técnicas *Prompt Direct*, *Prompt Chain-of-Thought* e *Prompt Few-Shot*.
- Comparar os seis LLMs *open-source* sob cinco experimentos: **EN1** (avaliação isolada do N1), **EN2** (avaliação isolada do N2 com o N1 de referência como entrada), **EN1N2** (pipeline encadeado $N1 \rightarrow N2$), **EN3** (avaliação isolada do N3 com o N2 de referência como entrada) e **EN1N2N3** (pipeline encadeado completo $N1 \rightarrow N2 \rightarrow N3$).
- Avaliar a qualidade das saídas com métricas de similaridade lexical (FuzzyWuzzy e TF-IDF), semântica (SBERT, BERTimbau e Multilingual), de distância (*Word Mover's Distance* com FastText e NILC) e voltadas para geração (BERTScore e ROUGE-L), complementadas por métricas de classificação (Acurácia, Precisão, Revocação e F1-score).
- Analisar a propagação de erros entre as etapas do pipeline e o custo computacional (tempo) por modelo.

1.2 Organização do Documento

O restante do documento segue a estrutura abaixo. O **Capítulo 2: Fundamentação Teórica** cobre os conceitos necessários ao trabalho: BIM, Aprendizado de Máquina, PLN, redes neurais (com LSTM, BiLSTM e *Transformers*), LLMs e a metodologia RASE, encerrando com as métricas adotadas e os trabalhos relacionados. O **Capítulo 3: Metodologia** detalha os níveis de normalização N1, N2 e N3, as técnicas de Engenharia de Prompt usadas, os seis LLMs avaliados, o *dataset* de 79 normas, o pipeline de execução e as métricas de validação. Já o **Capítulo 4: Resultados** traz o desempenho por modelo e por métrica nos cinco experimentos (EN1, EN2, EN1N2, EN3 e EN1N2N3), somado aos tempos de execução e à análise comparativa. Por fim, o **Capítulo 5: Conclusão** consolida as contribuições, expõe as limitações e aponta direções de trabalho futuro.

2

Fundamentação Teórica

2.1 *Building Information Modeling* (BIM)

BIM (*Building Information Modeling*, ou Modelagem da Informação da Construção) não é uma tecnologia específica: é um processo articulado ao planejamento e à execução de obras. Para [Penttilä \(2006\)](#), trata-se de um conjunto estruturado de políticas, processos e ferramentas que organiza os dados digitais do projeto ao longo de todo o ciclo de vida do edifício.

A metodologia surgiu logo após o CAD 3D paramétrico, tecnologia que já tinha sido adotada em setores como o aeroespacial, o automotivo e o de manufatura antes de chegar à construção civil ([EASTMAN; TEICHOLZ; SACKS, 2011](#)). O BIM vai além da representação 3D tradicional: incorpora acabamento, custos, fabricante e as relações entre os elementos. O ganho prático aparece quando há alterações estruturais onde todos os dados se atualizam de forma centralizada, sem propagação manual. Em compensação, a adoção exige equipes preparadas tanto no controle quanto no acompanhamento, o que pesa no investimento inicial mas se paga na gestão do ciclo de vida ([COELHO; NOVAES, 2018](#)).

Outro ganho é trabalhar várias dimensões em um único projeto. As informações que alimentam o modelo permitem reduzir falhas, encurtar prazos e ganhar controle sobre as atualizações ([PARREIRA, 2013](#)).

Segundo [Eastman, Teicholz e Sacks \(2011\)](#), o BIM possui sete dimensões que podem ser exploradas independentemente, desde que o modelo 3D já esteja desenvolvido. Isso significa, por exemplo, que uma análise 6D pode ser realizada sem a necessidade de uma inspeção 4D, como ilustrado na Figura 01.

- **BIM 3D: Modelo Virtual 3D:** estende a modelagem 2D para uma representação completa em X, Y e Z. Algoritmos vindos da computação gráfica detectam colisões antes da obra começar.

- **BIM 4D: Programação:** imagine planejar uma laje sabendo, antes da concretagem, em qual semana cada elemento estará pronto. É o que o 4D entrega: o modelo é organizado em etapas, com sequência de montagem atribuída a cada componente. Cronogramas e prazos de entrega passam a ser simuláveis.
- **BIM 5D: Estimativa de Custos:** estima custos por fase do projeto, gera relatórios financeiros e permite comparar alternativas considerando tempo e investimento.
- **BIM 6D: Sustentabilidade:** incorpora dados de impacto ambiental e consumo energético ao modelo, ajustando custos e investimentos durante e depois da obra.
- **BIM 7D: Gerenciamento de Instalações:** cada elemento da construção (estrutura, acabamento, equipamento) ganha um registro detalhado em banco de dados. A partir dele, programam-se manutenções, verificam-se garantias e gerenciam-se os componentes ao longo da vida útil do edifício.

A resistência à adoção do BIM faz sentido quando se mede o tamanho da mudança em relação ao CAD. [Coelho e Novaes \(2018\)](#) mostra que ferramentas como o AutoCAD são, por si só, um obstáculo: trocar de *software* significa reconfigurar rotinas e treinar de novo. [Souza \(2009\)](#) acrescenta um segundo obstáculo, que são volumes maiores de dados e *hardware* mais robusto, o que pesa especialmente em empresas menores.

[Eastman, Teicholz e Sacks \(2011\)](#) lembra ainda que o conceito BIM não pertence a um único *software*. Cobrir o ciclo de vida completo em uma só aplicação seria inviável pela própria complexidade do processo. Na prática, monta-se uma cadeia de ferramentas especializadas, e essa cadeia frequentemente esbarra em problemas de compatibilidade na troca de dados ([IBRAHIM ROBERT KRAWCZYK, 2004](#)). [Bottega \(2012\)](#) relata que, mesmo depois da adoção do BIM, a transferência de arquivos sofre com o tamanho dos dados e com a diversidade de *softwares*.

Toda essa riqueza de informação, porém, só tem valor prático se o modelo respeitar as regras que disciplinam a construção. É aqui que o BIM encontra as normas técnicas: cada elemento modelado (uma rampa, uma porta, uma instalação elétrica) precisa atender a requisitos de segurança, desempenho e acessibilidade fixados em norma. Verificar essa conformidade é uma das aplicações mais promissoras do BIM, mas depende de traduzir o texto normativo para uma forma que o próprio modelo digital consiga checar.

2.2 Normas de Engenharia

As normas técnicas formam o arcabouço regulatório do setor AEC, as mesmas definem os requisitos mínimos de segurança, desempenho, qualidade e interoperabilidade dos projetos. No Brasil, a coordenação cabe à Associação Brasileira de Normas Técnicas (ABNT), que publica as NBRs. No plano internacional, a Organização Internacional de Padronização (ISO)

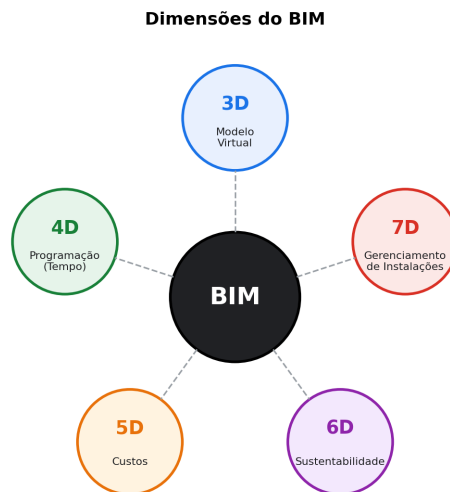


Figura 01 – Representação do BIM - Fonte: [HOCH](#)

publica normas adotadas por diversos países; em BIM, a referência consolidada é a série ISO 19650, voltada à organização e digitalização da informação sobre obras de construção ([INTERNATIONAL ORGANIZATION FOR STANDARDIZATION, 2018](#)).

2.2.1 Normas Regulatórias Brasileiras na AEC

O arcabouço normativo brasileiro do setor AEC é amplo e organiza-se por domínio técnico. Para o escopo desta pesquisa, interessa o subconjunto que regula **requisitos de projeto e desempenho da edificação**, isto é, as NBRs cujo texto se presta à formalização como regras computáveis. A Tabela 01 relaciona exemplos relevantes; o detalhamento da NBR 9050, escolhida como referência operacional do *dataset* desta dissertação, vem na sequência.

Tabela 01 – Exemplos de normas brasileiras relevantes à verificação automática em BIM.

Norma	Domínio	Tipo de requisito
NBR 9050	Acessibilidade	Geométrico, dimensional, de uso
NBR 9077	Saídas de emergência em edifícios	Geométrico, dimensional, de fluxo
NBR 15575	Desempenho de edificações habitacionais	Térmico, acústico, de durabilidade
NBR 6118	Projeto de estruturas de concreto	Estrutural, de detalhamento
NBR 5410	Instalações elétricas de baixa tensão	Elétrico, de segurança
NBR 5626	Sistemas prediais de água fria	Hidráulico, dimensional
NBR 13714	Sistemas de hidrantes prediais	De segurança contra incêndio

Cada uma dessas normas combina três camadas textuais: (i) *definições e referências*, sem valor de verificação direta; (ii) *requisitos prescritivos*, com limites numéricos e geométricos passíveis de checagem automática; e (iii) *recomendações e exceções*, em texto livre, frequentemente condicionadas a categorias específicas do edifício. Em qualquer das três camadas, o texto bruto é

longo, recursivo e fortemente dependente de definições prévias, fazendo com que a tradução para regras computáveis um trabalho não trivial e, hoje, predominantemente manual.

A NBR 9050 ([ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2020](#)) é o caso recorrente na construção civil brasileira: estabelece os critérios e parâmetros técnicos para acessibilidade a edificações, mobiliário, espaços e equipamentos urbanos. A norma cobre desde sinalização tátil e auditiva até dimensões de rampas, portas, sanitários adaptados e rotas acessíveis. O texto é majoritariamente prescritivo, com valores numéricos diretamente verificáveis em um modelo BIM, por exemplo, “a inclinação máxima admissível em rampas é de 8,33%”, ou “o vão livre de portas acessíveis deve ser igual ou superior a 0,80 m”. É justamente esse perfil prescritivo, com requisitos quantitativos e exceções bem delimitadas, que faz da NBR 9050 a escolha natural para o *dataset* desta pesquisa: 79 trechos normativos extraídos da NBR 9050 e de NBRs adjacentes foram anotados nos três níveis (N1, N2 e N3) descritos no Capítulo de Método.

O problema é que a aplicação dessas normas continua predominantemente manual. O profissional lê trechos longos de texto, identifica os requisitos que interessam e verifica item a item contra o projeto. Em um modelo BIM com centenas ou milhares de elementos sujeitos a normas simultâneas, esse esforço cresce de forma combinatória. [Eastman et al. \(2009\)](#) mostram que a verificação manual é cara, frágil e pouco escalável, é exatamente daí que vem a pesquisa em checagem automática baseada em regras.

Automatizar essa verificação, porém, esbarra em três obstáculos concretos. O primeiro é a linguagem natural em que os textos normativos são escritos: ambiguidade e referências cruzadas atrapalham a tradução direta para regras computáveis ([DIMYADI; AMOR, 2013](#)). O segundo é a variação regional, normas mudam entre países e contextos, o que limita o reaproveitamento do que já foi formalizado em outros lugares ([DIMYADI; AMOR, 2013](#); [SOLIHIN; EASTMAN, 2015](#)). O terceiro é semântico: distinguir o que é requisito, o que é aplicabilidade e o que é exceção dentro de um trecho normativo exige um esquema padronizado que sirva a ferramentas diferentes ([SOLIHIN; EASTMAN, 2015](#)).

A resposta que a literatura vem ensaiando é estruturar a norma em um esquema computável antes da verificação. [Beach et al. \(2015\)](#) demonstram que um sistema baseado em regras consegue automatizar a checagem regulatória da construção quando as normas estão formalizadas. A metodologia RASE, proposta por [Hjelseth e Nisbet \(2011\)](#), vai nessa direção: marca cada trecho normativo como Requisito, Aplicabilidade, Seleção ou Exceção, criando um substrato adequado à verificação automática em modelos BIM. Em paralelo, trabalhos recentes começaram a investigar LLMs como caminho para baratear essa tradução das normas em representações computáveis ([FUCHS et al., 2024](#)).

2.3 Metodologia RASE

Para capturar informações presentes em regulamentos, estruturar a conversão dessas regras em linguagem computacional e viabilizar a verificação automática em escala, [Hjelseth \(2015\)](#) propôs duas metodologias complementares: a Tx3 e a RASE.

A metodologia Tx3 propõe três ações a serem realizadas conforme a classificação das informações presentes nos regulamentos:

- **Transcrever (T1):** Deve ser aplicada para instruções que podem ser diretamente transcritas em regras computáveis;
- **Transformar (T2):** Deve ser aplicada para instruções que necessitam ser reestruturadas ou reescritas de forma a preservar o sentido original;
- **Transferir (T3):** Deve ser aplicada quando os requisitos são expressos de maneira imprecisa, sem conexão clara entre os objetivos do regulamento e os indicadores. Nestes casos, é necessária a análise de um profissional especializado.

Após a classificação do texto original, os elementos classificados como T1 e T2 são reescritos e submetidos à metodologia RASE. A metodologia RASE, baseada em conceitos semânticos, transforma documentos normativos em regras bem definidas, utilizáveis em *softwares* baseados em BIM ([HJELSETH; NISBET, 2011](#)). O processo consiste na identificação de quatro operadores lógicos:

- **Requisito (R):** Expressa o dever associado, frequentemente identificado pelo imperativo “deve”. Por exemplo, na frase “Toda rota acessível de espaço privado deve ser provida de iluminação natural ou artificial com nível mínimo de iluminância de 150 lux medidos a 1,00 m do chão”, o requisito é: “deve ser provida de iluminação natural ou artificial com nível mínimo de iluminância de 150 lux”.
- **Aplicabilidade (A):** Identifica a quem ou ao que o requisito se aplica. No exemplo anterior, a aplicabilidade é: “rota acessível”.
- **Seleção (S):** Indica um subconjunto da aplicabilidade. No exemplo, a seleção é: “de espaço privado”;
- **Exceção (E):** Define os elementos excluídos da regra. Completando o exemplo anterior com a norma “São aceitos níveis inferiores de iluminância para ambientes específicos, como cinemas, teatros ou outros, conforme normas técnicas específicas.”, a exceção é: “níveis inferiores para cinemas, teatros e outros”, pois isso não se aplica a regra.

Para facilitar a visualização, utiliza-se um sistema de cores, onde de costume é utilizado: Requisitos (azul), Aplicabilidade (verde), Seleção (vermelho) e Exceção (laranja). Cada operador identificado é associado a atributos específicos:

- **Objeto:** Termos padronizados, como “escada” ou “corrimão”.
- **Propriedade (A):** Termos classificados, como “nível de iluminância” ou “sistema de segurança”.
- **Comparador:** Pode ser maior, menor, igual, diferente, entre outros.
- **Valor:** Valores numéricos, unidades ou descrições.
- **Unidade:** Tipo de unidade do valor numérico caso exista.

Como exemplo, considere o texto da norma NBR 9050: “A inclinação transversal da superfície deve ser de até 2% para pisos internos e de até 3% para pisos externos. A inclinação longitudinal da superfície deve ser inferior a 5%. Inclinações iguais ou superiores a 5% são consideradas rampas e, portanto, devem atender a 6.6.”

Aplicando a metodologia Tx3 e RASE:

- **Tx3 (Transcrever):**
 - Pisos internos devem ter inclinação transversal menor ou igual a 2%;
 - Pisos externos devem ter inclinação transversal menor ou igual a 3%;
 - Pisos internos devem ter inclinação longitudinal menor do que 5%;
 - Pisos externos devem ter inclinação longitudinal menor do que 5%;
 - Pisos com inclinação longitudinal maior ou igual a 5% são considerados rampas e devem atender a 6.6;
- **RASE (Operadores):**
 - A{Pisos} S{internos} R{devem ter inclinação transversal menor ou igual a 2%};
 - A{Pisos} S{externos} R{devem ter inclinação transversal menor ou igual a 3%};
 - A{Pisos} S{internos} R{devem ter inclinação longitudinal menor do que 5%};
 - A{Pisos} S{externos} R{devem ter inclinação longitudinal menor do que 5%};
 - A{Pisos} E{com inclinação longitudinal maior ou igual a 5% são consideradas rampas} R{devem atender a 6.6};

Figura 02 – Utilização da metodologia RASE na NBR 9050

Frase original: "Pisos internos devem ter inclinação transversal menor ou igual a 2%"

A — Aplicabilidade	S — Seleção	R — Requisito			
Pisos	internos	devem ter inclinação transversal menor ou igual a 2%			
Tipo	Objeto	Propriedade	Comparador	Valor	Unidade
A	piso	—	—	—	—
S	piso	tipo	=	interno	—
R	piso	inclinação transversal	≤	2	%

Fonte: Própria (2025).

Concluída a aplicação dos operadores RASE, gera-se o atributo correspondente a cada operador, como mostra a Figura 02.

Aplicar a RASE à mão, porém, reintroduz o gargalo que se quer eliminar: marcar operador a operador exige um especialista e não escala para o volume de normas do setor AEC. Identificar, em uma sentença normativa, o que é Requisito, Aplicabilidade, Seleção ou Exceção é, no fundo, um problema de classificação e de estruturação de texto, que é exatamente o tipo de tarefa que o Aprendizado de Máquina aprende a partir de exemplos, em vez de regras codificadas à mão.

2.4 Aprendizado de Máquina

O Aprendizado de Máquina (AM) é o ramo da Inteligência Artificial (IA) que estuda como máquinas podem inferir regras a partir de exemplos, em vez de seguir regras codificadas à mão por um programador (Mitchell, 1997). Em vez de dizer ao computador como classificar um e-mail como spam, mostram-se a ele milhares de e-mails já rotulados e deixa-se que descubra os padrões. A partir desse processo de treinamento, o algoritmo gera um modelo capaz de lidar com situações novas (TAN; STEINBACH; KUMAR, 2006).

A literatura distingue três grandes paradigmas de aprendizagem:

- **Aprendizagem supervisionada:** o conjunto de treino traz cada exemplo já rotulado, e o modelo aprende a prever esses rótulos em dados novos. Tipicamente, isso se desdobra em duas tarefas: *classificação* (categorizar uma instância em uma classe discreta) e *regressão* (estimar um valor contínuo a partir do histórico) (Faceli et al., 2011).
- **Aprendizagem não supervisionada:** aqui os dados não vêm com rótulos. O objetivo passa a ser descobrir estrutura nos próprios dados, agrupar instâncias parecidas, identificar

anomalias ou gerar recomendações. Mesmo sem rótulos, o modelo consegue separar os dados em grupos a partir dos padrões observados (Faceli et al., 2011).

- **Aprendizagem por reforço:** o modelo aprende por tentativa e erro. A cada ação, o agente recebe uma recompensa ou uma punição; ao longo do tempo, aprende a sequência de decisões que maximiza a recompensa acumulada. É a abordagem por trás de jogos, sistemas de controle e simulações em ambientes complexos (Cerri; Carvalho, 2017).

Em classificação, o modelo deve generalizar os padrões aprendidos no treino para categorizar instâncias novas. Em regressão, o alvo é um valor contínuo, como por exemplo, prever a venda futura de um produto, estimar o preço de um imóvel a partir de variáveis históricas, calcular a expectativa de vida de um país (Guimarães; Meireles; Almeida, 2019).

Exemplos clássicos de classificação: a partir de uma imagem, decidir se a pessoa é homem ou mulher; a partir de um texto, identificar o sentimento como positivo ou negativo. Já a regressão aparece, por exemplo, na previsão de vendas, na avaliação imobiliária ou no cálculo de expectativa de vida.

Esse processo todo segue um fluxo de etapas relativamente estável: obter os dados; preparar, limpar e tratar; escolher e treinar o modelo; testar; avaliar; refinar (TAN; STEINBACH; KUMAR, 2006). A qualidade do que se obtém no final depende muito da qualidade dos dados de entrada (Guimarães; Meireles; Almeida, 2019). Daí a importância de um pré-processamento rigoroso.

O pré-processamento ataca os ruídos que aparecem nos dados brutos, tratando anomalias, valores ausentes, atributos irrelevantes que prejudicam o aprendizado. O sucesso dessa etapa depende, em boa medida, da habilidade do profissional em diagnosticar o problema e escolher a técnica certa (Neves, 2003).

Outros problemas aparecem no próprio treinamento. O *overfitting* ocorre quando o modelo decora demais os dados de treino e perde a capacidade de generalizar. O *underfitting* é o oposto: o modelo não captura padrão suficiente e entrega desempenho insatisfatório (RUSSELL; NORVIG, 2010).

Uma vez que o modelo é construído, torna-se essencial medir seu desempenho. Entre as métricas mais comuns utilizadas para esse propósito estão:

- **A acurácia (ACC ou *accuracy*):** É uma métrica muito utilizada na literatura, e sua fórmula calcula o número de acertos dividido pelo número total (número de acertos + número de erros). Essas informações são retiradas da matriz de confusão (Equação 2.1). A matriz de confusão (Figura 03) é uma tabela que mostra o desempenho de um algoritmo de classificação. Se o problema de classificação for binário, ela apresentará quatro informações, que são:

$$ACC = \frac{(TP + TN)}{(TP + FN + FP + TN)} \quad (2.1)$$

- **Verdadeiro Positivo (TP ou *True Positive*)**: Ocorre quando a classe que estamos buscando foi prevista corretamente. Exemplo: a pessoa está doente, e o modelo previu corretamente que está doente;
 - **Falso Positivo (FP ou *False Positive*)**: Ocorre quando a classe que estamos buscando foi prevista incorretamente. Exemplo: a pessoa não está doente, mas o modelo previu incorretamente que está doente;
 - **Verdadeiro Negativo (TN ou *True Negative*)**: Ocorre quando a classe que não estamos buscando prever foi prevista corretamente. Exemplo: a pessoa não está doente, e o modelo previu corretamente que não está doente;
 - **Falso Negativo (FN ou *False Negative*)**: Ocorre quando a classe que estamos buscando foi prevista incorretamente. Exemplo: a pessoa está doente, mas o modelo previu incorretamente que não está doente.
- A **precisão (PRE ou *precision*)**: É utilizada para trabalhar somente com os valores positivos, e sua fórmula é calculada pelos valores verdadeiros positivos divididos pelo total de predições positivas (verdadeiro positivo + falso positivo) (Equação 2.2).

$$PRE = \frac{TP}{(TP + FP)} \quad (2.2)$$

- A **revocação (REC ou *recall*)**: É utilizada para encontrar exemplos de uma classe, e sua fórmula é calculada pelos valores verdadeiros positivos divididos pelo número de resultados que deveriam ser apresentados (verdadeiro positivo + falso negativo) (Equação 2.3).

$$REC = \frac{TP}{(TP + FN)} \quad (2.3)$$

- A **medida F1 (F1S ou *F1-Score*)**: É a junção da precisão com a revocação, resultando em um número único que indica a qualidade geral do modelo (Equação 2.4). O cálculo da medida F1 é dado por:

$$F1S = \frac{2 \cdot PRE \cdot REC}{PRE + REC} \quad (2.4)$$

Outro conceito recorrente é a transferência de aprendizado (*Transfer Learning*). A ideia é reaproveitar um modelo já treinado em uma tarefa para acelerar o aprendizado em outra tarefa relacionada, mesmo quando os dois conjuntos de dados são diferentes (Shao; Zhu; Li, 2015).

Figura 03 – Matriz de confusão.

		Valor Real	
		Positivo	Negativo
Predição	Positivo	Verdadeiro Positivo (TP)	Falso Positivo (FP)
	Negativo	Falso Negativo (FN)	Verdadeiro Negativo (TN)

Fonte: Própria (2025).

Já o Aprendizado Profundo (*Deep Learning*) é a vertente do AM que se apoia em redes neurais profundas, inspiradas na estrutura dos neurônios biológicos. A rede tem camadas de entrada, ocultas e de saída; em cada uma, os dados são transformados de modo a se aproximar gradualmente da resposta desejada. As aplicações conhecidas vão do reconhecimento de imagens ao PLN e aos sistemas de recomendação (Goodfellow; Bengio; Courville, 2016; Schmidhuber, 2015).

2.5 Processamento de Linguagem Natural (PLN)

Em 1950, Alan Turing, propôs o Teste de Turing: um experimento em que um avaliador tenta decidir se está conversando com uma pessoa ou com uma máquina. É comum apontar esse teste como o marco fundador da reflexão sobre Processamento de Linguagem Natural (PLN) (RUSSELL; NORVIG, 2010).

O PLN tem por objetivo viabilizar tarefas computacionais sobre a linguagem humana,

intermediando a interação entre máquinas e usuários por texto ou fala (Jurafsky; Martin, 2009). Para Liddy, trata-se de uma técnica que analisa e representa o texto de modo a se aproximar do processamento que um humano faria em cada aplicação (Liddy, 1998).

A diferença para linguagens formais como Python ou Java é grande. Linguagens naturais como o português ou o inglês não são determinísticas e estão cheias de ambiguidade. Nas formais, a sintaxe precisa ser seguida à risca, e a semântica é desenhada justamente para evitar ambiguidade.

Os sistemas de PLN podem ser classificados de acordo com o nível de unidade linguística processada (Liddy, 1998), como segue:

- **Nível Fonológico:** Interpretação dos sons da língua. Um exemplo é a diferenciação entre as palavras "faca" e "vaca" por meio do som.
- **Nível Morfológico:** Análise da função, estrutura e flexão das palavras. Por exemplo, identificar que "andar" é um verbo que indica ação.
- **Nível Lexical:** Análise do significado das palavras, incluindo suas relações semânticas. Por exemplo, compreender que "pedra" está associada a palavras como "pedregulho", "pedraria" e "pedrinha".
- **Nível Sintático:** Estudo da estrutura das frases e de como são organizadas para transmitir significado.
- **Nível Semântico:** Análise do significado das palavras e frases, como identificar que o verbo "levar" pode significar transportar, carregar ou guiar, dependendo do contexto.
- **Nível de Discurso:** Interpretação de textos mais extensos, considerando sua estrutura e significado.
- **Nível Pragmático:** Compreensão do contexto comunicacional. Por exemplo, quando alguém diz "Nossa, as janelas estão todas abertas! Como está frio aqui!" e outra pessoa responde "Vou fechá-las", é possível inferir a intenção implícita da primeira pessoa.

Bulengo propõe quatro etapas para o PLN, executadas nessa ordem: morfológica, sintática, semântica e pragmática (Bulegon; Moro, 2010). Santos discorda em parte e argumenta que normalmente só as três primeiras são utilizadas. Na análise morfológica identificam-se substantivos, verbos e adjetivos, montando um “dicionário” que alimenta a análise sintática, esta, por sua vez, decide quem é sujeito, predicado e complemento. A análise semântica fecha o ciclo lidando com os termos ambíguos a partir do contexto (Santos et al., 2015).

Implementar todos os níveis em um único sistema é raro: a complexidade cresce a cada camada. Mesmo assim, o PLN aparece em uma quantidade enorme de aplicações, como por

exemplo, reconhecedores e sintetizadores de fala, corretores ortográficos, tradutores automáticos, geração e sumarização de texto, extração de informações, interfaces de linguagem natural (Vieira; Lima, 2001).

À medida que o campo cresceu, o PLN passou a beber em Filosofia da Linguagem, Psicologia, Matemática, Linguística e Ciência da Computação. Assim como no AM, há um conjunto de etapas de pré-processamento que estruturam a linguagem, reduzem o vocabulário e melhoram o desempenho dos modelos (Rezende; Marcacini; Moura, 2011).

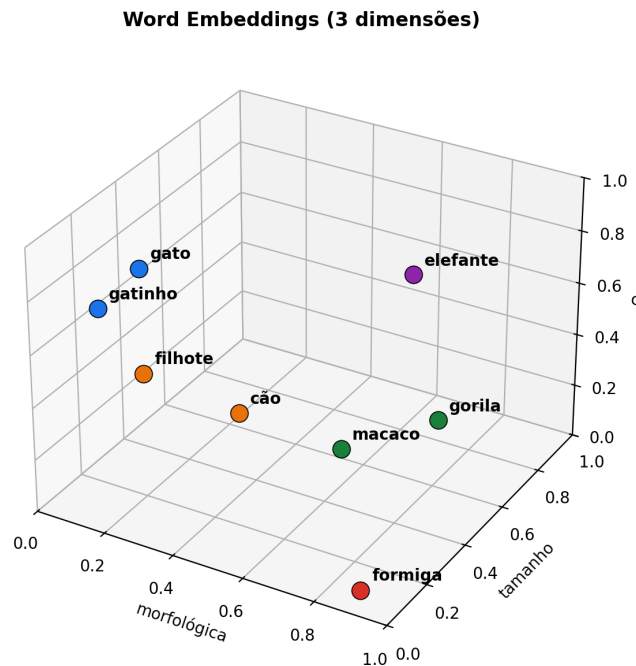
Entre as principais tarefas de pré-processamento no PLN, destacam-se:

- **Normalização:** Remoção de caracteres especiais, tags HTML/CSS, e transformação de letras maiúsculas em minúsculas.
- **Tokenização:** Separação do texto em unidades mínimas chamadas "tokens", que podem ser palavras, símbolos ou caracteres.
- **Stemização:** Redução das palavras ao seu radical, como transformar "meninas" em "menin".
- **Lematização:** Redução das palavras à sua forma base ou lema, como transformar "meninas" e "meninos" em "menino".
- **Remoção de Stopwords:** Exclusão de palavras frequentes e pouco informativas, como "a", "de", "e".
- **Correção Ortográfica:** Tratamento de erros de digitação e abreviações para evitar problemas nos modelos.

A representação vetorial de palavras é conhecida como *word embeddings*: a ideia é transformar palavras em números preservando suas relações semânticas. O processo parte de um *corpus* (conjunto de textos do qual as relações serão aprendidas). Primeiro o texto é quebrado em palavras; em seguida, cada palavra ganha um vetor cujo tamanho corresponde ao número de dimensões definidas pelo *corpus*. A Figura 04 traz uma ilustração com três dimensões (morfológica, tamanho e cor) representando um conceito completo.

Entre as vantagens do *word embedding* estão:

- **Representação sem contexto:** cada palavra é analisada independentemente do contexto imediato. Algumas nuances se perdem, mas a categorização permanece útil.
- **Relações semânticas:** em um espaço n-dimensional, palavras com significado próximo ficam vizinhas, como “macaco” e “gorila”.
- **Operações matemáticas:** vetores podem ser somados ou subtraídos para descobrir relações entre palavras. Adicionar um vetor associado a uma característica a outro vetor pode resultar em uma palavra relacionada.

Figura 04 – Exemplo da transformação de palavras em *word embeddings*.

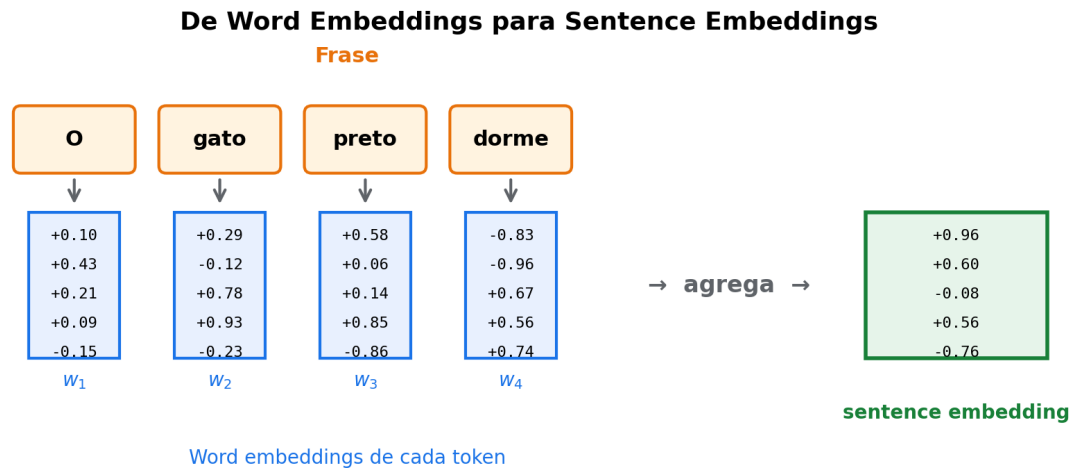
Fonte: Própria (2025).

No exemplo acima, as dimensões foram criadas a partir da intuição. Na prática, é mais comum usar Aprendizado Profundo: o modelo aprende as dimensões a partir do contexto em que cada palavra aparece. Quanto maior o *corpus*, melhor o resultado (Bengio et al., 2003).

Há ainda uma extensão importante: o *sentence embedding*, que considera não só as palavras individuais como também a ordem em que aparecem na frase (Conneau et al., 2017). Cada palavra é representada pelo seu próprio *word embedding*, e o conjunto forma a frase completa, com informação semântica e contextual preservada (Figura 05).

Há várias técnicas para representar palavras vetorialmente (*word embeddings*), e elas se diferenciam pela arquitetura e pela base de dados usada no treinamento (Martin; Jurafsky, 2009). Vale acrescentar que essas representações podem ser transferidas para domínios semelhantes via Transferência de Aprendizado, tema discutido em outra seção.

No contexto do PLN moderno, representações vetoriais como *word embeddings* e *sentence embeddings* têm sido amplamente utilizadas. Modelos como Word2Vec (Mikolov et al., 2013), GloVe (Pennington; Socher; Manning, 2014), fastText (Bojanowski et al., 2017) e os *embeddings* disponibilizados pela OpenAI destacam-se nesse campo, oferecendo desempenho robusto em diversas tarefas de PLN.

Figura 05 – Exemplo da transformação de *word embeddings* para *sentence embeddings*.

Fonte: Própria (2025).

2.6 Métricas de Similaridade Textual

Avaliar a saída de um LLM em tarefas de conversão de texto exige comparar o texto produzido com um texto de referência. Como a correspondência exata raramente é desejável (uma paráfrase correta também precisa contar como acerto), usa-se um conjunto de métricas de similaridade. Elas se dividem em quatro famílias: *lexicais*, *semânticas*, *de distância* e *voltadas para geração* (Jurafsky; Martin, 2009).

Lexicais. Comparam diretamente as cadeias de caracteres ou os *tokens* dos dois textos, ignorando o significado. A mais difundida é baseada na distância de Levenshtein, que mede o número mínimo de inserções, remoções e substituições necessárias para transformar uma cadeia em outra. A biblioteca *FuzzyWuzzy* oferece variantes dessa medida; uma delas é o *partial ratio*, útil quando uma das *strings* é fragmento da outra. Outra alternativa clássica é a similaridade do cosseno sobre vetores TF-IDF (*Term Frequency–Inverse Document Frequency*), que pondera cada termo pela sua importância relativa no corpus (Manning; Raghavan; Schütze, 2008).

Semânticas. Apoiam-se em *sentence embeddings*: cada texto vira um vetor em um espaço de alta dimensionalidade, treinado para colocar sentenças de significado próximo perto umas das outras. A similaridade vira o cosseno entre os dois vetores. Modelos como o *Sentence-BERT* (SBERT) (REIMERS; GUREVYCH, 2019) estendem o BERT para gerar essas representações com eficiência. Para o português, vale destacar o BERTimbau (Souza; Nogueira; Lotufo, 2020), especializado em PT-BR, com variantes ajustadas ao domínio jurídico. Modelos multilíngues,

como o *paraphrase-multilingual-mpnet-base*, permitem comparar textos em línguas diferentes em um mesmo espaço vetorial.

De distância. Medem o custo de transformar um texto em outro no espaço de *embeddings*. O *Word Mover's Distance* (WMD) (KUSNER et al., 2015) formaliza esse custo como um problema de transporte ótimo entre as palavras das duas sentenças, ponderado pela distância euclidiana entre seus *embeddings*. No português, o WMD pode rodar sobre *embeddings* do FastText (Bojanowski et al., 2017) ou sobre modelos do projeto NILC, treinados em corpora em PT-BR. Quando normalizado, o WMD aproxima-se de 1 para sentenças semanticamente próximas e tende a 0 quando estão distantes.

Voltadas para geração. Surgiram na literatura de tradução automática e sumarização, em que a forma exata da resposta pode variar bastante. O *BERTScore* (ZHANG et al., 2020) alinha cada *token* da saída ao *token* mais próximo da referência usando *embeddings* do BERT, e agrega o resultado em precisão, revocação e F1 contextuais. O *ROUGE-L* (LIN, 2004) calcula a razão entre a maior subsequência comum (*Longest Common Subsequence*) e o comprimento da referência, captando paralelismo sintático mesmo com inserções no meio.

As quatro famílias são complementares. As lexicais e o ROUGE-L punem reformulações que preservam o sentido; semânticas, de distância e BERTScore enxergam equivalência por trás da variação de superfície. Combinar as quatro fornece um retrato bem mais completo da qualidade das saídas de LLMs, prática consolidada na literatura de PLN aplicada (REIMERS; GUREVYCH, 2019).

2.7 Redes Neurais Artificiais

O sistema nervoso humano é formado por neurônios, células que transmitem as informações que sustentam o funcionamento, o comportamento e o raciocínio do corpo (Tafner; Xerez; Filho, 1995). As Redes Neurais Artificiais (RNAs) são técnicas computacionais inspiradas nessa estrutura biológica: aprendem por experiência, ajustando-se aos dados que recebem (Haykin, 1999). Vale ler a RNA como uma abstração computacional do neurônio biológico, usada para modelar processos de aprendizado e resolver problemas complexos.

A primeira simulação computacional de uma rede neural biológica veio em 1943, das mãos dos psiquiatras Warren McCulloch e Walter Pitts. O trabalho descreveu o comportamento e a memória dos neurônios biológicos em forma artificial, mas deixou de fora a parte do aprendizado (McCulloch; Pitts, 1943). Esse passo foi dado em 1949, por Donald Hebb, no primeiro estudo dedicado à fase de aprendizado: ele mostrou que o peso de cada conexão entre neurônios resulta em uma força de excitação análoga à observada nos neurônios biológicos (Hebb, 1949). O modelo, ainda assim, não reconhecia padrões.

A virada veio em 1958, quando Rosenblatt apresentou o primeiro modelo capaz de

detectar padrões e características: ajustando os pesos das conexões, o modelo aprendia a classificar (Rosenblatt, 1958). Esse modelo ficou conhecido como *perceptron* e organiza-se em três camadas:

- **Camada de entrada:** Responsável por receber informações do ambiente externo e repassá-las às camadas intermediárias.
- **Camadas intermediárias (ou ocultas):** Processam os sinais recebidos, ajustam os pesos e enviam as informações para a camada seguinte.
- **Camada de saída:** Produz e apresenta o resultado final.

O *perceptron*, porém, esbarra em um teto: Minsky e Papert (1969) demonstraram que um modelo de camada única não resolve problemas que não são linearmente separáveis, como o ou-exclusivo (XOR). A superação veio com o empilhamento de várias camadas intermediárias — o *Multi-layer Perceptron* (MLP) —, cujo treinamento só se tornou viável anos depois, com o algoritmo de retropropagação do erro (Rumelhart; Hinton; Williams, 1986).

O MLP é um conjunto de neurônios conectados em paralelo, cada conexão com um peso que mede sua influência na rede. A saída de um neurônio vira entrada de outro nas camadas intermediárias, e dessa interação em cadeia emerge a capacidade de reconhecer padrões e características (Soares; Teive, 2015).

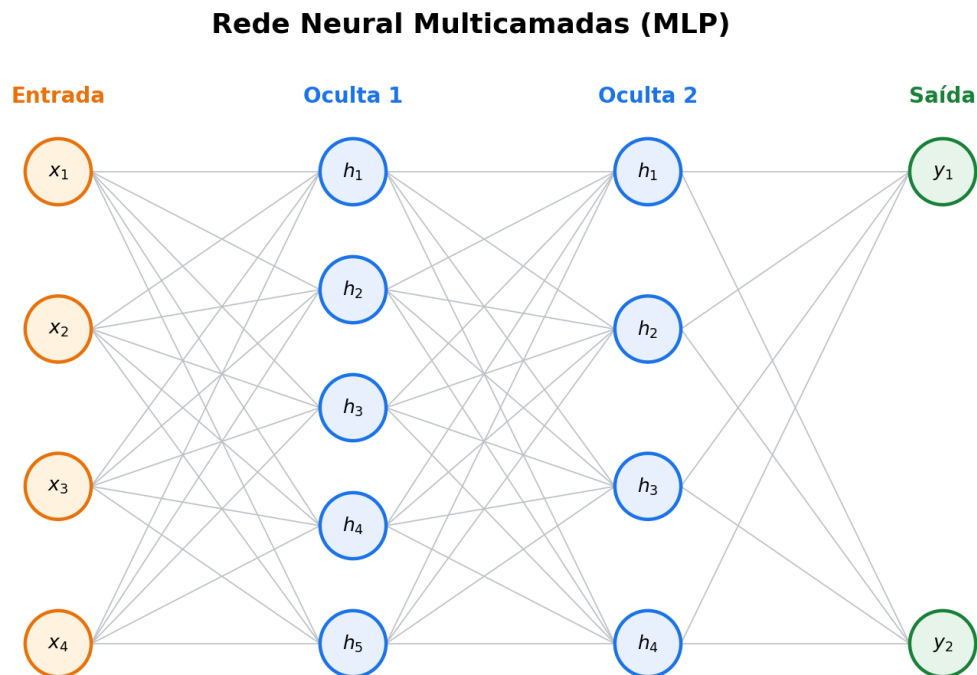
A estrutura de um neurônio em uma rede MLP processa informações recebidas do ambiente externo (valores x) associadas a pesos (w). Uma soma ponderada dos produtos de x e w é então calculada, somando-se o limiar de ativação (bias ou polarização). O valor resultante, chamado de ponto de ativação (u), é avaliado por uma função de ativação que limita a amplitude da saída do neurônio, geralmente dentro do intervalo $[0, 1]$, podendo ser interpretado como uma probabilidade (Goodfellow; Bengio; Courville, 2016).

A Figura 07 demonstra o processamento de informações por um neurônio em uma rede MLP, onde, dado um conjunto de valores ' x ' (que representam as informações do mundo externo), esses valores são associados a pesos ' w '. A partir disso, é efetuada uma soma ponderada dos produtos ' x ' e ' w ', à qual é somado o limiar de ativação (bias ou polarização), como mostrado na Equação 2.5.

$$u = \sum_{i=1}^n x_i \cdot w_i + \theta \quad (2.5)$$

O bias (θ) é responsável pelo deslocamento inicial da função de ativação no plano onde ela é representada. Caso o bias seja positivo, o movimento é realizado para a esquerda, diminuindo o valor do eixo x . Caso seja negativo, o movimento é realizado para a direita, aumentando o valor do eixo x (Rojas, 1996), como ilustrado na Figura 08.

Figura 06 – Rede Neural MLP.



Fonte: Própria (2025).

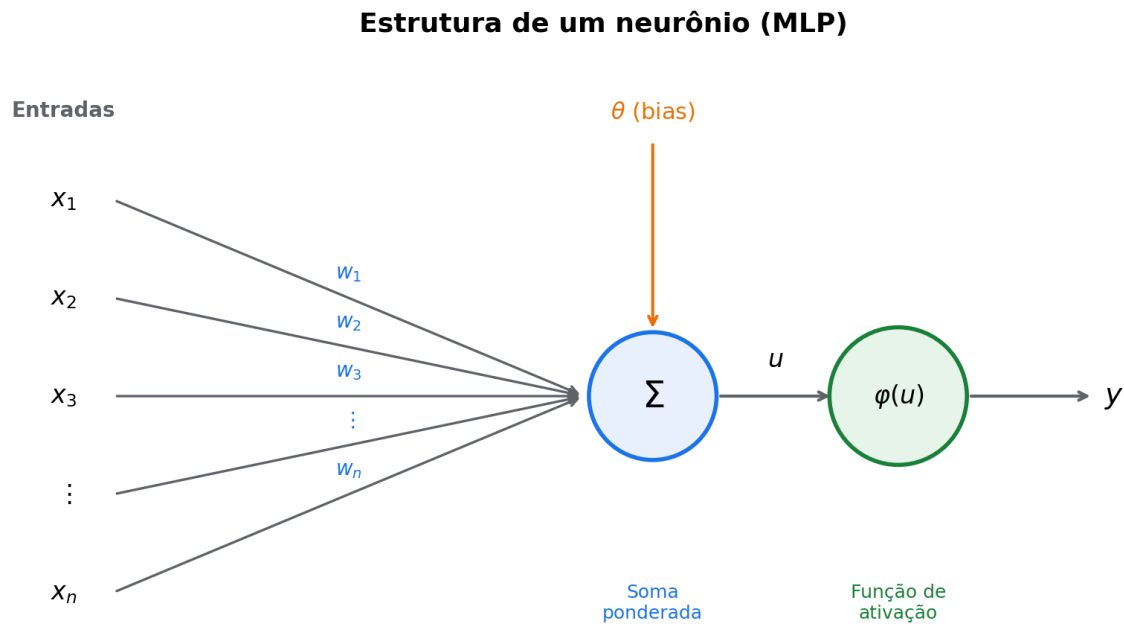
O valor u obtido, chamado de ponto de ativação, é avaliado pela função de ativação, que é responsável por calcular o sinal da saída do neurônio. A função de ativação, também conhecida como função de transferência, tem o objetivo de limitar a amplitude da saída do neurônio. Ou seja, o valor obtido estará no intervalo entre 0 e 1, podendo ser interpretado como uma probabilidade do resultado (Goodfellow; Bengio; Courville, 2016).

Existem diversos tipos de funções de ativação, e não há uma regra fixa para determinar qual é melhor ou pior. A escolha depende do problema, embora, dependendo das propriedades específicas, seja possível optar por uma função que facilite e acelere a convergência da rede neural (Goodfellow; Bengio; Courville, 2016).

As funções de ativação mais populares são:

- **Função de Etapa Binária** ($f(x) = 1$, se $x \geq 0$ e $f(x) = 0$, se $x < 0$): Essa função determina se o neurônio deve ou não estar ativo. Caso o valor ' u ' seja maior que um limite pré-determinado, o neurônio é ativado; caso contrário, ele permanece desativado. Embora útil em classificadores binários, é mais utilizada como referência teórica, pois, no processo de retropropagação (backpropagation), os gradientes das funções de ativação retornam zero, o que impede a melhoria dos resultados (Feng; Lu, 2019).
- **Função Linear** ($f(x) = ax$): cada camada passa por uma transformação linear. O problema

Figura 07 – Estrutura de um neurônio no MLP.

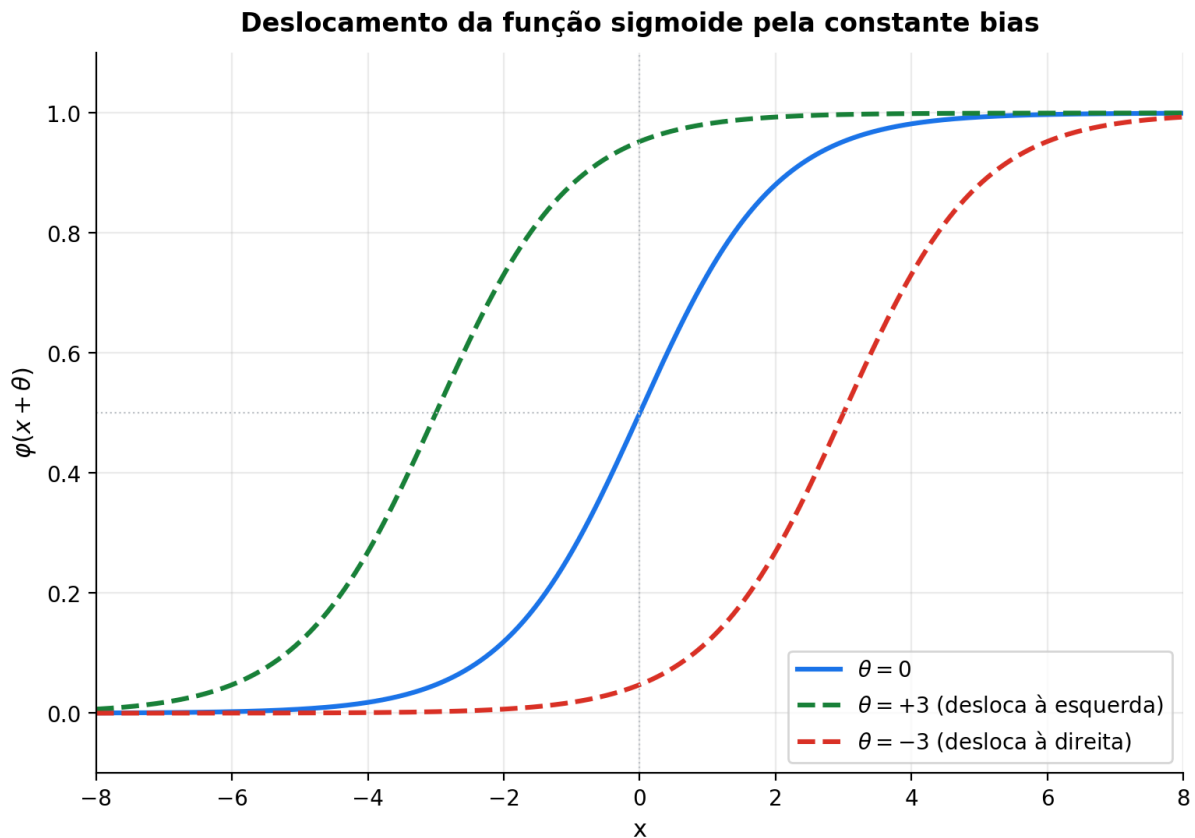


Fonte: Própria (2025).

é que essa função sofre da mesma limitação da etapa binária: a derivada é constante, independentemente do valor de entrada, o que impede aprimoramentos via *backpropagation* (Feng; Lu, 2019).

- **Função Sigmoid** ($f(x) = 1/(1 + e^{-x})$): Essa função comprime os valores de saída para o intervalo entre 0 e 1, o que é útil para classificar uma classe específica. Embora não seja linear e permita saídas não lineares em redes neurais, apresenta dois problemas principais: 1) os gradientes podem se aproximar de zero, o que prejudica o aprendizado da rede, e 2) os valores de saída são sempre positivos, o que pode não ser desejável em todos os casos. Apesar disso, ela é amplamente utilizada em classificadores (Feng; Lu, 2019).
- **Função Tanh** ($f(x) = 2/(1 + e^{(-2x)}) - 1$): É uma variação escalonada da função Sigmoid, mas simétrica em relação à origem, variando de -1 a 1. Isso elimina o problema dos valores positivos constantes. As demais propriedades são semelhantes às da Sigmoid (Feng; Lu, 2019).
- **Função ReLU** ($f(x) = \max(0, x)$): é a função de ativação mais usada no *design* de redes neurais. Como não é linear, facilita a propagação do erro e os aprimoramentos durante o treinamento. A principal vantagem é não ativar todos os neurônios ao mesmo tempo, o que torna a rede mais eficiente. O ponto fraco aparece quando os gradientes convergem para zero (Feng; Lu, 2019).

Figura 08 – Exemplo do deslocamento do gráfico através da mudança da constante bias.



Fonte: Própria (2025).

- **Função Leaky ReLU** ($f(x) = ax$, se $x < 0$ e $f(x) = x$, se $x \geq 0$): Uma variação da ReLU que corrige o problema do gradiente zero. Quando ' x ' < 0, é aplicada uma componente linear em vez de zero, garantindo um gradiente constante mesmo para valores negativos. É especialmente útil em camadas ocultas e para lidar com neurônios defeituosos (Feng; Lu, 2019).
- **Função Softmax**: É uma extensão da Sigmoid, projetada para problemas de classificação com múltiplas classes. Transforma as saídas de cada classe em valores entre 0 e 1, normalizando-as para que sua soma seja igual a 1. Assim, é possível interpretar os resultados como probabilidades de pertencimento a cada classe (Feng; Lu, 2019).

O treinamento de uma rede neural artificial multicamadas consiste em ajustar os pesos sinápticos e o bias de modo que as saídas se aproximem das saídas esperadas. Esse processo ocorre em duas etapas (RUSSELL; NORVIG, 2010):

- **Propagação (forward-propagation)**: Nesta etapa, aplica-se a Equação 2.5 em cada neurônio. O fluxo de dados começa na camada de entrada, atravessa as camadas ocultas e finaliza na camada de saída (Hafemann, 2014).

- **Retropropagação (backpropagation):** Após a etapa de propagação, os valores de saída são comparados com os valores desejados usando uma função custo, que mede o desempenho da rede. Caso o desempenho seja insatisfatório, inicia-se a retropropagação para minimizar o erro, ajustando os pesos e os bias. Isso é feito utilizando o método do Gradiente Descendente (Hafemann, 2014).

- **Função Custo (Loss):** Mede o quão distante está a saída da rede em relação à saída esperada. Para regressão, a função mais comum é o Erro Quadrático Médio (MSE). Para classificação, é a Entropia Cruzada (McCaffrey, 2013).
- **Erro Quadrático Médio (MSE):** Mede a diferença entre os valores previstos e os reais, elevando ao quadrado e calculando a média (Equação 2.6) (Willmott, 1981).

$$MSE = \frac{1}{n} \sum_{i=1}^n (\theta_{ip} - \theta_i)^2 \quad (2.6)$$

- **Raiz do Erro Quadrático Médio (RMSE):** Similar ao MSE, mas calcula a raiz quadrada da média dos erros (Equação 2.7) (Chai; Draxler, 2014).

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n e_i^2} \quad (2.7)$$

- **Erro Médio Absoluto (MAE):** Mede o desvio médio entre valores previstos e reais, com pesos iguais para todos os desvios (Equação 2.8) (Willmott, 1981).

$$MAE = \frac{\sum_{i=1}^n |O_i - P_i|}{n} \quad (2.8)$$

- **Entropia Cruzada:** Mede a distância entre os vetores de saída e os valores esperados (Equação 2.9) (Nasr; Badr; Joun, 2002).

$$E(w) = \frac{1}{n} \cdot \sum_{i=1}^n [t_i \cdot \log(y_i) + (1 - t_i) \cdot \log(1 - y_i)] \quad (2.9)$$

- **Gradiente Descendente:** Método de otimização que ajusta pesos e bias na direção que minimiza o erro. A cada iteração, busca-se o mínimo local da função custo (Equação 2.10) (Rumelhart; Hinton; Williams, 1986).

$$(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \partial_{(\mathbf{w}, b)} l^{(i)}(\mathbf{w}, b) \quad (2.10)$$

Cada ciclo de propagação e retropropagação ajusta os pesos para reduzir a função de perda, repetindo o processo até atingir o erro mínimo local (RUSSELL; NORVIG, 2010).

Um dos ramos do Aprendizado de Máquina baseado em Redes Neurais Artificiais é o Aprendizado Profundo (ou Deep Learning), que utiliza diversas camadas capazes de

extrair características em diferentes níveis. Por exemplo, em um processamento de imagem, a primeira camada pode extrair formas gerais, como objetos ou rostos humanos, enquanto camadas posteriores extraem características mais específicas, como bordas e cantos. Ou seja, as camadas iniciais extraem recursos de baixo nível, enquanto as camadas mais profundas extraem recursos de alto nível e/ou médio nível (Schmidhuber, 2015).

Um dos ganhos centrais da Aprendizagem Profunda é a extração automática de características: pesos adaptáveis no *kernel* dispensam o pré-processamento manual (Schmidhuber, 2015). Em termos de topologia, uma RNA combina três tipos de camadas e os nós (ou nodos) podem ter dois tipos de conexão:

- **Redes de camada única:** ocorre quando há apenas um nó entre qualquer entrada e saída da rede (Figura 09 (a) e (c)) (Braga; Ludermir; Carvalho, 2000).
- **Redes de múltiplas camadas:** ocorre quando há mais de um neurônio entre qualquer entrada e saída da rede (Figura 09 (b) e (d)) (Braga; Ludermir; Carvalho, 2000).
- **Redes recorrentes:** ocorre quando existe pelo menos um laço de realimentação em alguma camada (Figura 09 (c) e (d)) (Braga; Ludermir; Carvalho, 2000).
- **Nó acíclico (feedforward):** ocorre quando a saída de um neurônio não é utilizada como entrada em camadas anteriores ou iguais (Figura 09 (a) e (b)) (Haykin, 2001).
- **Nó cíclico (feedback):** ocorre quando a saída de um neurônio é utilizada como entrada em camadas anteriores ou iguais (Figura 09 (c) e (d)) (Haykin, 2001).

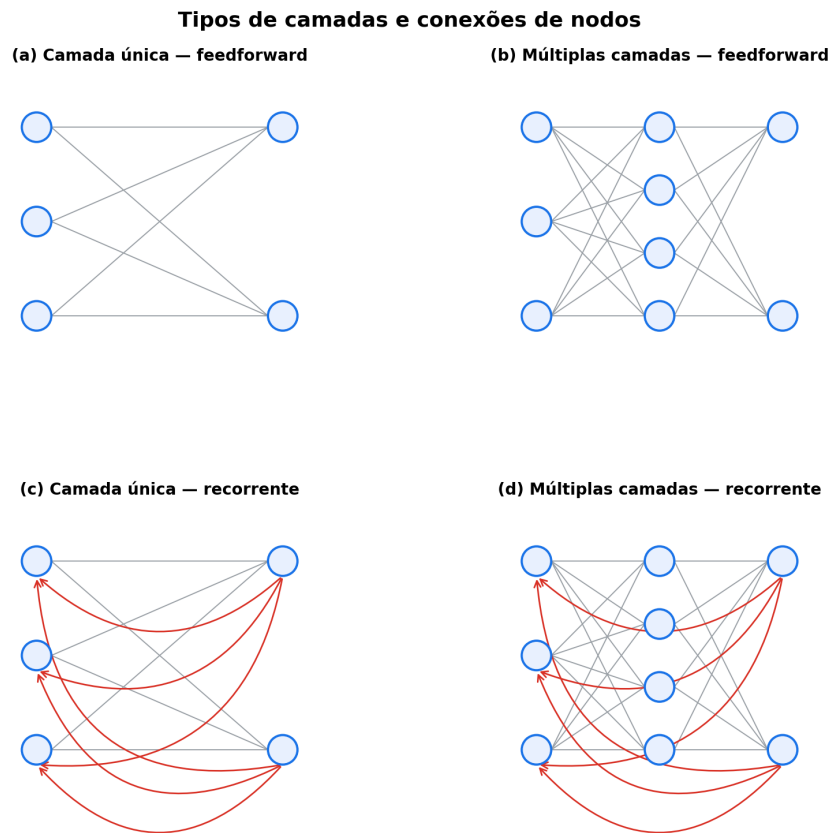
A diferença entre Redes Neurais Recorrentes (RNN) e Redes Neurais Feedforward (FNN) está justamente nos ciclos: nas RNNs, a saída pode realimentar camadas anteriores. Com isso, o modelo passa a capturar dependências temporais entre os dados (Goodfellow; Bengio; Courville, 2016).

Esse ciclo dá à RNN um estado interno (uma espécie de memória) que processa entradas sequenciais, característica útil em reconhecimento de fala, PLN, previsão de séries temporais e correlatos.

Há um preço a pagar. A repetição de operações no ciclo da RNN, somada à matriz de pesos W de cada camada, torna a rede progressivamente profunda. Isso abre espaço para dois problemas conhecidos: dissipação do gradiente (*gradient vanishing*) e explosão do gradiente (*gradient explosion*) (Goodfellow; Bengio; Courville, 2016).

A dissipação acontece quando o erro das últimas camadas pouco afeta as camadas iniciais: na atualização, pesos e *bias* das primeiras camadas mal se mexem, e o treinamento estanca. A explosão é o oposto: os gradientes das camadas iniciais ficam tão grandes que desestabilizam toda a rede (Goodfellow; Bengio; Courville, 2016).

Figura 09 – Tipos de camadas e conexões de nodos.



Fonte: Própria (2025).

A resposta veio em 1997, com Hochreiter e Schmidhuber: o LSTM (*Long Short-Term Memory*). A ideia foi substituir o neurônio tradicional por uma célula de memória, capaz de regular o fluxo de informação e contornar tanto a dissipação quanto a explosão do gradiente (Hochreiter; Schmidhuber, 1997).

2.8 Memória de Longo e Curto Prazo (LSTM)

O LSTM é uma evolução da RNN. Seu funcionamento se organiza em cadeia: cada camada representa um intervalo de tempo t , e a rede contém células de memória, cada uma com três portões (entrada, esquecimento e saída).

- **Portão de entrada (*input gate*)**: a partir do fluxo de dados que chega, atualiza o estado de memória da célula (Hochreiter; Schmidhuber, 1997).
- **Portão de esquecimento (*forget gate*)**: decide qual informação permanece na memória (Hochreiter; Schmidhuber, 1997).

- **Portão de saída (*output gate*):** define como a saída da célula é influenciada pela entrada atual e pela memória armazenada (Hochreiter; Schmidhuber, 1997).

Segundo Tai, Socher e Manning (2015), o primeiro passo dentro da célula LSTM é decidir o que descartar e o que manter no estado atual. Esse cálculo cabe ao portão de esquecimento, que retorna um valor entre 0 (esquecer) e 1 (manter), aplicado pela função *sigmoid* (Equação 2.11).

$$f_t = \text{sigmoid}(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (2.11)$$

Em seguida, decide-se qual nova informação será armazenada. Essa decisão ocorre em duas etapas: a primeira determina qual valor deverá ser atualizado (calculada pela função *sigmoid*) (Equação 2.12), e a segunda cria um vetor de novos valores candidatos que podem ser adicionados ao estado (calculada pela função *tanh*) (Equação 2.13).

$$i_t = \text{sigmoid}(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2.12)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (2.13)$$

Depois disso, multiplica-se o estado antigo pelos candidatos passados (C_{t-1}), para esquecer as informações desnecessárias, e multiplica-se o estado atualizado pelos candidatos atuais ($\tilde{C}_t$), para manter as informações úteis (Equação 2.14).

$$C_t = f_t \times C_{t-1} + i_t \times \tilde{C}_t \quad (2.14)$$

No passo final, a camada *sigmoid* decide quais informações da célula irão para a saída (Equação 2.15); essas informações são multiplicadas pela função *tanh* aplicada aos candidatos atuais, deixando passar apenas o que é relevante (Equação 2.16).

$$o_t = \text{sigmoid}(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (2.15)$$

$$h_t = o_t \times \tanh(C_t) \quad (2.16)$$

A LSTM tem uma estrutura computacional pesada: opera ativando diferentes funções para manter, alterar ou descartar informação. Apesar do custo, é uma rede que se mostrou bem-sucedida em tarefas sequenciais (Tai; Socher; Manning, 2015). Desse sucesso surgiu uma variante: a *Bidirectional LSTM* (BiLSTM), formada por duas LSTMs em paralelo: uma percorre a sequência para frente, outra para trás. Com isso, a rede passa a capturar informação passada e futura, ampliando seu poder de aprendizado (Graves; Jaitly; Mohamed, 2013).

RNNs e LSTMs lidam bem com sequências e foram projetadas para tarefas *sequence-to-sequence* (*seq2seq*), em que se recebe uma sequência de dados e se produz outra. No PLN, porém, a estrutura neural que vem dominando o cenário é o *transformer*, em razão do desempenho que entrega em tradução, sumarização, legendagem de imagens e tarefas correlatas.

2.9 Transformers

Por anos, redes neurais recorrentes (Goodfellow; Bengio; Courville, 2016), LSTMs (Hochreiter; Schmidhuber, 1997) e RNNs com portas (Chung et al., 2014) foram o estado da arte em modelagem de sequência. A premissa do *seq2seq* é simples: receber uma sequência de dados em um domínio e devolver outra sequência em um domínio diferente, usando para isso um modelo de AM ou de Aprendizado Profundo (Sutskever; Vinyals; Le, 2014).

A arquitetura típica tem dois módulos: o *encoder* (codificador) processa a sequência de entrada e a transforma em um contexto; o *decoder* (decodificador) recebe esse contexto e produz a sequência de saída (Sutskever; Vinyals; Le, 2014). Um exemplo concreto é a tradução inglês-português: o *encoder* comprime o texto em inglês em um “contexto livre de idioma”, e o *decoder* usa esse contexto para gerar a versão em português.

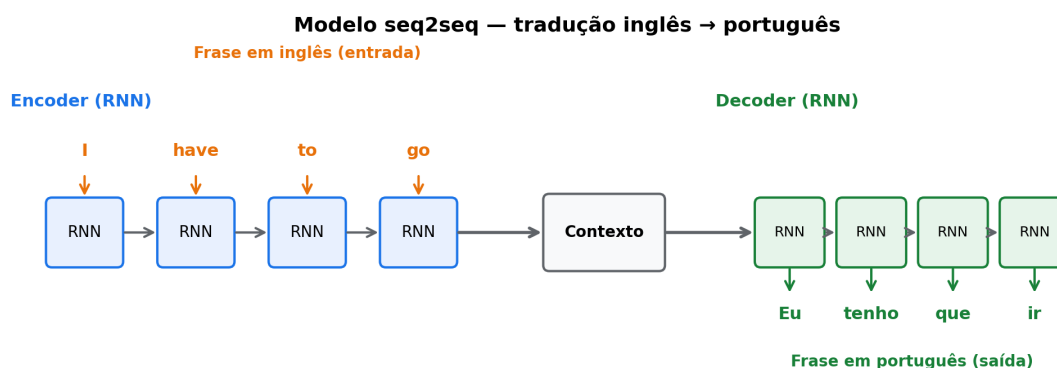
No exemplo de um modelo de tradução inglês-português, será demonstrado o *encoder* e o *decoder* na forma de RNNs (Redes Neurais Recorrentes), onde o *encoder* processa as palavras da sequência de entrada uma a uma (de forma recorrente). A cada etapa do processamento, a palavra que entra na rede produz uma saída e um estado oculto (contexto). A saída geralmente do *encoder* é descartada, enquanto o estado oculto é passado para a próxima etapa, sendo utilizado nas palavras seguintes da sequência para produzir mais uma saída e criando um novo estado oculto (ou seja, atualizando o contexto). Esse processo se repete até a última palavra da sequência de entrada, onde o *encoder* cria o estado oculto final (contexto final).

Esse contexto final é passado para a segunda RNN, o *decoder*, que também recebe um “vetor genérico”(os valores são ajustados durante o treinamento). Esse vetor representa o início da frase de saída e, da mesma forma que o *encoder*, o *decoder* utiliza o vetor e o contexto para produzir uma saída e um estado atualizado, passando por etapas (nesse exemplo, palavras), e, em cada etapa, a saída representa a palavra traduzida até chegar na última etapa (palavra), que irá entregar a frase inteira traduzida (Figura 10).

Não é necessário, entretanto, utilizar apenas RNNs para codificadores e decodificadores. É possível combinar diversos modelos, como demonstra Mun, Cho e Han (2016), que utiliza um codificador CNN (*Convolutional Neural Networks*) (Lecun et al., 1998), um codificador RNN e um decodificador LSTM em um modelo que gera legendas de imagens.

Como pode ser visto na Figura 10, a frase original tem 7 palavras, enquanto a frase traduzida contém 5 palavras. Por isso, dependendo da complexidade do problema, e no caso de

Figura 10 – Exemplo de um modelo seq2seq para tradução do inglês para português.



Fonte: Própria (2025).

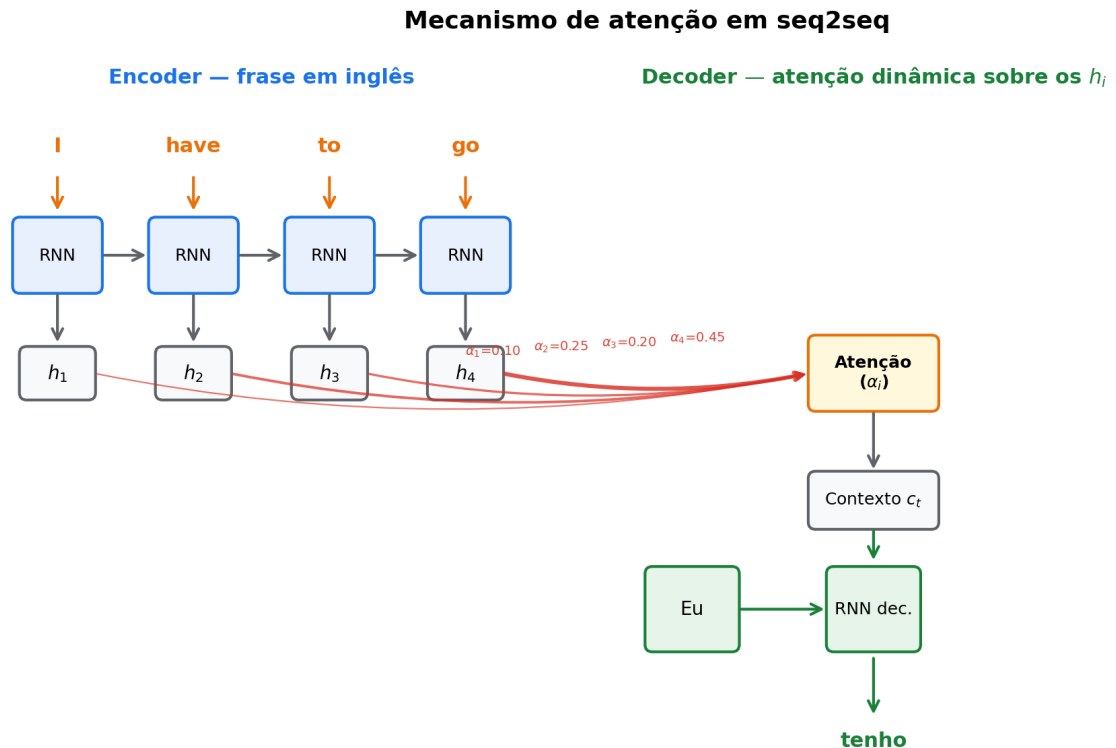
traduções, a tarefa de traduzir não se trata de um simples alinhamento das palavras individuais. É necessário que o modelo seja capaz de focar em uma palavra, caso seja necessário, ou em um trecho específico da frase, criando assim o conceito de atenção.

Nas RNNs, o mecanismo de atenção geralmente é implementado da seguinte forma: no *encoder*, além do contexto ser representado pelo estado oculto final, ele também é representado por todos os estados ocultos produzidos após o processamento de cada palavra, fazendo com que cada estado oculto contenha o contexto acumulado até aquela palavra, o qual, após processar todas as palavras, é enviado para o *decoder* juntamente com todos os estados ocultos (Bahdanau; Cho; Bengio, 2015). Ou seja, no exemplo da Figura 10, a palavra 'go' após ser processada, possuirá o estado oculto (contexto) acumulado das palavras 'I', 'have', 'to' e 'go'.

O *decoder* recebe e extrai do estado oculto final as informações necessárias para produzir o próximo estado oculto e utiliza o acúmulo de estados ocultos anteriores para calcular uma pontuação de atenção, indicando quanto foco a etapa dará para esses contextos parciais. O resultado da pontuação de atenção é multiplicado por cada estado oculto respectivo, e depois todos são somados, gerando o resultado denominado estado de contexto (Figura 11). Esse estado de contexto se espera que carregue a informação de interesse de forma amplificada, ou seja, indicando qual parte da frase o modelo deve prestar mais atenção. No fechamento da etapa, o estado de contexto é concatenado com o estado final produzido nessa rodada do *decoder*; o vetor resultante passa por uma rede adicional totalmente conectada, e dela sai o vetor que representa a palavra no segundo idioma (português) (Bahdanau; Cho; Bengio, 2015).

De acordo com [Bahdanau, Cho e Bengio \(2015\)](#), o mecanismo de atenção obtém melhores resultados na maioria das tarefas que necessitam “traduzir” uma sequência em outra, podendo

Figura 11 – Exemplo de um modelo com o mecanismo de atenção para tradução do inglês para português.



Fonte: Própria (2025).

transformar um texto em seu resumo ou uma imagem em sua legenda. Porém, modelos recorrentes, mesmo utilizando o mecanismo de atenção (*Attention*), dada a sua natureza sequencial, acabam realizando processos computacionais intensivos para captar o contexto de uma sequência. Por isso, foi criada a arquitetura *Transformer*, uma técnica que proporciona melhorias significativas tanto na eficiência computacional quanto na performance dos modelos baseados em arquiteturas recorrentes.

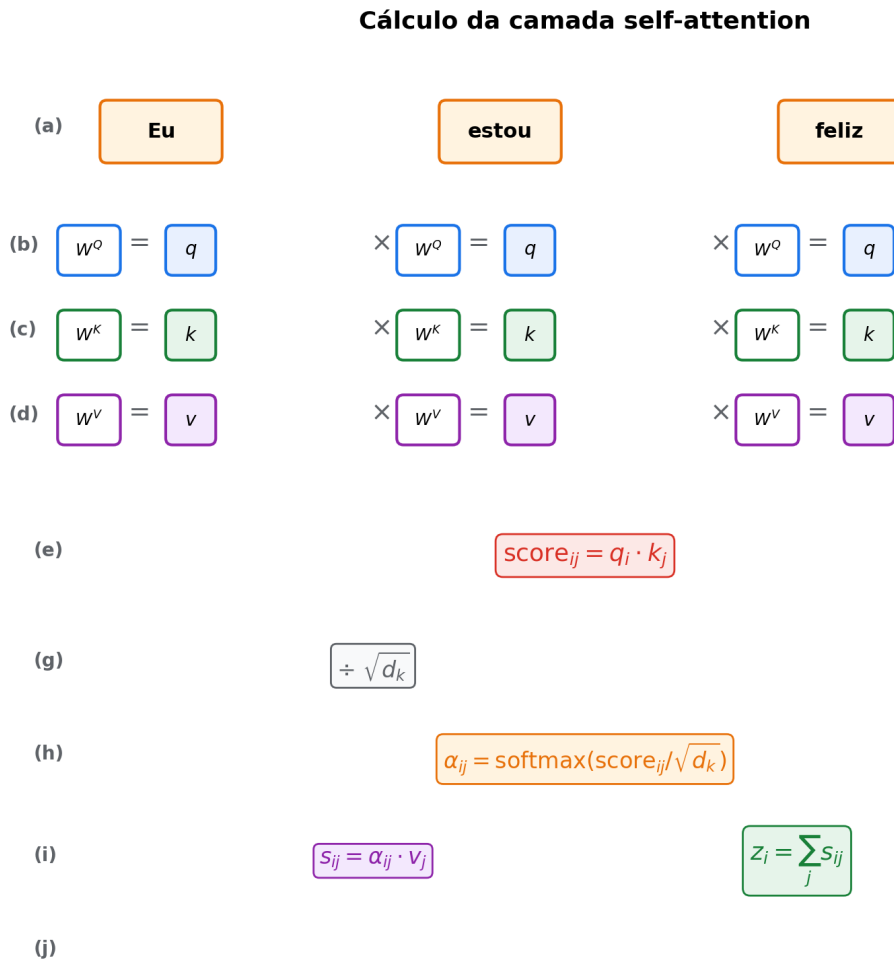
Assim como outras redes com mecanismo de atenção, o *transformer* também possui os módulos *encoder* e *decoder*, onde os criadores do modelo criaram 6 pilhas (*stacks*) de *encoders* e *decoders* individuais, arranjados linearmente. Cada *encoder* de cada pilha possui duas unidades (camadas) chamadas de *self-attention*, uma aplicando o mecanismo de atenção sobre o texto que ela recebe e outra que transforma o resultado da camada de *self-attention* em um *embedding* de comprimento menor (Vaswani et al., 2017).

O *decoder* é construído com as mesmas unidades do *encoder*, porém, possui uma camada adicional denominada *encoder-decoder attention*, cujo objetivo é mapear o resultado da camada de *self-attention* do *decoder* com os *embeddings* construídos pelo *encoder* (Vaswani et al., 2017). Com isso, enquanto nos modelos *seq2seq* as palavras são processadas sequencialmente, o

transformer recebe o texto que deve processar tudo de uma vez, onde, através de um método independente, todas as palavras são convertidas em *embeddings*.

A camada *self-attention* (Figura 12) converte cada palavra em três vetores, denominados q (*query*), k (*key*) e v (*value*), onde é feita a multiplicação dos vetores da palavra por três matrizes (pesos) W^q , W^k e W^v , respectivamente (Figura 12 (b), (c) e (d)), cujos parâmetros são ajustados durante o treinamento da rede. Esses vetores (q , k e v) são abstrações para permitir o cálculo de atenção, que é realizado por meio de uma multiplicação entre as *queries* e as *keys*. Essa operação irá resultar em uma pontuação que será normalizada e aplicada às *values*, como mostrado na Figura 12 (e). O resultado final de uma camada de *self-attention* é uma nova sequência que leva em consideração o foco nas palavras mais importantes de toda a sequência.

Figura 12 – Exemplo do cálculo de uma camada de *self-attention*.



Fonte: Própria (2025).

Essa pontuação é dividida pela raiz quadrada do comprimento dos vetores k ($\sqrt{d^k}$) para garantir maior estabilidade dos gradientes (Figura 12 (g)). Com isso, o resultado de cada par de palavras é submetido à transformação *softmax*, que gera valores no intervalo entre 0 e 1, sendo

que a soma de todos os valores é igual a 1 (Figura 12 (h)). Esse resultado representa a “atenção” que a rede deve dar a cada par de palavras. Em seguida, o valor do *softmax* é multiplicado pelo vetor v das palavras correspondentes (ou seja, a palavra com a qual a palavra-foco faz par), resultando em vetores s (Figura 12 (i)), que são somados, produzindo, finalmente, um vetor z para a palavra-foco (Figura 12 (j)), que incorpora as relações entre a palavra-foco e todas as demais palavras da sequência, com a devida atenção considerada (Vaswani et al., 2017).

Todos os valores da Figura 12 são ilustrativos; porém, os vetores s e z estão em diferentes tons para demonstrar que, após a multiplicação dos valores pelo *softmax*, o valor de uma palavra terá maior impacto que o de outra, fazendo com que o vetor z de uma tenha maior peso que o de seu par (neste exemplo, o z_1), demonstrando a relação entre as palavras.

O módulo *encoder* possui um conjunto de matrizes W^q , Q^k e W^v , e esse conjunto é denominado de *attention head* (ou cabeça de atenção). A quantidade de *attention heads* é a raiz quadrada do comprimento dos vetores k ($\sqrt{d^k}$). Com isso, o *transformer* realiza o mecanismo de atenção mais de uma vez, resultando, no final da camada de *self-attention*, em $n(\sqrt{d^k})$ vetores z para cada texto, fazendo com que haja atenção em diversos dados (Popel; Bojar, 2018). Esse processo é similar ao que ocorre com o ser humano, que, por exemplo, ao ler um livro, dá atenção a diversas partes do texto ao mesmo tempo, como o nome de um personagem, o ambiente em que se situa, entre outros, e o *transformer* funciona de maneira análoga.

Na etapa final, todos os vetores z (*attention heads*) são concatenados e multiplicados pela matriz de peso W^o , também ajustada durante o treinamento. Cada palavra acaba representada por um vetor z de mesmo tamanho do vetor v ; esses vetores passam por uma camada *feed-forward*, que produz os *embeddings* finais enviados do *encoder* para o *decoder*.

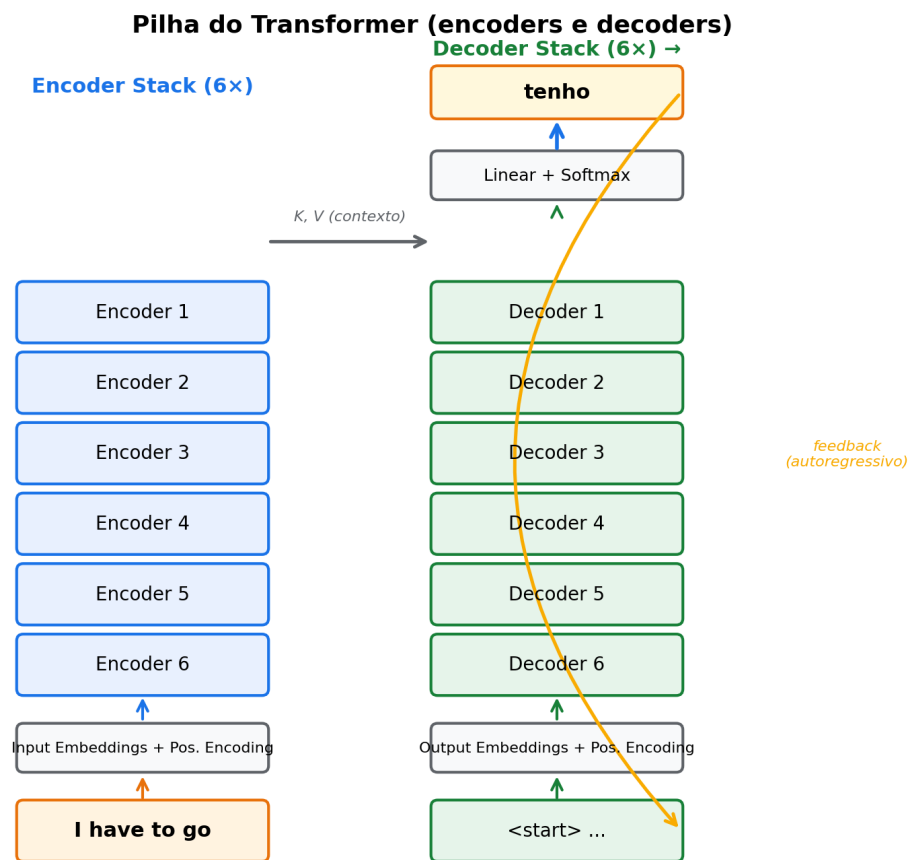
O *decoder* recebe os *embeddings* e os transforma em matrizes de atenção K e V , que serão usadas nas camadas *encoder-decoder attention*. Essa camada aplica o *self-attention* sobre o primeiro vetor recebido (no exemplo da tradução, o início da sequência traduzida), com o mesmo processamento da camada *self-attention* do *encoder*. A diferença está nas matrizes: aqui, em vez de W^k e W^v , usam-se as matrizes K e V , que são produto do processamento da sequência de entrada.

O resultado da pilha que forma o *decoder* é, da mesma forma que no *encoder*, um *embedding* de comprimento igual ao *embedding* que representa o texto original. Esse *embedding* passa por uma camada linear, convertendo-o em um vetor que, no exemplo de tradução, tem comprimento igual ao do vocabulário do segundo idioma (português), resultando na representação de cada uma das palavras traduzidas.

Esse vetor é ativado pela função *softmax*, resultando em um vetor de probabilidades, e o valor com maior probabilidade indica a palavra que o modelo deve gerar, produzindo, finalmente, uma saída (Figura 13 (seta azul)). Ao retornar essa saída como resultado, a palavra volta para o início do *decoder* (Figura 13 (seta amarela)), para ser utilizada na previsão da próxima palavra,

até atingir a última, gerando um *token* que indica que o texto terminou.

Figura 13 – Representação da pilha (*stack*) do *transformer*.



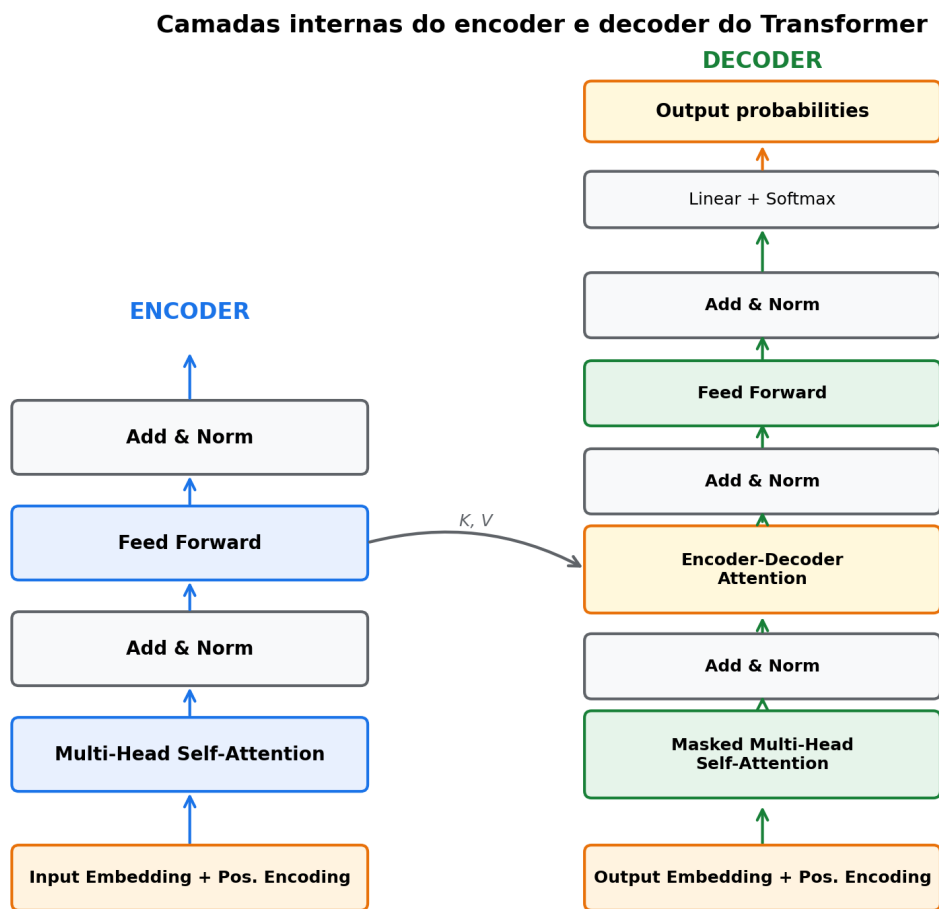
Fonte: Adaptado de (Alammar, 2018).

Por essas razões, o *transformer* acabou ultrapassando RNNs e LSTMs em uma série de tarefas. As palavras não precisam entrar na rede na ordem original: todas são processadas ao mesmo tempo, o que abre espaço para paralelização e ganho de velocidade. O mecanismo de atenção, por sua vez, captura relações entre palavras distantes na sequência, ajudando a precisão (Popel; Bojar, 2018).

Há ainda dois detalhes da arquitetura: depois de cada unidade de atenção e *feed-forward* existem camadas de adição e normalização (Popel; Bojar, 2018); e cada *embedding* recebe um *positional encoding*, que reintroduz a ordem das palavras; sem ele, o modelo trataria a sequência como um saco de palavras (Vaswani et al., 2017). A Figura 14 mostra a configuração de uma pilha do *encoder* e do *decoder*.

2.10 Modelos de Linguagem de Grande Porte (LLMs)

Grandes Modelos de Linguagem (*Large Language Models*, LLMs) são modelos de IA que usam AM para entender e gerar linguagem humana. Apoiam-se em redes neurais profundas

Figura 14 – Camadas do *encoder* e *decoder* do *transformer*.

Fonte: Adaptado de (Alammar, 2018).

e em técnicas de PLN para computar suas respostas (TOUVRON et al., 2023).

O treinamento de um LLM costuma seguir o paradigma de aprendizado não supervisionado: o modelo é exposto a bilhões de palavras e frases e aprende sobre a linguagem a partir desse volume bruto (TOUVRON et al., 2023).

À medida que o modelo aprende os padrões de associação entre palavras, ganha capacidade de prever a estrutura de frases com base em probabilidade. O resultado é um modelo capaz de capturar relações complexas entre palavras e frases (TOUVRON et al., 2023).

A escala que sustenta esses modelos vem com um custo: o volume de dados e a quantidade de cálculos probabilísticos exigem hardware especializado. GPUs (unidades de processamento gráfico) entram no jogo justamente por processarem cargas pesadas em paralelo, o que as torna o hardware natural para LLMs (STRUBELL; GANESH; MCCALLUM, 2019).

A arquitetura por trás do desempenho desses modelos é o *Transformer*: seus parâmetros absorvem as dependências e relações contextuais entre os elementos da sequência, e definem os

limites a partir dos quais é possível analisar volumes massivos de dados (NAVEED et al., 2024).

O lançamento do ChatGPT e do GPT-4 (*Generative Pre-trained Transformer*), pela OpenAI, virou a chave da percepção sobre LLMs (OPENAI et al., 2024). Apesar do desempenho notável, a OpenAI optou por não publicar detalhes da estratégia de treinamento nem os pesos do modelo.

Como resposta a esse fechamento, a Meta lançou o LLaMA (*Large Language Model Meta AI*), de código aberto, em tamanhos que vão de 7 a 405 bilhões de parâmetros. Segundo os autores, o LLaMA atinge desempenho equivalente ao dos principais modelos (caso do GPT-4) e fica próximo do estado da arte em uma variedade de tarefas (TOUVRON et al., 2023). O treinamento envolve duas etapas principais:

- **Pré-treinamento:** no qual o modelo é treinado em grande escala usando tarefas simples como a previsão da próxima palavra (AI@META, 2024).
- **Pós-treinamento:** no qual o modelo é ajustado para seguir instruções, alinhar-se às preferências humanas e melhorar capacidades específicas (como a codificação) (AI@META, 2024).

O modelo LLaMA, atualmente na versão 3.3 (2025), suporta nativamente multilinguismo, codificação e raciocínio. Seu maior modelo Transformer possui 405B de parâmetros e processa informações em uma janela de contexto de até 128k tokens (AI@META, 2024).

Para obter esses resultados, os autores afirmam que um grande modelo de linguagem de alta qualidade precisa de:

- **Dados:** Qualidade e quantidade de dados utilizados tanto no pré-treinamento quanto no pós-treinamento são essenciais. Foi utilizado um corpus de cerca de 15T de tokens multilíngues (AI@META, 2024).
- **Escala:** O modelo foi pré-treinado utilizando $3,8 \times 10^{25}$ FLOPs no pré-treinamento. FLOPs ou Operações de Ponto Flutuante representam o número de cálculos de ponto flutuante (adições, multiplicações, etc.) realizados durante o treinamento. A estimativa de FLOPs foi desenvolvida pela OpenAI: $FLOPs = 6ND$, onde N são parâmetros e D são tokens. O LLaMA 3 possui 400B de parâmetros, resultando em $6 \times 400 \times 15T = 3,8 \times 10^{25}$ (AI@META, 2024).
- **Gerenciamento da complexidade:** Maximizar a capacidade de escalar o processo de desenvolvimento do modelo utilizando o modelo Transformer padrão (Vaswani et al., 2017), além de adotar um procedimento de pós-treinamento simples com base em ajuste fino supervisionado (SFT), amostragem de rejeição (RS) e otimização de preferência direta (DPO) (RAFAILOV et al., 2024).

Para a criação do modelo LLaMA 3, o corpus multilíngue foi convertido em tokens e pré-treinado para executar a previsão do próximo token. Nesse estágio, o modelo aprende a estrutura da linguagem (AI@META, 2024).

O pré-treinamento é realizado em grande escala, utilizando um modelo de 405B de parâmetros em 15,6T de tokens, com uma janela de contexto inicial de 8k tokens. Esse processo é sequencial, aumentando progressivamente a janela de contexto até atingir 128k tokens. O modelo utiliza o algoritmo AdamW com uma taxa de aprendizado de pico de 8×10^{-5} , aquecimento linear de 8000 passos e uma programação de taxa de aprendizado decrescente em cosseno até 8×10^{-7} (AI@META, 2024).

Os dados do pré-treinamento incluem informações até o final de 2023 obtidas na web. Foram utilizados métodos de deduplicação e mecanismos de limpeza de dados em cada fonte para garantir tokens de alta qualidade, além de remover domínios com conteúdo adulto e informações pessoalmente identificáveis (AI@META, 2024).

Um classificador foi desenvolvido para categorizar os tipos de informações contidas nos dados, permitindo a inclusão de categorias específicas. Após vários testes, foi estabelecida a seguinte distribuição de tokens: 50% de conhecimento geral, 25% matemáticos e de raciocínio, 17% de código e 8% multilíngues (AI@META, 2024).

No estágio final do pré-treinamento, sequências longas foram utilizadas para dar suporte às janelas de contexto de até 128k tokens. O treinamento em sequências longas foi evitado inicialmente devido ao aumento quadrático na complexidade computacional das camadas de autoatenção. O comprimento do contexto foi aumentado gradualmente até que o modelo se adaptasse com sucesso (AI@META, 2024).

Este processo foi realizado em seis estágios, utilizando aproximadamente 800B de *tokens* de treinamento. Assim como a OpenAI relatou, a Meta também observou que o recozimento melhorou o desempenho dos modelos em até 24%, embora em algumas versões a melhoria tenha sido insignificante. Os autores afirmam que modelos com forte capacidade de aprendizado e raciocínio em contexto não necessitam de amostras específicas de treinamento em domínios para obter bom desempenho (AI@META, 2024).

O LLaMA 3 utiliza a arquitetura Transformer densa e padrão (Vaswani et al., 2017), com atenção de consulta agrupada (AINSLIE et al., 2023) e oito cabeças de chave-valor para melhorar a velocidade de inferência e reduzir o tamanho dos caches durante a decodificação. O modelo também emprega uma máscara de atenção que impede a autoatenção entre diferentes documentos dentro da mesma sequência.

O modelo LLaMA 3 de 405B de parâmetros possui uma arquitetura com 126 camadas e 128 cabeças de atenção, atingindo o tamanho computacionalmente ótimo de acordo com as leis de escala para o orçamento de treinamento da Meta. Essas leis determinam o tamanho ideal do modelo conforme o orçamento computacional (AI@META, 2024).

Após o modelo adquirir uma rica compreensão da linguagem, mas ainda não ser capaz de seguir instruções, inicia-se o pós-treinamento. Este estágio utiliza um modelo de recompensa sobre o ponto de verificação pré-treinado com dados de preferência anotados por humanos. Em seguida, os pontos de verificação são ajustados com Ajuste Fino Supervisionado (SFT) em dados de ajuste de instrução e Otimização de Preferência Direta (DPO) (AI@META, 2024).

O Ajuste Fino Supervisionado (SFT) é uma etapa de treinamento em modelos AM, especialmente usada em LLMs. Essa etapa ocorre após o treinamento inicial (pré-treinamento) do modelo em um grande conjunto de dados não supervisionados. No SFT, o modelo é ajustado usando um conjunto de dados rotulado e supervisionado, onde há exemplos de entrada e saída esperada. O objetivo é alinhar o modelo com tarefas específicas, reduzindo erros e melhorando seu desempenho em problemas delimitados.

A Otimização de Preferência Direta (DPO) é uma técnica emergente que busca alinhar os modelos diretamente com preferências humanas ou critérios de qualidade especificados. Em vez de ajustar o modelo com base em dados supervisionados ou respostas específicas, a DPO utiliza comparações de preferências humanas (como exemplo, a DPO retornará "Resposta A é melhor que Resposta B"). Com base nessas comparações, o modelo é ajustado para produzir respostas que maximizem a preferência do usuário.

O Modelo de Recompensa (RM) utiliza dados de preferência para gerar pares de respostas (escolhida e rejeitada), além de gerar anotações para edição de respostas. Cada amostra de classificação de preferência possui duas ou três respostas com hierarquia clara (editado > escolhido > rejeitado) (AI@META, 2024).

No passo final, o *prompt* é concatenado com as respostas em uma única linha, embaralhadas aleatoriamente, e na sequência separadas para o cálculo da pontuação. Essa abordagem ganha eficiência sem perder precisão.

2.10.1 Outros LLMs *open-source* avaliados

Além do LLaMA, esta pesquisa avalia outros cinco LLMs *open-source* servidos localmente via Ollama. Todos compartilham a arquitetura *Transformer* decodificadora e o paradigma de pré-treinamento massivo seguido de pós-treinamento por instruções, mas se distinguem em escolhas de dados, alinhamento e foco de uso. A síntese a seguir cobre os pontos relevantes para a tarefa de conversão de normas em RASE.

Mistral. A Mistral AI lançou em 2023 o Mistral 7B, modelo decodificador com 7 bilhões de parâmetros que combinou *grouped-query attention* (GQA) e *sliding-window attention* (SWA) para reduzir o custo computacional sem sacrificar qualidade (JIANG et al., 2023). Apesar do tamanho modesto, superou o LLaMA 2 13B em diversas avaliações de raciocínio, matemática e geração de código, e ainda hoje serve como referência de modelo enxuto para inferência local (JIANG et al., 2023). A versão Mistral 7B-Instruct foi ajustada para seguir instruções por meio

de SFT, e é a variante disponibilizada por padrão no Ollama.

Gemma. Família de modelos *open-weights* desenvolvida pelo Google DeepMind a partir da mesma pesquisa que originou o Gemini (Gemma Team, 2024). Disponível em tamanhos de 2B e 7B parâmetros, o Gemma adota a arquitetura *Transformer* decodificadora padrão, com SentencePiece BPE e treinamento em corpus multilíngue de cerca de 6 trilhões de *tokens*, seguido de SFT e RLHF. Os relatórios técnicos apontam ganho relativo em raciocínio matemático e código em comparação com modelos abertos de porte equivalente (Gemma Team, 2024).

Dolphin. Variante *fine-tuned* do Mistral 7B, criada pela Cognitive Computations sobre o dataset Dolphin, uma reimplementação livre do conjunto Orca da Microsoft, focada em raciocínio passo a passo. O modelo herda a arquitetura e os pesos base do Mistral, mas adiciona uma camada de SFT em diálogos sintéticos longos no estilo *Chain-of-Thought*, que tende a melhorar o seguimento de instruções estruturadas. É o modelo que apresenta a melhor relação custo-benefício nos experimentos desta dissertação.

Alpaca. Lançado em 2023 pelo Stanford Center for Research on Foundation Models, o Alpaca é um SFT do LLaMA original (7B) sobre 52 mil instruções autogeradas pelo método *Self-Instruct* (TAORI et al., 2023). O experimento foi pioneiro em demonstrar que um SFT barato sobre instruções sintéticas geradas por modelos maiores (text-davinci-003, à época) conseguia aproximar o comportamento de um modelo *instruction-tuned* a uma fração do custo (TAORI et al., 2023). O modelo serve, nesta pesquisa, como linha de base de SFT instrucional sobre um LLM aberto de primeira geração.

Qwen. Família de modelos *open-source* desenvolvida pela Alibaba Cloud, treinada em corpus de até 3 trilhões de *tokens* com forte ênfase em chinês e inglês (BAI et al., 2023). Adota *Transformer* decodificador com *rotary positional embeddings* (RoPE) e variantes *instruction-tuned* (Qwen-Chat) baseadas em SFT e RLHF. Em *benchmarks* multilíngues, o Qwen costuma se aproximar dos modelos de porte equivalente, mas a fração reduzida de português no pré-treinamento influencia o desempenho observado em normas brasileiras, inclusive a tendência, registrada nesta dissertação, de gerar saídas em chinês simplificado.

Os seis modelos foram escolhidos por representarem perfis complementares (peso, alinhamento, idioma, foco de *fine-tuning*), por estarem disponíveis em pesos abertos sob licenças permissivas e por serem executáveis no Ollama com requisitos de hardware compatíveis com o ambiente experimental adotado.

Entre as técnicas que especializam LLMs em tarefas de domínio sem alterar seus pesos, a *Engenharia de Prompt* é a mais leve. Consiste em desenhar com cuidado a instrução enviada ao modelo, de forma a obter respostas concisas e adequadas ao contexto. Hoje, é prática essencial para extrair o máximo dos LLMs (BROWN et al., 2020), e é a abordagem adotada nesta pesquisa.

O *prompt* funciona como uma configuração do modelo para a tarefa em questão. Não se trata só de escrever uma instrução isolada: é uma habilidade que exige entendimento profundo

das capacidades e limites do LLM. O processo costuma ser iterativo: refina, repete, testa, refina de novo (NASCIMENTO, 2024).

Um bom *prompt* é claro e preciso: dá ao modelo o que ele precisa para compreender a tarefa e indica o formato esperado da resposta. Em geral, *prompts* curtos e bem estruturados favorecem o entendimento da tarefa pelo modelo (NASCIMENTO, 2024).

Prompts longos demais fazem o modelo perder informação relevante no meio do caminho. A quantidade de exemplos também pesa: poucos exemplos bem escolhidos (*few-shot*) tornam a extração mais precisa, porque o modelo entende melhor o que se espera (BROWN et al., 2020).

2.11 Trabalhos Relacionados

A integração do BIM com tecnologias de IA generativa (IAG), como ChatGPT e congêneres, ainda é pouco explorada no setor AEC. O tema cobre aplicações práticas, *frameworks* propostos, desafios em aberto e direções futuras, sendo objeto de uma onda recente de trabalhos.

O primeiro estudo que analisamos trata da integração entre BIM e IAG, em particular ChatGPT e Bard. Os autores identificam o uso dessas tecnologias em *design* e tomada de decisão (RANE; CHOUDHARY; RANE, 2023). Em contrapartida, apontam um conjunto de desafios: falta de padronização dos dados, deficiências em segurança e a necessidade de capacitar os profissionais do setor (RANE; CHOUDHARY; RANE, 2023). Os modelos generativos ainda têm dificuldade em interpretar a semântica de dados BIM e enfrentam problemas gerais como alta demanda computacional e ausência de diretrizes éticas (RANE; CHOUDHARY; RANE, 2023). Como resposta, os autores propõem um *framework* que ataca a otimização de processos, a redução de custo e tempo e a adoção da IA via interfaces mais intuitivas (RANE; CHOUDHARY; RANE, 2023).

O segundo trabalho apresenta o JULE, um *chatbot* projetado para gerentes de obra (LIN, 2023). Ele combina BIM com PLN para entregar informação em tempo real no canteiro: o usuário consulta dimensões de componentes estruturais e ferramentas necessárias sem precisar abrir as interfaces convencionais do BIM (LIN, 2023). A solução obteve alta precisão na identificação de intenção e na recuperação das informações solicitadas, melhorando o uso prático de sistemas BIM no canteiro (LIN, 2023).

Um terceiro estudo aplicou LLMs (GPT-3.5 e GPT-4) a projetos BIM voltados a segurança (SAMMOUR et al., 2024). Os autores testaram os modelos em 385 questões de múltipla escolha de exames da BCSP, cobrindo gestão de sistemas, identificação de perigos e ergonomia (SAMMOUR et al., 2024). Usaram três técnicas de Engenharia de Prompt (*Prompt Direct*, *Chain-of-Thought* e *Few-Shot*) e mediram precisão e consistência. O GPT-4 atingiu 84,6% de acerto contra 73,8% do GPT-3.5. Apesar do bom desempenho em gerenciamento e controle de perigos, ambos os modelos travaram em questões de emergência e cálculos matemáticos (SAMMOUR et al., 2024).

O trabalho aponta o potencial dos modelos, mas reforça a necessidade de aprimoramento contra respostas imprecisas ou enviesadas (SAMMOUR et al., 2024).

O quarto estudo explorou técnicas de engenharia de *prompt* para otimizar LLMs no setor AEC. Os autores mostram que interações genéricas limitam a eficácia dos modelos e geram saídas desalinhadas com as necessidades do setor (SAMSAMI, 2024). Confiabilidade, adaptação às regulamentações e aceitação pelo mercado aparecem como barreiras adicionais. Para enfrentá-las, os pesquisadores coletaram *prompts* eficazes (instrucionais e baseados em restrições) e testaram a abordagem em projetos reais (SAMSAMI, 2024). *Prompts* bem desenhados reduziram erros de cronograma e melhoraram o reconhecimento de perigos. Mesmo assim, persistem alucinações em tarefas complexas e a necessidade de validação externa (SAMSAMI, 2024).

No conjunto, os quatro trabalhos oferecem um panorama relativamente atualizado da integração entre LLMs e BIM no setor AEC. Há avanços, mas pontos sensíveis seguem sem solução robusta: interpretação semântica dos dados BIM, segurança e adaptação dos modelos às demandas específicas do setor são exemplos. O campo segue aberto para pesquisa e inovação.

2.11.1 Análise dos Trabalhos Relacionados

A integração entre BIM, LLMs e IA generativa no setor AEC já mostra avanços, mas o terreno ainda é acidentado. O primeiro estudo identifica ganhos em *design* e tomada de decisão a partir de ChatGPT e Bard. Os obstáculos relatados (falta de padronização dos dados, segurança e complexidade semântica dos dados BIM) continuam abertos (RANE; CHOUDHARY; RANE, 2023). Faz sentido usar LLM aqui pela capacidade de varrer grandes volumes e produzir *insights*; o limite continua sendo a interpretação semântica.

O segundo estudo, sobre o JULE, combina BIM com PLN e mostra ganhos concretos no canteiro: consultas rápidas, sem depender das interfaces tradicionais. A condição para esse ganho é ter dados BIM bem estruturados, e é aí que aparece a barreira. Falhas semânticas e dificuldades de adaptar os dados ao modelo seguem como pontos sensíveis (LIN, 2023). Para interpretar regras e padrões regulatórios, os LLMs continuam sendo um caminho atraente pela agilidade na recuperação.

O terceiro trabalho aplicou GPT-3.5 e GPT-4 à segurança em BIM e teve resultados desiguais: precisão considerável no agregado, mas queda visível em questões complexas como emergências e cálculos. O ponto fraco fica claro: confiabilidade em questões técnicas. Os LLMs interpretam normas e regulamentos, mas exigem ajuste para evitar respostas imprecisas ou enviesadas (SAMMOUR et al., 2024).

O quarto estudo fecha o quadro com uma constatação prática: *prompts* genéricos não funcionam no setor AEC. Trocá-los por *prompts* específicos melhorou a precisão, ainda que alucinações em tarefas complexas e a necessidade de validação externa não tenham sumido (SAMSAMI, 2024). Adaptar o modelo às regulamentações do setor é um requisito de viabilidade,

e o uso de LLMs para interpretar regras regulatórias se mostra útil, embora ainda dependente de ajustes finos e revisão humana.

Diferencial desta pesquisa. Os trabalhos revisados concentram-se em LLMs proprietários (ChatGPT, GPT-3.5, GPT-4, Bard) e em aplicações pontuais de Engenharia de Prompt, sem comparações sistemáticas entre modelos *open-source* nem decomposição explícita da tarefa em níveis. Esta dissertação distingue-se ao avaliar comparativamente seis LLMs *open-source* servidos localmente via Ollama (Llama, Dolphin, Gemma, Mistral, Alpaca e Qwen), aplicando-os à conversão de normas brasileiras da AEC para o formato RASE em três níveis sucessivos de normalização (N1, N2 e N3) e em cinco experimentos (EN1, EN2, EN1N2, EN3 e EN1N2N3). A avaliação combina métricas lexicais, semânticas, de distância e voltadas para geração (BERTScore e ROUGE-L), complementadas por métricas de classificação, oferecendo um retrato multidimensional do desempenho de cada modelo.

3

Metodologia

Este capítulo descreve a metodologia da pesquisa, que se sustenta em três decisões: (i) decompor a tarefa de conversão de normas para o formato RASE em três níveis sucessivos de normalização (N1, N2 e N3); (ii) usar Engenharia de Prompt como técnica de especialização dos modelos; e (iii) comparar de forma sistemática seis LLMs *open-source* servidos localmente via Ollama. Na sequência, o capítulo apresenta o *dataset* adotado, o *pipeline* de execução, os cinco experimentos (EN1, EN2, EN1N2, EN3 e EN1N2N3), as métricas de validação e o ambiente computacional.

3.1 Visão Geral

A pesquisa avaliou LLMs *open-source* na conversão de normas técnicas da AEC para representações computáveis no formato RASE, usando somente Engenharia de Prompt. A conversão foi modelada como um processo de normalização em três níveis: N1 segmenta a sentença normativa em regras atômicas; N2 identifica os operadores RASE; N3 estrutura a saída em JSON. Seis modelos foram comparados em cinco experimentos (EN1, EN2, EN1N2, EN3 e EN1N2N3) sobre um *dataset* de 79 normas brasileiras anotadas, com treze métricas no total: nove de similaridade textual (lexicais, semânticas, de distância e voltadas para geração) e quatro de classificação derivadas da matriz de confusão. O nível N3 exige um critério de avaliação adicional (correspondência exata por campo do JSON), tratado em EN3 e em EN1N2N3 no capítulo de Resultados. A Figura 15 mostra o *pipeline* do começo ao fim (do texto bruto à representação RASE), com os *prompts* usados em cada nível, o servidor Ollama que hospeda os modelos e a etapa de validação par a par contra o *dataset* de referência.

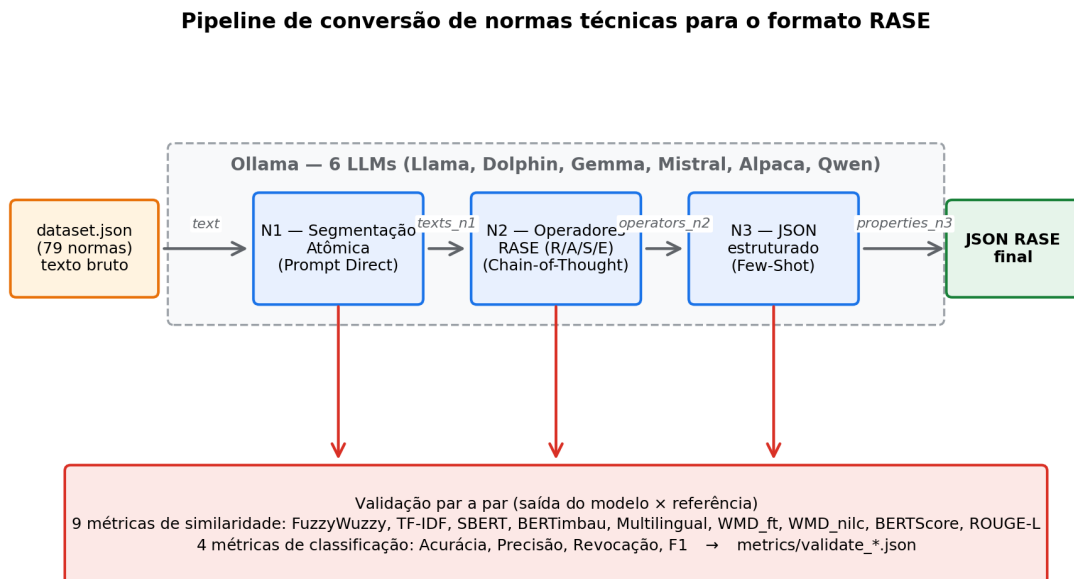


Figura 15 – Pipeline de conversão de normas técnicas para o formato RASE.

3.2 Necessidade de Normalização em Três Níveis

Sair do texto bruto de uma norma e chegar à representação RASE em JSON exige três raciocínios distintos: segmentação sintática, classificação semântica e extração de atributos estruturados. Pedir tudo isso em uma única chamada ao LLM costuma degradar a saída: aparecem alucinações, omissões e JSONs inválidos. Decompor em três níveis sucessivos isola cada decisão, mantém o escopo de cada chamada bem delimitado e permite validar cada etapa em separado.

3.2.1 N1: Segmentação Atômica

Sentenças normativas costumam embutir mais de uma regra coordenada. Veja o trecho “A inclinação transversal deve ser de até 2% para pisos internos e de até 3% para pisos externos”: são duas regras computáveis em uma mesma frase. O N1 existe para quebrar a sentença original em itens atômicos, com uma única regra por item. Sem essa segmentação, as etapas seguintes não conseguem atribuir corretamente os operadores RASE.

O *prompt* N1 vive em um arquivo de texto próprio. Instrui o modelo a produzir sentenças curtas e diretas, uma regra por linha, sem inventar elementos, e traz um exemplo demonstrativo.

3.2.2 N2: Identificação dos Operadores RASE

A partir de uma regra atômica vinda do N1, o N2 identifica quais trechos correspondem a Requisito (R), Aplicabilidade (A), Seleção (S) e Exceção (E), conforme a metodologia RASE. Essa classificação é o coração da formalização: sem ela, o texto continua em linguagem natural.

O *prompt* N2 também é mantido em arquivo próprio. Fixa a ordem dos quatro operadores e exige um formato exato de saída em quatro linhas, marcando o campo vazio com aspas duplas (""). Um exemplo único embutido serve de âncora para o modelo.

3.2.3 N3: Estruturação Semântica em JSON

Classificar um trecho como R, A, S ou E ainda deixa o conteúdo em linguagem natural. Para que a regra possa ser verificada automaticamente contra um modelo BIM, é preciso extrair os dados para objeto, propriedade, valor, comparador e unidade. O N3 produz esse JSON com os campos *type*, *object*, *property*, *comparison*, *target* e *unit*.

Como cada operador RASE tem natureza distinta, optei por um *prompt* por operador em N3: arquivos de texto separados para Aplicabilidade, Requisito, Seleção e Exceção. A Aplicabilidade descreve *a quem* a regra se aplica; o Requisito, *o que* é exigido; a Seleção restringe o escopo; a Exceção define os casos não cobertos. Cada *prompt* traz o esquema JSON explícito, heurísticas de mapeamento por padrões textuais comuns (por exemplo, “uso público” → *property* “uso”, *comparison* “=”, *target* “público”) e exemplos *few-shot* no próprio arquivo.

3.3 Engenharia de Prompt

A especialização dos LLMs foi feita por Engenharia de Prompt, sem alteração de pesos nem infraestrutura de recuperação. Toda a adaptação à tarefa vive nos *prompts*, organizados em arquivos de texto por nível e por operador. Três técnicas clássicas foram aplicadas, cada uma escolhida pela natureza do nível em que atua.

3.3.1 Prompt Direct

O *Prompt Direct* é uma instrução direta e objetiva, sem cadeia explícita de raciocínio. Foi a escolha para o N1, cuja tarefa, segmentar uma sentença em sub-regras, é estruturalmente simples e não exige raciocínio profundo sobre semântica normativa. Como vantagens, gasta menos *tokens* e tem menor latência; como risco, pode perder robustez em sentenças longas ou ambíguas. O *prompt* do N1, reproduzido na íntegra no Código A.1 (Apêndice A), define o modelo como um *reescritor* e reúne cinco regras imperativas: (i) quebrar o texto em sentenças curtas e diretas; (ii) garantir uma única regra computável por sentença; (iii) não inventar elementos (aplicabilidade, seleção, requisito, exceção), preservando apenas o que existir; (iv) não acrescentar explicações, títulos, marcadores ou numeração; e (v) devolver somente as sentenças, uma por linha. Fecha com um único exemplo demonstrativo no padrão *entrada* → *saída* e com o marcador {text}, substituído em tempo de execução pelo texto bruto da norma.

3.3.2 *Prompt Chain-of-Thought*

O *Chain-of-Thought* pede ao modelo que explicita o raciocínio em etapas antes de responder. Coube ao N2, em que classificar um trecho como R, A, S ou E depende de entender o papel sintático-semântico dentro da regra. A técnica reduz alucinações e melhora a classificação em sentenças com múltiplos operadores, ao preço de mais *tokens* consumidos. O *prompt* do N2, reproduzido no Código A.2 (Apêndice A), conduz o modelo por uma cadeia de decisão explícita: fixa a ordem dos passos (1. aplicabilidade → 2. seleção → 3. exceção → 4. requisito) e caracteriza cada operador. Exige uma saída de exatamente quatro linhas, com o campo inexistente marcado por "", e embute um único exemplo no formato esperado. Os marcadores {text} e {text_n1} recebem, respectivamente, o texto bruto da norma e a regra atômica vinda do N1.

3.3.3 *Prompt Few-Shot*

O *Few-Shot* embute, no próprio *prompt*, exemplos completos de entrada e saída, o que ancora o formato esperado. Foi aplicado ao N3, em que a saída precisa obedecer a um esquema JSON rígido: sem exemplos, os modelos divergem do formato ou inventam campos. A técnica entrega alta aderência ao formato e ao vocabulário esperado, mas consome parte da janela de contexto com os exemplos. Como cada operador RASE tem natureza distinta, optou-se por um *prompt* por operador (Aplicabilidade, Requisito, Seleção e Exceção), reproduzidos nos Códigos A.3 a A.6 (Apêndice A). Os quatro compartilham a mesma estrutura: fixam o esquema JSON de saída (campos *type*, *object*, *property*, *comparison*, *target* e *unit*); trazem heurísticas de mapeamento por padrões textuais comuns (por exemplo, “uso público” → *property* “uso”, *comparison* “=”); e embutem de dois a quatro exemplos completos no padrão *operador N2* → *JSON*. Variam apenas nas heurísticas e nos exemplos próprios de cada operador. Os marcadores {text} e {text_n1} recebem o texto bruto e a regra N1, e o marcador do operador correspondente ({aplicabilidade}, {requisito}, {selecao} ou {execao}) recebe o operador N2 a estruturar.

3.4 Modelos LLM Avaliados

Os seis modelos *open-source* rodaram localmente via Ollama (Ollama, 2024) (ollama serve), conforme a Tabela 02. Todos foram executados no mesmo *hardware*, com os mesmos *prompts* e o mesmo *dataset*, pois a comparabilidade entre os resultados depende dessa padronização. Cinco dos seis modelos vêm de *checkpoints* adaptados para português brasileiro publicados por terceiros (em especial cnmoro/*, splitpierre/* e brunoconterato/*), todos publicamente disponíveis no registro do Ollama; o Llama foi avaliado em sua versão oficial llama3.1:8b. A coluna *tag* identifica de forma precisa o *checkpoint* reproduzível usado em cada caso.

Tabela 02 – Modelos LLM *open-source* avaliados, com origem e *tag* servida via Ollama (definidos em `config/models.py`).

Modelo	Origem	Tag servida via Ollama	Parâmetros
Llama	Meta	llama3.1:8b	8B
Dolphin	cnmoro (LLaMA-3 PT)	cnmoro/llama-3-8b-dolphin-portuguese-v0.3:4_k_m	8B (Q4_K_M)
Gemma	brunoconterato (PT-BR)	brunoconterato/Gemma-3-Gaia-PT-BR-4b-it:f16	4B (FP16)
Mistral	cnmoro (PT)	cnmoro/mistral_7b_portuguese:q4_K_M	7B (Q4_K_M)
Alpaca	splitpierre (Bode PT-BR)	splitpierre/bode-alpaca-pt-br:13b-Q4_0	13B (Q4_0)
Qwen	cnmoro (PT)	cnmoro/Qwen2.5-0.5B-Portuguese-v1:q4_k_m	0,5B (Q4_K_M)

3.5 Dataset

O *dataset* foi consolidado em um único arquivo JSON, com cerca de 270 KB e **79 normas** anotadas. As normas vêm de regulamentações brasileiras da AEC, com ênfase na NBR 9050 ([ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS, 2020](#)) (acessibilidade a edificações, mobiliário, espaços e equipamentos urbanos), adaptadas a partir do trabalho de [Mendonça e Manzione \(2020\)](#).

Cada entrada do *dataset* é estruturada da seguinte forma:

- **text**: texto bruto da norma.
- **texts_n1**: lista de regras atômicas, cada uma com **text_n1** e **operators_n2**.
- **operators_n2**: dicionário com as chaves **requirements**, **applicability**, **selection** e **exception**, cada uma com **text_n2** e **properties_n3**.
- **properties_n3**: representação em JSON com os campos **type**, **object**, **property**, **comparison**, **target** e **unit**.

O Código 3.1 apresenta uma entrada real do *dataset*, extraída da NBR 9050, com a estrutura aninhada completa: o texto bruto da norma (**text**), uma das regras atômicas segmentadas (**text_n1**) e seus quatro operadores RASE (**operators_n2**), cada um com o trecho classificado (**text_n2**) e as propriedades estruturadas em JSON (**properties_n3**). Operadores ausentes preservam a estrutura com *strings* vazias, como ilustra a **exception**.

Código 3.1 – Exemplo de entrada do *dataset* (NBR 9050), com a decomposição RASE de referência nos três níveis.

```
1 "text": "As áreas de qualquer espaço ou edificação de uso público ou
   coletivo devem ser servidas de uma ou mais rotas acessíveis.
   [...]",
```

```

2 "texts_n1": [
3   {
4     "text_n1": "As áreas de qualquer espaço ou edificação de uso
        público ou coletivo devem ser servidas de uma ou mais rotas
        acessíveis.",
5     "operators_n2": {
6       "applicability": {
7         "text_n2": "As áreas de qualquer espaço ou edificação",
8         "properties_n3": { "type": "aplicabilidade", "object":
            "espaço", "property": "", "comparation": "", "target":
            "", "unit": "" }
9       },
10      "selection": {
11        "text_n2": "uso público ou coletivo",
12        "properties_n3": { "type": "selecao", "object": "espaço",
            "property": "uso", "comparation": "=", "target": "público
            ou coletivo", "unit": "" }
13      },
14      "exception": {
15        "text_n2": "",
16        "properties_n3": { "type": "", "object": "", "property":
            "", "comparation": "", "target": "", "unit": "" }
17      },
18      "requeriments": {
19        "text_n2": "devem ser servidas de uma ou mais rotas
            acessíveis",
20        "properties_n3": { "type": "requisito", "object": "espaço ou
            edificação", "property": "uso", "comparation": "=",
            "target": "VERDADEIRO", "unit": "" }
21      }
22    }
23  ]
24 ]

```

O mesmo arquivo serve como entrada para os geradores e como *target* de referência para os validadores, simplificando a reprodutibilidade dos experimentos.

3.6 Pipeline de Execução e Experimentos

A orquestração dos experimentos vive em um *script* principal, que oferece um menu para a etapa de geração e outro para a de validação. As saídas de cada modelo, em cada experimento,

são persistidas em arquivos JSON próprios, no padrão `generate_<nível>_<modelo>.json` (por exemplo, `generate_n1_alpaca.json` para o N1 produzido pelo Alpaca). Os resultados das validações seguem o mesmo princípio, agora um arquivo por experimento, no padrão `validate_<experimento>.json` (`validate_n1.json`).

Para isolar o que cada nível contribui e medir como o erro se propaga, defini cinco experimentos, ilustrados na Figura 16. EN1, EN2 e EN3 avaliam cada etapa em separado: cada um deles recebe o nível anterior *de referência* (extraído do *dataset*) como entrada, eliminando o ruído das etapas precedentes. Em paralelo, EN1N2 simula o *pipeline* encadeado dos dois primeiros níveis (o N1 produzido pelo próprio modelo alimenta sua etapa N2), e EN1N2N3 fecha o *pipeline* completo N1→N2→N3, encadeando as três etapas com saídas geradas pelo próprio modelo. Esses dois últimos cenários expõem a **propagação de erros** ao longo da cadeia.

Os cinco experimentos: EN1, EN2 e EN3 (isolados); EN1N2 e EN1N2N3 (encadeados)

EN1 — Segmentação N1 isolada



EN2 — Identificação RASE isolada (N1 de referência como entrada)



EN3 — Estruturação N3 isolada (N2 de referência como entrada)



EN1N2 — Pipeline encadeado (N1 → N2 do mesmo modelo)



EN1N2N3 — Pipeline completo encadeado (N1 → N2 → N3 do mesmo modelo)



Figura 16 – Estrutura dos cinco experimentos: EN1, EN2 e EN3 (isolados) e EN1N2 e EN1N2N3 (encadeados).

3.6.1 Experimento EN1

O EN1 avalia a geração N1 em isolamento. A entrada é o texto bruto da norma; a saída gerada vai contra o N1 de referência do *dataset*. O que se mede é a qualidade da segmentação a partir do texto bruto, sem interferência das etapas posteriores.

3.6.2 Experimento EN2

O EN2 avalia o N2 sozinho, sem o ruído da etapa anterior. A entrada é o N1 de referência (tirado direto do *dataset*), e a saída é comparada ao N2 de referência, operador por operador (R, A, S e E). O foco aqui é a qualidade pura da classificação RASE.

3.6.3 Experimento EN1N2

O EN1N2 é o *pipeline* encadeado dos dois primeiros níveis: o N1 gerado pelo modelo alimenta a etapa N2 do mesmo modelo. A saída final é comparada ao N2 de referência. O objetivo é medir a **propagação de erros** entre as duas etapas em um cenário totalmente automatizado.

3.6.4 Experimento EN3

O EN3 avalia o N3 em isolamento, novamente sem o ruído das etapas anteriores. A entrada é o N2 de referência por operador (extraído do *dataset*), e a saída gerada é comparada à representação JSON de referência, nos campos *type*, *object*, *property*, *comparison*, *target* e *unit*. A métrica aqui é a qualidade pura da estruturação semântica em JSON.

3.6.5 Experimento EN1N2N3

O EN1N2N3 fecha o *pipeline* encadeado: o N1 do modelo alimenta o N2 do mesmo modelo, que alimenta o N3 do mesmo modelo. A saída final (JSON estruturado por operador) é comparada à referência do *dataset*. Este cenário replica o uso real do *pipeline* em produção e expõe a **propagação de erros** ao longo dos três níveis.

3.7 Métricas de Validação

A avaliação combinou treze métricas em cinco famílias: similaridade lexical, similaridade semântica, distância semântica, métricas voltadas para geração e métricas de classificação. As nove primeiras (similaridade) são calculadas par a par entre saída prevista e referência do *dataset*, sempre em escala normalizada [0, 1]: quanto maior, mais próxima a saída está da referência. As quatro métricas de classificação derivam da matriz de confusão entre saída e referência.

3.7.1 Similaridade Lexical

- **FuzzyWuzzy**: utiliza *fuzz.partial_ratio*, baseado na distância de Levenshtein parcial; tolera fragmentos da string-alvo.
- **TF-IDF**: cosseno entre vetores TF-IDF das duas *strings*, ponderando os termos pela sua importância no corpus (Manning; Raghavan; Schütze, 2008).

3.7.2 Similaridade Semântica

- **SBERT (PT)**: *embeddings* produzidos pelo modelo *tgsc/sentence-transformer-ult5-pt-small* (REIMERS; GUREVYCH, 2019).
- **BERTimbau**: *embeddings* de *rufimelo/Legal-BERTimbau-sts-large-ma-v3*, especializado em texto jurídico em português brasileiro (Souza; Nogueira; Lotufo, 2020).
- **Multilingual**: *embeddings* de *sentence-transformers/paraphrase-multilingual-mpnet-base-v2*, treinado em mais de cinquenta idiomas.

3.7.3 Distância Semântica (WMD)

- **WMD_ft**: *Word Mover's Distance* (KUSNER et al., 2015) com *embeddings* do FastText (*fasttext-wiki-news-subwords-300*) (Bojanowski et al., 2017).
- **WMD_nilc**: WMD com *embeddings* do projeto NILC (*cbow_s300.txt*), treinados sobre corpora em português brasileiro, com valor normalizado para o intervalo [0, 1].

3.7.4 Métricas Voltadas para Geração

- **BERTScore**: similaridade contextual baseada em *embeddings* do BERT, com alinhamento ótimo entre os *tokens* da saída e da referência (ZHANG et al., 2020).
- **ROUGE-L**: razão entre a maior subsequência comum (LCS) e a referência, originalmente proposta para sumarização (LIN, 2004).

3.7.5 Métricas de Classificação

Complementarmente às métricas de similaridade, foram calculadas quatro métricas de classificação. O critério de acerto foi definido em função do nível de normalização:

- Em **N1**, considera-se uma sentença atômica prevista como *verdadeiro positivo* (VP) se sua similaridade semântica em relação a alguma sentença da referência exceder um limiar

pré-definido (0,7). Como N1 não possui rótulos negativos no *dataset*, calculam-se apenas as estatísticas baseadas em VP, FP e FN.

- Em **N2**, a métrica é calculada por operador (Requisito, Aplicabilidade, Seleção e Exceção) e reporta-se tanto a média macro entre os quatro operadores quanto a média *overall*; o emparelhamento utiliza o mesmo critério baseado em similaridade semântica de N1.
- Em **N3**, a métrica é calculada por campo do JSON (*object*, *property*, *comparison*, *target* e *unit*) por correspondência exata, sendo reportada a média macro entre os cinco campos.

As métricas são definidas como:

- **Acurácia:** $ACC = (VP + VN) / (VP + VN + FP + FN)$.
- **Precisão:** $PRE = VP / (VP + FP)$.
- **Revocação:** $REC = VP / (VP + FN)$.
- **F1-score:** $F1 = 2 \cdot (PRE \cdot REC) / (PRE + REC)$.

Para tornar concretos esses conceitos, convém fixar a convenção adotada. Em todos os níveis, o item avaliado: uma sentença, em N1; um operador, em N2; um campo do JSON, em N3; é tratado como *positivo* quando está preenchido e como *negativo* quando está vazio (). Disso decorre que: **VP** significa preenchido na referência e na previsão, e os dois “casam”; **VN**, vazio em ambos; **FN**, preenchido na referência mas vazio na previsão (omissão); e **FP**, vazio na referência mas preenchido na previsão (alucinação). O critério de “casar” é a similaridade semântica acima do limiar (0,7) em N1 e N2, e a igualdade exata (após normalização de caixa, acentos e pontuação final) em N3. Quando ambos os lados estão preenchidos, mas não casam, a ocorrência é contada simultaneamente como FP e FN (emitiu algo errado e perdeu o correto). Note que N1 não gera VN, pois não há item que “deva estar vazio”: toda sentença, prevista ou de referência, é conteúdo.

Todas as métricas são persistidas em *metrics/validate_*.json*, e o detalhamento por instância é registrado em *metrics/details/validate_<nível>_<modelo>.jsonl*.

3.8 Ambiente Computacional

Todas as execuções foram conduzidas em uma única estação de trabalho, com a seguinte configuração:

- **CPU:** Intel Core i9-13900K, 13ª geração, 3,00 GHz.
- **Memória RAM:** 32 GB.

- **GPU:** NVIDIA RTX 4080.
- **Sistema operacional:** Ubuntu 22.04.
- **Python:** 3.11.9.
- **Servidor LLM:** Ollama (`ollama serve`).
- **Bibliotecas principais:** `sentence-transformers`, `gensim`, `fuzzywuzzy` e `scikit-learn` (para TF-IDF).

Padronizar ambiente, *prompts* e *dataset* entre os seis modelos é o que permite atribuir as diferenças observadas nos capítulos seguintes ao comportamento dos próprios LLMs, e não a oscilações de infraestrutura ou de protocolo de avaliação.

4

Resultados

Este capítulo apresenta os resultados da aplicação dos seis LLMs *open-source* sobre as 79 normas brasileiras do *dataset*, nos cinco experimentos definidos no capítulo anterior: EN1 (segmentação N1 isolada), EN2 (identificação RASE com N1 de referência), EN1N2 (*pipeline* encadeado $N1 \rightarrow N2$), EN3 (estruturação JSON em isolamento, com N2 de referência) e EN1N2N3 (*pipeline* encadeado completo $N1 \rightarrow N2 \rightarrow N3$). Todos os números vêm diretamente dos arquivos *metrics/validate_*.json*, gerados pela rotina de validação descrita na Seção 3.7. Para cada experimento, o texto traz as médias agregadas por métrica, os resultados detalhados por modelo, os tempos de execução e uma discussão integrada.

Convenção de leitura das tabelas. Em todas as tabelas numéricas deste capítulo, o realce é feito **por coluna**: o melhor valor de cada coluna aparece em **verde** e o pior em **vermelho**. Nas tabelas de qualidade (similaridade e classificação), os valores em **negrito** são aqueles dentro do limiar de similaridade adotado na validação, isto é, $\geq 0,7$ (Seção 3.7.5). Já nas tabelas de tempo de execução, em que *menor é melhor*, o **verde** marca o modelo mais rápido e o **vermelho** o mais lento, e o negrito não se aplica.

4.1 Visão Geral dos Experimentos

A Tabela 03 resume as médias por métrica nos cinco experimentos, agregadas sobre os seis modelos. Três padrões já saltam aos olhos: as métricas baseadas em *embeddings* (SBERT, BERTimbau, Multilingual e BERTScore) mantêm desempenho consistentemente mais alto; as lexicais (FuzzyWuzzy, TF-IDF e ROUGE-L) perdem terreno à medida que se sai do EN1 e se entra no EN1N2; e o EN3 (N3 isolado) sobe a patamares próximos do EN1 graças ao vocabulário restrito dos campos JSON.

As métricas semânticas (SBERT, BERTimbau e Multilingual), junto do BERTScore, preservaram melhor o valor agregado ao longo dos cinco experimentos, por serem mais robustas

Tabela 03 – Médias por métrica nos cinco experimentos, agregadas sobre os seis modelos.

Métrica	EN1	EN2	EN1N2	EN3	EN1N2N3
<i>Similaridade lexical</i>					
FuzzyWuzzy	0,475	0,461	0,464	0,847	0,617
TF-IDF	0,636	0,312	0,417	0,597	0,514
<i>Similaridade semântica</i>					
SBERT	0,801	0,490	0,594	0,906	0,777
BERTimbau	0,816	0,542	0,631	0,836	0,752
Multilingual	0,850	0,595	0,679	0,873	0,793
<i>Distância semântica (WMD)</i>					
WMD_ft	0,716	0,580	0,625	0,846	0,757
WMD_nilc	0,688	0,546	0,595	0,842	0,742
<i>Métricas voltadas para geração</i>					
BERTScore	0,866	0,753	0,790	0,880	0,843
ROUGE-L	0,599	0,308	0,404	0,765	0,616
<i>Métricas de classificação</i>					
Acurácia	0,475	0,273	—	0,576	—
Precisão	0,545	0,202	—	0,024	—
Revocação	0,774	0,198	—	0,380	—
F1-score	0,631	0,197	—	0,043	—

a variações de superfície. As lexicais e o ROUGE-L sofrem com a reformulação produzida pelos LLMs. No N3, as métricas de similaridade textual sobem para patamares bem mais altos do que em N2 (saída quase-estruturada e curta), mas as métricas de classificação por correspondência exata, exigida em EN3, expõem dificuldades sistemáticas: a precisão macro fica abaixo de 4%. O EN1N2N3 confirma a tendência observada em EN1N2: as métricas semânticas absorvem bem a propagação de erros, enquanto as lexicais (TF-IDF e ROUGE-L) acusam queda visível.

4.2 Resultados por Experimento

4.2.1 EN1: Segmentação N1

No EN1, a comparação foi entre as regras N1 geradas pelos LLMs e o N1 de referência. As métricas semânticas ficaram bem acima das lexicais (Multilingual 0,850, BERTimbau 0,816 e SBERT 0,801, contra TF-IDF 0,636 e FuzzyWuzzy 0,475), o que indica que a segmentação tolera reformulação lexical quando o sentido é preservado. O **Dolphin** dominou: liderou em *todas* as nove métricas de similaridade e em três das quatro de classificação, com o melhor desempenho isolado do experimento. Os números detalhados estão na Tabela 04.

A Figura 17 apresenta o comparativo visual entre os seis modelos nas nove métricas de similaridade em EN1.

Tabela 04 – Resultados por modelo em EN1 (similaridade e classificação).

Modelo	Lexical		Semântica			WMD		Geração	
	FW	TF-IDF	SBERT	BERTimbau	Multi.	WMD_ft	WMD_nllc	BERTScore	ROUGE-L
Alpaca	0,473	0,561	0,748	0,762	0,803	0,694	0,664	0,841	0,540
Dolphin	0,641	0,756	0,873	0,891	0,913	0,774	0,751	0,905	0,703
Gemma	0,412	0,695	0,842	0,850	0,885	0,726	0,699	0,883	0,648
Llama	0,422	0,672	0,835	0,844	0,873	0,711	0,684	0,871	0,605
Mistral	0,534	0,704	0,842	0,874	0,899	0,726	0,699	0,888	0,651
Qwen	0,367	0,429	0,668	0,673	0,726	0,663	0,632	0,808	0,448

Métricas de classificação

Modelo	Acurácia	Precisão	Revocação	F1
Alpaca	0,419	0,512	0,699	0,591
Dolphin	0,676	0,812	0,801	0,806
Gemma	0,466	0,486	0,917	0,636
Llama	0,446	0,479	0,865	0,616
Mistral	0,610	0,670	0,872	0,758
Qwen	0,235	0,311	0,487	0,380

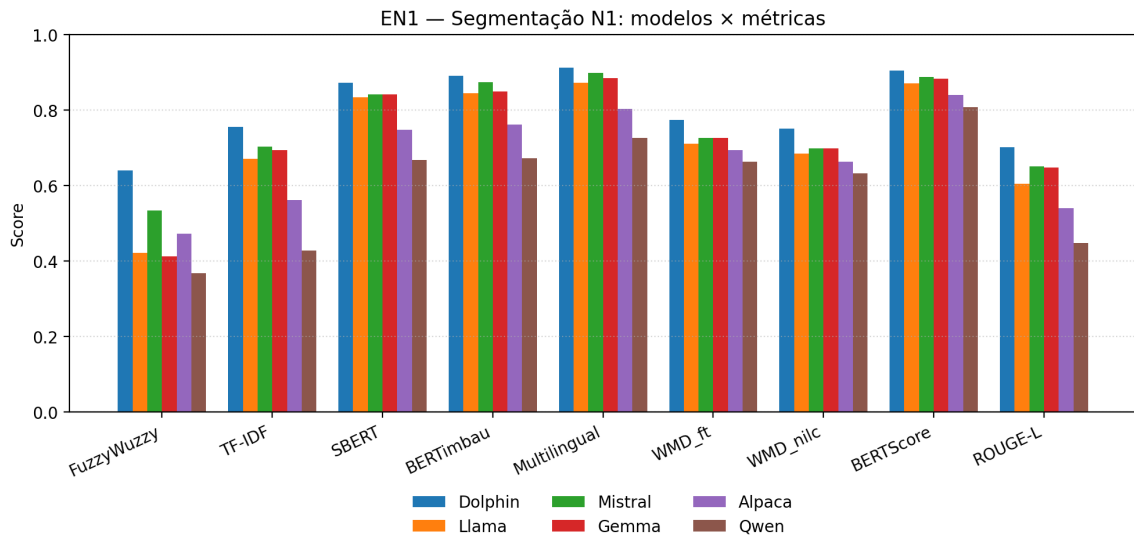


Figura 17 – Comparativo modelo × métrica em EN1.

4.2.2 EN2: Identificação dos Operadores RASE

O EN2 mediu a identificação e a estruturação dos operadores RASE a partir do N1 de referência. As médias caíram em relação ao EN1 (SBERT 0,490; Multilingual 0,595), o que reflete a maior dificuldade da formalização estruturada. As métricas de classificação caem ainda mais (F1 macro 0,197): os modelos têm dificuldade real em distinguir Seleção e Exceção. O **Llama** lidera nas métricas semânticas e lexicais; o **Dolphin** fica com a melhor F1 macro (0,286). Os números detalhados estão na Tabela 05.

A Figura 18 consolida o desempenho dos seis modelos nas nove métricas de similaridade em EN2.

Tabela 05 – Resultados por modelo em EN2 (similaridade e classificação macro por operador).

Modelo	Lexical		Semântica			WMD		Geração	
	FW	TF-IDF	SBERT	BERTimbau	Multi.	WMD_ft	WMD_nilc	BERTScore	ROUGE-L
Alpaca	0,334	0,168	0,386	0,418	0,484	0,522	0,486	0,702	0,172
Dolphin	0,511	0,423	0,557	0,621	0,665	0,620	0,589	0,786	0,412
Gemma	0,384	0,364	0,520	0,579	0,625	0,608	0,577	0,772	0,350
Llama	0,638	0,447	0,595	0,657	0,685	0,617	0,586	0,796	0,427
Mistral	0,563	0,385	0,575	0,607	0,665	0,607	0,575	0,779	0,377
Qwen	0,336	0,083	0,304	0,369	0,444	0,496	0,462	0,684	0,111

Métricas de classificação (macro por operador)

Modelo	Acurácia	Precisão	Revocação	F1
Alpaca	0,081	0,103	0,109	0,105
Dolphin	0,339	0,301	0,276	0,286
Gemma	0,182	0,252	0,300	0,265
Llama	0,439	0,237	0,250	0,243
Mistral	0,434	0,268	0,214	0,238
Qwen	0,160	0,051	0,039	0,044

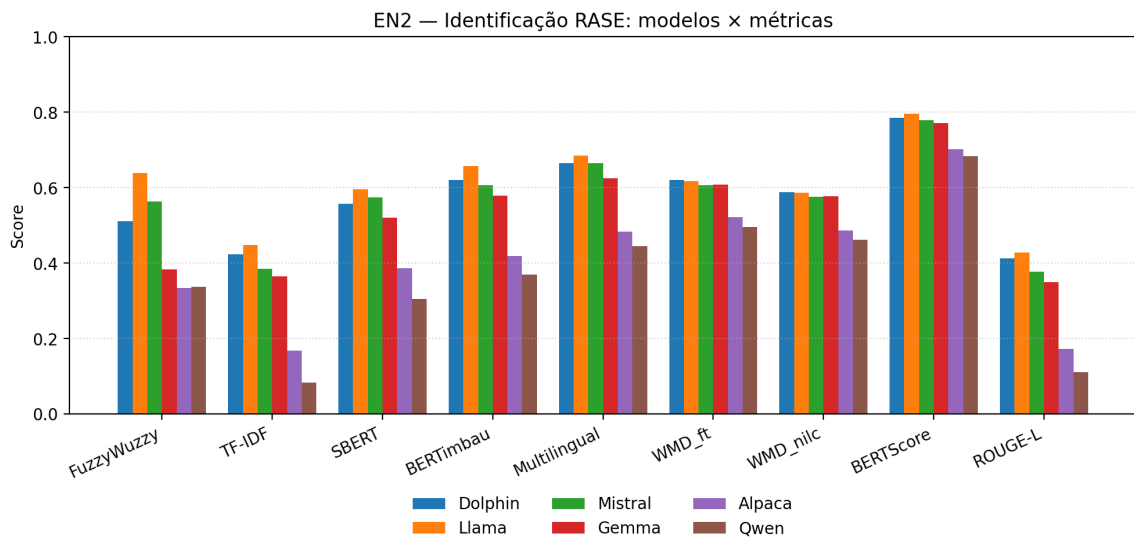


Figura 18 – Comparativo modelo × métrica em EN2.

4.2.3 EN1N2: Pipeline Encadeado

O EN1N2 testou a robustez do *pipeline* totalmente automatizado: o N1 produzido pelo modelo na primeira etapa alimenta o N2 do mesmo modelo. Os resultados confirmam a hipótese de **propagação de erros**, mas com uma surpresa. As médias caem em relação ao EN2 em parte das métricas e se mantêm próximas ou até melhoram em outras, caso das semânticas: Multilingual sobe de 0,595 para 0,679; BERTimbau de 0,542 para 0,631; SBERT de 0,490 para 0,594. O motivo é que o *pipeline* produz saídas mais próximas das frases originais e, por isso, mais alinhadas ao texto de referência no nível semântico, ainda que perca precisão na atribuição dos operadores. A liderança é compartilhada: o **Dolphin** domina TF-IDF, WMD_ft, WMD_nilc, BERTScore e ROUGE-L; o **Llama** lidera SBERT e empata em Multilingual. Os números completos estão na Tabela 06.

A Figura 19 apresenta o comparativo visual em EN1N2, evidenciando a alternância de liderança entre Dolphin e Llama por métrica.

Tabela 06 – Resultados por modelo em EN1N2 (similaridade).

Modelo	Lexical		Semântica			WMD		Geração	
	FW	TF-IDF	SBERT	BERTimbau	Multi.	WMD_ft	WMD_nllc	BERTScore	ROUGE-L
Alpaca	0,375	0,285	0,494	0,523	0,581	0,573	0,538	0,742	0,282
Dolphin	0,549	0,529	0,662	0,711	0,749	0,673	0,644	0,826	0,509
Gemma	0,391	0,458	0,615	0,656	0,701	0,646	0,617	0,804	0,436
Llama	0,570	0,521	0,677	0,721	0,749	0,649	0,620	0,821	0,488
Mistral	0,559	0,489	0,667	0,698	0,740	0,648	0,618	0,817	0,468
Qwen	0,342	0,219	0,448	0,478	0,551	0,562	0,530	0,731	0,240

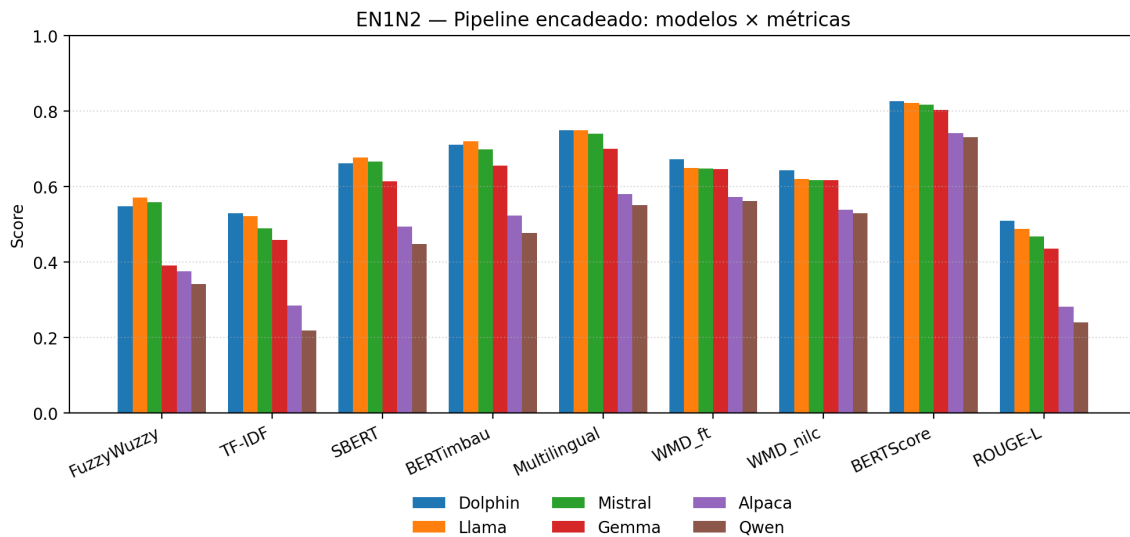


Figura 19 – Comparativo modelo × métrica em EN1N2.

4.2.4 EN3: Estruturação Semântica em JSON com N2 de referência

O nível N3 (estruturação semântica em JSON) foi avaliado em dois cenários complementares: o EN3, em isolamento, recebendo como entrada os operadores N2 de referência; e o EN1N2N3, no *pipeline* encadeado de ponta a ponta, em que o N1 gerado alimenta o N2 do mesmo modelo, que por sua vez alimenta o N3 do mesmo modelo. Em ambos os cenários, a comparação não é entre *strings* normativas inteiras, e sim campo a campo (*type*, *object*, *property*, *comparation*, *target* e *unit*), sobre a serialização canônica do JSON gerado contra o JSON anotado.

Como a saída é *quase-estruturada* e composta por campos curtos com vocabulário restrito, as métricas de similaridade textual sobem para patamares bem mais altos do que em N1 e N2 (ver Tabela 07). As métricas de classificação por correspondência exata, em compensação, mostram um quadro bem mais duro: a precisão macro fica abaixo de 4%. Isso sinaliza que os modelos preservam o significado, mas variam o vocabulário em relação à referência, por exemplo *target* = “público” em vez de *target* = “uso público”. O **Llama** dominou: liderou em *todas* as nove métricas de similaridade, com SBERT 0,947 e Multilingual 0,928. Mistral, Dolphin e Qwen ficam próximos em métricas semânticas, mostrando que a etapa N3 isolada é a mais homogênea entre os modelos avaliados.

Tabela 07 – Resultados por modelo em EN3 (similaridade e classificação macro por campo).

Modelo	Lexical		Semântica			WMD		Geração	
	FW	TF-IDF	SBERT	BERTimbau	Multi.	WMD_ft	WMD_nilc	BERTScore	ROUGE-L
Alpaca	0,729	0,395	0,846	0,745	0,798	0,756	0,751	0,815	0,624
Dolphin	0,860	0,694	0,912	0,848	0,887	0,842	0,839	0,901	0,799
Gemma	0,751	0,480	0,858	0,755	0,823	0,762	0,757	0,810	0,652
Llama	0,892	0,736	0,947	0,897	0,928	0,915	0,912	0,925	0,871
Mistral	0,877	0,689	0,938	0,891	0,914	0,903	0,900	0,922	0,842
Qwen	0,872	0,590	0,937	0,877	0,889	0,898	0,895	0,905	0,804

Métricas de classificação (macro por campo)

Modelo	Acurácia	Precisão	Revocação	F1
Alpaca	0,283	0,012	0,434	0,024
Dolphin	0,705	0,034	0,447	0,062
Gemma	0,292	0,017	0,603	0,033
Llama	0,725	0,038	0,361	0,067
Mistral	0,760	0,037	0,371	0,066
Qwen	0,692	0,004	0,063	0,008

O setup foi idêntico ao dos experimentos anteriores: os mesmos seis LLMs servidos via Ollama, os mesmos *prompts few-shot* por operador, o mesmo *dataset* de 79 normas. Saídas por modelo ficam em arquivos JSON dedicados (`generate_n3_<modelo>.json`); o agregado vive em um único arquivo de métricas (`validate_n3.json`). A correspondência exata por campo é mais dura que o limiar de similaridade adotado em N1 e N2, o que derruba a precisão observada, em especial nos modelos com menor aderência ao esquema (Alpaca e Qwen).

4.2.5 EN1N2N3: Pipeline Encadeado Completo

O EN1N2N3 é o *pipeline* automatizado de ponta a ponta. Este é o cenário que replica o uso real em produção: a única entrada é o texto bruto da norma, e a saída final é o JSON estruturado por operador. Como cada etapa acumula transformações, é também o experimento mais sujeito à **propagação de erros**: as médias caem em relação ao EN3 isolado em todas as métricas (SBERT 0,777 vs. 0,906; Multilingual 0,793 vs. 0,873; BERTScore 0,843 vs. 0,880). Ainda assim, os patamares absolutos permanecem superiores aos observados em EN2 e EN1N2, beneficiados pelo vocabulário restrito do JSON. A Tabela 08 traz os resultados por modelo.

Tabela 08 – Resultados por modelo em EN1N2N3 (similaridade sobre o JSON estruturado).

Modelo	Lexical		Semântica			WMD		Geração	
	FW	TF-IDF	SBERT	BERTimbau	Multi.	WMD_ft	WMD_nilc	BERTScore	ROUGE-L
Alpaca	0,525	0,345	0,687	0,645	0,701	0,676	0,658	0,782	0,471
Dolphin	0,685	0,600	0,811	0,794	0,832	0,776	0,762	0,871	0,683
Gemma	0,536	0,419	0,747	0,710	0,767	0,712	0,697	0,807	0,553
Llama	0,701	0,630	0,834	0,823	0,853	0,804	0,790	0,882	0,711
Mistral	0,693	0,600	0,830	0,814	0,845	0,801	0,788	0,879	0,691
Qwen	0,561	0,490	0,750	0,724	0,762	0,773	0,759	0,838	0,590

A saída de cada modelo é persistida em um arquivo JSON próprio (`generate_n1n2n3_<modelo>.json`), e as métricas agregadas ficam em um arquivo único (`validate_n1n2n3.json`). O **Llama** mantém a liderança que conquistou em EN3 isolado, agora em todas as nove métricas de similaridade; **Mistral** fica imediatamente atrás, com diferenças marginais (Multilingual 0,845 vs. 0,853). Em linha com o padrão observado em EN1N2, Llama, Mistral e Dolphin preservam vantagem

relativa também na cadeia completa, ao passo que Alpaca e Gemma acumulam queda em todas as métricas e entregam o pior desempenho global, evidenciando que falhas em N1/N2 contaminam a estruturação final em N3 mesmo nas métricas semânticas.

4.3 Resultados por Modelo

A Tabela 09 traz a média global por modelo, agregando as nove métricas de similaridade textual sobre os cinco experimentos (EN1, EN2, EN1N2, EN3 e EN1N2N3). O ranking final foge do esperado: o líder é o **Llama**, sustentado pela liderança absoluta em EN3 e EN1N2N3 (ambas as etapas N3) e pelo melhor desempenho em EN2. O **Dolphin** fica em segundo: domina o EN1 com folga e mantém desempenho competitivo nas etapas seguintes. Mistral aparece em terceiro, seguido por Gemma, Qwen e Alpaca.

Tabela 09 – Média global por modelo (nove métricas de similaridade × cinco experimentos).

Modelo	EN1	EN2	EN1N2	EN3	EN1N2N3	Média Global
Llama	0,724	0,605	0,646	0,892	0,781	0,730
Dolphin	0,801	0,576	0,650	0,843	0,757	0,725
Mistral	0,757	0,570	0,634	0,875	0,771	0,721
Gemma	0,738	0,531	0,591	0,739	0,661	0,652
Qwen	0,602	0,366	0,456	0,852	0,694	0,594
Alpaca	0,676	0,408	0,488	0,718	0,610	0,580

A seguir, destacam-se as observações por modelo.

Llama. Lidera a média global (0,730) e domina os experimentos do nível N3: em EN3 isolado ficou à frente em *todas* as nove métricas (SBERT 0,947; Multilingual 0,928; BERTScore 0,925); em EN1N2N3, repetiu a liderança nas nove métricas (SBERT 0,834; Multilingual 0,853; BERTScore 0,882). Foi também o melhor em EN2 em sete métricas: FuzzyWuzzy (0,638), TF-IDF (0,447), SBERT (0,595), BERTimbau (0,657), Multilingual (0,685), BERTScore (0,796) e ROUGE-L (0,427), além da maior acurácia macro de classificação no mesmo experimento (0,439). No EN1N2, liderou em FuzzyWuzzy (0,570), SBERT (0,677) e BERTimbau (0,721), empatando com o Dolphin em Multilingual (0,749). É a escolha quando se tolera o custo computacional.

Dolphin. Foi o melhor segmentador em EN1, com domínio absoluto nas nove métricas (BERTimbau 0,891; Multilingual 0,913; SBERT 0,873; BERTScore 0,905) e a melhor F1 macro de classificação no experimento (0,806). No EN2, liderou em WMD_ft (0,620), WMD_nilc (0,589) e F1 macro (0,286). No EN1N2, ficou à frente em TF-IDF (0,529), WMD_ft (0,673), WMD_nilc (0,644), BERTScore (0,826) e ROUGE-L (0,509). Em EN3 e EN1N2N3, mantém desempenho competitivo (SBERT 0,912 e 0,811, respectivamente), embora atrás de Llama e Mistral. Soma-se a isso o baixo custo computacional (Seção 4.5), o que rende a melhor relação custo-benefício observada.

Mistral. Desempenho consistente em todos os cenários, e o mais regular da bateria: terceiro colocado na média global (0,721), a menos de um ponto percentual do líder. Ficou próximo dos líderes em EN1 (Multilingual 0,899; BERTimbau 0,874) e em EN1N2 (Multilingual 0,740; BERTimbau 0,698), entregou a segunda melhor acurácia macro do EN2 (0,434), e fica colado ao Llama em EN3 (SBERT 0,938) e em EN1N2N3 (SBERT 0,830).

Gemma. Aparece como líder em revocação macro de classificação tanto no EN1 (0,917) quanto no EN2 (0,300), ou seja, boa cobertura, com precisão menor. As médias semânticas ficam em patamar intermediário.

Alpaca. Desempenho fraco em todos os experimentos, com queda acentuada no EN2 (SBERT 0,386; F1 macro 0,105).

Qwen. Pior desempenho geral. Colapsou no EN2 (TF-IDF 0,083; F1 macro 0,044) e teve a maior latência do *pipeline* encadeado (ver Seção 4.5). Pesa também a tendência do modelo a gerar saídas em chinês simplificado, fora do idioma esperado.

4.4 Análise por Métrica

Métricas lexicais. FuzzyWuzzy e TF-IDF caem claramente de EN1 para EN2 (FuzzyWuzzy: 0,475 $\rightarrow$ 0,461; TF-IDF: 0,636 $\rightarrow$ 0,312). Como dependem de sobreposição lexical, essas métricas punem as reformulações sintáticas e as sinonímias geradas pelos LLMs. Em EN3 e EN1N2N3, voltam a subir (FuzzyWuzzy: 0,847 e 0,617; TF-IDF: 0,597 e 0,514), beneficiadas pelo vocabulário curto e restrito do JSON.

Métricas semânticas. SBERT, BERTimbau e Multilingual mantêm médias altas e estáveis. O BERTimbau, treinado sobre texto jurídico em português, fica ligeiramente acima do SBERT em quase todos os modelos e experimentos (EN1: 0,816 vs. 0,801; EN1N2: 0,631 vs. 0,594), evidência prática do ganho de usar *embeddings* de domínio para validar transformações normativas. Em EN3, atingem os valores mais altos da bateria (SBERT 0,906; Multilingual 0,873), porque os campos JSON contêm sintagmas curtos com baixa variação léxica.

Métricas voltadas para geração. O BERTScore teve a maior média absoluta em EN2 (0,753) e em EN1N2 (0,790), graças ao alinhamento ótimo entre *tokens* mesmo quando há forte reformulação. Em EN3 e EN1N2N3, segue alto (0,880 e 0,843). Já o ROUGE-L comportou-se como métrica lexical, com queda acentuada em EN2 (0,308) e recuperação em EN3 (0,765).

Métricas de distância (WMD). Desempenho intermediário, entre lexicais e semânticas. Os valores foram mais altos em EN1 (WMD_ft 0,716; WMD_nilc 0,688) e em EN3 (WMD_ft 0,846; WMD_nilc 0,842), refletindo a proximidade lexical maior entre as saídas e o texto/JSON de referência nesses dois cenários.

Métricas de classificação. Em EN1, a F1 macro chega a 0,806 (Dolphin), o que indica que a maior parte das sentenças atômicas é segmentada corretamente. Em EN2, a F1 desaba para

0,286 no melhor caso (Dolphin), com colapso em Seleção e Exceção: F1 próxima de zero para todos os modelos. Esses dois operadores são especialmente difíceis de identificar mesmo para os líderes. Em EN3, a baixa precisão por correspondência exata (precisão macro abaixo de 4% para todos os modelos) mostra que o emparelhamento *token a token* é severo demais para um campo cuja semântica resiste a variações léxicas legítimas.

4.5 Tempo de Execução

O tempo total de geração foi medido por modelo e por experimento. A Tabela 10 traz os tempos em N1; a Tabela 11, em N2; a Tabela 12, no *pipeline* encadeado N1N2; a Tabela 13 traz os tempos do EN3 (N3 isolado, com N2 de referência como entrada); e a Tabela 14, do *pipeline* completo EN1N2N3.

Tabela 10 – Tempo total de geração por modelo em N1.

Modelo	Tempo (s)	Tempo (h:mm:ss)
Alpaca	104,55	0:01:44
Dolphin	62,65	0:01:02
Gemma	133,04	0:02:13
Llama	5 676,97	1:34:36
Mistral	70,04	0:01:10
Qwen	309,59	0:05:09

Tabela 11 – Tempo total de geração por modelo em N2.

Modelo	Tempo (s)	Tempo (h:mm:ss)
Alpaca	249,10	0:04:09
Dolphin	124,69	0:02:04
Gemma	244,19	0:04:04
Llama	6 895,71	1:54:55
Mistral	123,63	0:02:03
Qwen	6 515,09	1:48:35

Tabela 12 – Tempo total de geração por modelo em N1N2.

Modelo	Tempo (s)	Tempo (h:mm:ss)
Alpaca	330,84	0:05:30
Dolphin	104,28	0:01:44
Gemma	427,22	0:07:07
Llama	9 450,14	2:37:30
Mistral	138,39	0:02:18
Qwen	10 093,84	2:48:13

Os tempos expõem um *trade-off* nítido entre qualidade e custo computacional. Em EN1 e EN1N2, Llama e Qwen são os mais lentos: respectivamente 1h34min e 5min em N1; 2h37min

Tabela 13 – Tempo total de geração por modelo em N3 (EN3 isolado).

Modelo	Tempo (s)	Tempo (h:mm:ss)
Alpaca	2 104,92	0:35:05
Dolphin	584,96	0:09:45
Gemma	5 982,38	1:39:42
Llama	800,78	0:13:21
Mistral	627,68	0:10:28
Qwen	847,41	0:14:07

Tabela 14 – Tempo total de geração por modelo em N1N2N3 (*pipeline* completo).

Modelo	Tempo (s)	Tempo (h:mm:ss)
Alpaca	1 395,35	0:23:15
Dolphin	757,65	0:12:38
Gemma	6 364,48	1:46:04
Llama	808,62	0:13:29
Mistral	637,88	0:10:38
Qwen	808,72	0:13:29

e 2h48min em N1N2. Em EN3 e EN1N2N3, o quadro se inverte: o **Gemma** passa a ser o mais oneroso (1h39min em N3; 1h46min em N1N2N3), enquanto Llama, Mistral e Qwen ficam em torno de 10 a 14 minutos. O Dolphin combina alta qualidade com tempos na casa de 1 a 13 minutos em todos os experimentos (62 s em N1; 1m44s em N1N2; 9m45s em N3; 12m38s em N1N2N3), e segue como a melhor relação custo-benefício observada na bateria.

4.6 Discussão Integrada

A leitura conjunta dos três experimentos e das treze métricas permite consolidar cinco observações.

1. *Embeddings* semânticos e BERTScore são as métricas mais adequadas a esta tarefa. SBERT, BERTimbau, Multilingual e BERTScore capturam equivalência mesmo quando há reformulação lexical pesada, e são as métricas mais estáveis ao longo dos experimentos. O BERTimbau, treinado em corpus jurídico em PT-BR, fica ligeiramente acima do SBERT, evidência do ganho de *embeddings* de domínio.

2. O *pipeline* encadeado não degrada todas as métricas de forma uniforme. A queda esperada pela propagação de erros aparece nas lexicais (FuzzyWuzzy, TF-IDF e ROUGE-L). Já nas semânticas, o EN1N2 chega a *superar* o EN2 isolado nas médias agregadas (Multilingual 0,679 vs. 0,595): o N1 produzido pelo próprio modelo tende a gerar frases próximas das originais, o que favorece o emparelhamento semântico, mesmo quando o N2 erra na atribuição dos operadores. As métricas de classificação confirmam, porém, que esse ganho não se converte em classificação RASE melhor: o *pipeline* continua dependendo de saída atomizada e corretamente rotulada para que a verificação automática faça sentido.

3. Há perfis complementares entre os modelos líderes. O Llama lidera a qualidade global (0,730), com domínio absoluto sobre o nível N3 (as nove métricas em EN3 e em EN1N2N3) e também sobre o EN2; é a escolha quando se tolera o custo computacional. O Dolphin (0,725) entrega o melhor custo-benefício: domina o EN1, fica à frente nas métricas de distância do EN1N2 e tem o menor tempo de geração entre os líderes. O Mistral (0,721) é o terceiro e o mais consistente: nunca lidera com folga, mas nunca está longe.

4. Qwen e Alpaca não são recomendados. Ambos têm desempenho consistentemente inferior em todas as métricas e experimentos. O Qwen ainda registra a maior latência do *pipeline* encadeado (2h48min) e tende a gerar saídas fora do idioma esperado.

5. A decomposição em três níveis foi decisiva. Separar segmentação (N1), classificação RASE (N2) e estruturação JSON (N3) viabilizou saídas válidas sem utilizar *Fine-Tuning* nem *Retrieval-Augmented Generation*, ou seja, Engenharia de Prompt sozinha basta para alcançar similaridade semântica perto de 0,90 nos melhores cenários (Dolphin em EN1, com BERTimbau 0,891; Llama em N3 isolado, com SBERT 0,947). O que sobra de dificuldade está concentrado na classificação correta dos operadores Seleção e Exceção em N2, as classes mais difíceis para todos os modelos avaliados.

5

Conclusão

Esta dissertação investigou a aplicação de seis LLMs *open-source* (Llama, Dolphin, Gemma, Mistral, Alpaca e Qwen) na conversão automática de normas técnicas brasileiras da AEC para o formato RASE, usando **apenas Engenharia de Prompt**. A tarefa foi decomposta em três níveis sucessivos de normalização (N1, N2 e N3) e avaliada em cinco experimentos (EN1, EN2, EN1N2, EN3 e EN1N2N3), sobre um *dataset* de 79 normas anotadas. A avaliação combinou treze métricas: nove de similaridade textual (lexicais, semânticas, de distância e voltadas para geração) e quatro de classificação, derivadas da matriz de confusão.

5.1 Síntese dos Resultados

A decomposição em três níveis viabilizou a conversão de normas AEC em representações RASE sem recorrer a *Fine-Tuning* nem a *Retrieval-Augmented Generation*. Engenharia de Prompt sozinha bastou para alcançar similaridade semântica perto de 0,90 nos melhores cenários (Dolphin em EN1, com BERTimbau 0,891 e Multilingual 0,913; Llama em EN3 isolado, com SBERT 0,947; e novamente Llama em EN1N2N3, com SBERT 0,834 mesmo na cadeia completa). As métricas baseadas em *embeddings* (SBERT, BERTimbau, Multilingual) e o BERTScore foram as mais adequadas para validar transformações normativas, porque toleram reformulações que preservam o sentido. O BERTimbau manteve ganho consistente sobre o SBERT, evidência prática do valor de *embeddings* de domínio jurídico em PT-BR.

No agregado das nove métricas de similaridade sobre os cinco experimentos, o **Llama** fechou com a melhor média global (0,730), puxado pela liderança absoluta em EN3 e EN1N2N3 (as duas etapas do nível N3) e em EN2. O **Dolphin** ficou em segundo (0,725), com a melhor relação custo-benefício: cerca de noventa vezes mais rápido que o Llama no *pipeline* encadeado, e líder absoluto em EN1. O Mistral fechou em terceiro (0,721), o mais consistente da bateria. Qwen e Alpaca apresentaram desempenho insatisfatório nas etapas mais sensíveis a falhas semânticas

(EN2 e EN1N2), mas se recuperaram em EN3, beneficiados pelo vocabulário restrito do JSON. As métricas de classificação confirmaram um achado importante: os operadores Seleção e Exceção são as classes mais difíceis em N2 para todos os modelos, com F1 próxima de zero; já em EN3, a correspondência exata por campo derruba a precisão macro para abaixo de 4%, apontando a necessidade de critérios de classificação mais flexíveis para o nível N3.

5.2 Contribuições

A pesquisa entrega cinco contribuições centrais:

- A definição operacional de três níveis de normalização sobre a metodologia RASE: **N1** (segmentação atômica), **N2** (identificação dos operadores RASE) e **N3** (estruturação semântica em JSON), com *prompts* dedicados a cada nível e a cada operador.
- Uma comparação sistemática de seis LLMs *open-source* servidos localmente via Ollama, na tarefa de conversão de normas brasileiras da AEC, sob protocolo experimental idêntico (mesmo *dataset*, mesmos *prompts*, mesmo *hardware*).
- Um conjunto reproduzível de *prompts* especializados por nível e por operador, organizado em arquivos de texto dedicados, pronto para ser reutilizado e estendido em pesquisas futuras.
- Um *dataset* anotado de 79 normas brasileiras, com referência em todos os níveis (N1, N2 e N3) e por operador RASE, viabilizando avaliações replicáveis.
- Uma avaliação multimétrica com treze métricas (nove de similaridade textual e quatro de classificação), entregando um retrato multidimensional do desempenho de cada modelo, com detalhamento por instância em *metrics/details/* para apoiar reanálises.

5.3 Limitações

A pesquisa tem limitações que delimitam o alcance das conclusões:

- O *dataset* é pequeno (79 normas) e está concentrado em normas brasileiras de acessibilidade (NBR 9050 e similares), de modo que os resultados podem mudar em outros domínios normativos.
- As métricas de classificação dependem do critério de acerto adotado: limiar sobre similaridade semântica em N1 e N2 (parcialmente arbitrário) e correspondência exata em N3 (rígida demais para campos com variações léxicas legítimas). Critérios mais flexíveis em N3 e múltiplos limiares em N1 e N2 ficam como linha de aprofundamento.

- A classificação de Seleção e Exceção em N2 obteve F1 próxima de zero para todos os modelos, sinalizando uma limitação estrutural da formulação atual do *prompt* N2: nenhum dos seis LLMs conseguiu superá-la.
- Não houve aplicação prática em *software* BIM. A validação fim a fim em projetos reais ainda está em aberto; o ciclo completo de verificação automática de conformidade precisa ser fechado.
- Todos os experimentos rodaram em uma única estação de trabalho, sem análise de variância entre execuções, sem variação de temperatura e sem variação de *seed*.

5.4 Trabalhos Futuros

Algumas direções de pesquisa futura que considero promissoras:

- **Aplicação em BIM:** integrar os resultados ao Revit ou ao Dynamo e fechar o ciclo de verificação automática de conformidade em projetos reais.
- **Prompt N2 dedicado a Seleção e Exceção:** redesenhar o *prompt* N2 ou introduzir uma etapa específica para esses dois operadores. Ambos colapsaram em F1 próxima de zero nos seis modelos, o que torna esse o ponto de maior alavancagem.
- **Fine-Tuning e RAG:** explorar essas técnicas como evoluções naturais da abordagem baseada apenas em *prompts*, medindo o ganho em qualidade e em custo computacional, em especial para os operadores difíceis citados acima.
- **Ampliação do dataset:** incluir um espectro maior de NBRs (estruturais, hidráulicas, elétricas) e expandir para normas internacionais.
- **Avaliação humana:** complementar as métricas automáticas com avaliação por especialistas da AEC, capazes de capturar nuances semânticas que escapam aos *embeddings*.
- **Estudo de variância:** rodar múltiplas execuções por modelo, variando temperatura e *seed*, para quantificar a estabilidade das saídas.
- **Critérios alternativos de classificação em N3:** trocar a correspondência exata por correspondência normalizada (*lowercase*, remoção de *stopwords*, comparação por sinônimos) e revisitar Acurácia, Precisão, Revocação e F1 em N3.
- **Otimização do pipeline:** estudar estratégias de mitigação da propagação de erros em EN1N2 e em EN1N2N3: reescrita iterativa, *self-consistency* ou roteamento de operadores difíceis para o modelo mais adequado (*ensemble* Dolphin/Llama por nível).

No conjunto, a pesquisa mostra que LLMs *open-source*, somados a uma decomposição cuidadosa da tarefa e a *prompts* especializados, são alternativa viável e acessível para automatizar a conversão de normas técnicas da AEC em representações computáveis, abrindo caminho para a verificação automática de conformidade em projetos baseados em BIM.

Referências

- AI@META. Llama 3 model card. *github*, 2024. Disponível em: <https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md>. Citado 4 vezes nas páginas 13, 45, 46 e 47.
- AINSLIE, J. et al. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. 2023. Disponível em: <<https://arxiv.org/pdf/2305.13245v3>>. Citado na página 47.
- Alammar, J. *The Illustrated Transformer*. 2018. Acesso em: 15/05/2021. Disponível em: <<http://jalammar.github.io/illustrated-transformer/>>. Citado 2 vezes nas páginas 43 e 44.
- ALAWADHI, M.; YAN, W. Deep learning from parametrically generated virtual buildings for real-world object recognition. *arXiv preprint arXiv:2302.05283*, 2023. Disponível em: <<https://arxiv.org/abs/2302.05283>>. Citado na página 13.
- ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. *NBR 9050: Acessibilidade a edificações, mobiliário, espaços e equipamentos urbanos*. Rio de Janeiro, 2020. Citado 2 vezes nas páginas 18 e 56.
- Bahdanau, D.; Cho, K. H.; Bengio, Y. Neural machine translation by jointly learning to align and translate. In: *3rd International Conference on Learning Representations, ICLR 2015 ; Conference date: 07-05-2015 Through 09-05-2015*. [S.l.: s.n.], 2015. Citado na página 40.
- BAI, J. et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023. Citado na página 48.
- BEACH, T. H. et al. A rule-based semantic approach for automated regulatory compliance in the construction sector. *Expert Systems with Applications*, v. 42, n. 12, p. 5219–5231, 2015. Citado na página 19.
- Bengio, Y. et al. A neural probabilistic language model. *J. Mach. Learn. Res.*, JMLR.org, v. 3, p. 1137–1155, mar 2003. ISSN 1532-4435. Citado na página 28.
- Bojanowski, P. et al. Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, MIT Press, p. 135–146, 2017. Citado 2 vezes nas páginas 29 e 60.
- BOTTEGA, B. S. *Avaliação dos efeitos do uso da tecnologia BIM sobre a coordenação de projetistas*. [S.l.]: Dissertação de Mestrado - Universidade Federal do Rio Grande do Sul, 2012. Citado na página 17.
- Braga, A. de P.; Ludermir, T. B.; Carvalho, A. C. P. de L. F. *Redes neurais artificiais: teoria e aplicações*. [S.l.]: Ltc, 2000. Citado na página 35.
- BROWN, T. B. et al. *Language Models are Few-Shot Learners*. 2020. Disponível em: <<https://arxiv.org/abs/2005.14165>>. Citado na página 49.
- Bulegon, H.; Moro, C. M. C. Mineração de texto e o processamento de linguagem natural em sumários de alta hospitalar. In: *Mineração de texto e o processamento de linguagem natural em sumários de alta hospitalar*. SP - Brasil: Journal of Health Informatics, 2010. v. 2, n. 2. Citado na página 26.

Cerri, R.; Carvalho, A. C. P. de Leon Ferreira de. Aprendizado de máquina: Breve introdução e aplicações. *Cadernos de Ciência & Tecnologia*, Brasília, v. 34, n. 3, p. 297–313, 12 2017. Citado na página 22.

Chai, T.; Draxler, R. Root mean square error (rmse) or mean absolute error (mae)? – arguments against avoiding rmse in the literature. *Geoscientific Model Development*, v. 7, n. 3, p. 1247–1250, 2014. Citado na página 34.

Chung, J. et al. *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling*. 2014. Citado na página 38.

COELHO, S. S.; NOVAES, C. C. Modelagem de informações para construção (bim) e ambientes colaborativos para gestão de projetos na construção civil. *Exatas e Engenharia*, v. 8, n. 23, 12 2018. Citado 2 vezes nas páginas 16 e 17.

Conneau, A. et al. Supervised learning of universal sentence representations from natural language inference data. In: *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*. Copenhagen, Denmark: Association for Computational Linguistics, 2017. p. 670–680. Citado na página 28.

DAVENPORT, T. H.; PATIL, D. Data scientist: the sexiest job of the 21st century. *Harvard Business Review*, v. 1, n. 1, 2012. Disponível em: <<https://hbr.org/2012/10/data-scientist-the-sexiest-job-of-the-21st-century>>. Citado na página 12.

DIMYADI, J.; AMOR, R. Automated building code compliance checking – where is it at? In: *Proceedings of the 19th International CIB World Building Congress*. Brisbane: [s.n.], 2013. p. 172–185. Citado 2 vezes nas páginas 13 e 19.

EASTMAN, C. et al. Automatic rule-based checking of building designs. *Automation in Construction*, v. 18, n. 8, p. 1011–1033, 2009. Citado na página 19.

EASTMAN, C.; TEICHOLZ, P.; SACKS, R. *Bim Handbook: A Guide to Building Information Modeling for Owners, Managers, Designers, Engineers and Contractors*. 2. ed. Nova Jersey: Wiley, 2011. ISBN 978-04-7054-137-01. Citado 3 vezes nas páginas 12, 16 e 17.

Faceli, K. et al. *Inteligência Artificial - Uma Abordagem de Aprendizado de Máquina*. 1. ed. Rio de Janeiro: Ltc, 2011. 192 p. ISBN 9788521618805. Citado na página 22.

Feng, J.; Lu, S. Performance analysis of various activation functions in artificial neural networks. *Journal of Physics: Conference Series*, v. 1237, p. 22–30, 06 2019. Citado 3 vezes nas páginas 32, 33 e 34.

FUCHS, S. et al. Using large language models for the interpretation of building regulations. *arXiv preprint arXiv:2407.21060*, 2024. Disponível em: <<https://arxiv.org/abs/2407.21060>>. Citado 2 vezes nas páginas 13 e 19.

Gemma Team. *Gemma: Open Models Based on Gemini Research and Technology*. [S.l.], 2024. ArXiv:2403.08295. Citado na página 48.

Goodfellow, I.; Bengio, Y.; Courville, A. *Deep Learning*. [S.l.]: MIT Press, 2016. 166–223 p. Citado 6 vezes nas páginas 25, 31, 32, 36, 37 e 38.

Graves, A.; Jaitly, N.; Mohamed, A. rahman. Hybrid speech recognition with deep bidirectional lstm. In: *2013 IEEE Workshop on Automatic Speech Recognition and Understanding*. [S.l.: s.n.], 2013. p. 273–278. Citado na página 38.

Guimarães, L. M. S.; Meireles, M. R. G.; Almeida, P. E. M. de. Avaliação das etapas de pré-processamento e de treinamento em algoritmos de classificação de textos no contexto da recuperação da informação. *Perspectivas em Ciência da Informação*, Belo Horizonte, v. 24, n. 1, p. 169–190, 05 2019. Citado 2 vezes nas páginas 22 e 23.

Hafemann, L. G. *An Analysis of Deep Neural Networks for Texture Classification*. [S.l.]: Dissertação de Mestrado - Universidade Federal do Paraná, 2014. Citado na página 34.

Haykin, S. *Neural Networks: A Comprehensive Foundation*. Upper Saddle River, NJ: Prentice Hall, 1999. 2nd EDITION. Citado na página 30.

Haykin, S. *Redes Neurais - 2ed*. [S.l.]: Bookman, 2001. ISBN 9788573077186. Citado 2 vezes nas páginas 35 e 36.

Hebb, D. O. *The organization of behavior: neuropsychological theory*. New York: Wiley, 1949. Jun. ISBN 0-8058-4300-0. Citado na página 30.

HJELSETH, E. Bim-based model checking (bmc). In: _____. [S.l.]: Building Information Modeling: Applications and Practices, 2015. p. 33–61. ISBN 978-0-7844-1398-2. Citado na página 19.

HJELSETH, E.; NISBET, N. N. Capturing normative constraints by use of the semantic mark-up rase methodology. In: *Capturing normative constraints by use of the semantic mark-up RASE methodology*. [s.n.], 2011. Disponível em: <<https://api.semanticscholar.org/CorpusID:62759547>>. Citado 3 vezes nas páginas 13, 19 e 20.

HOCH. 2016. Citado 2 vezes nas páginas 4 e 17.

Hochreiter, S.; Schmidhuber, J. Long short-term memory. *Neural Comput.*, MIT Press, Cambridge, MA, USA, v. 9, n. 8, p. 1735–1780, nov 1997. ISSN 0899-7667. Citado 2 vezes nas páginas 37 e 38.

IBRAHIM ROBERT KRAWCZYK, G. S. M. Two approaches to bim: A comparative study. *Illinois Institute of Technology*, Chicago, 2004. Citado na página 17.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. *ISO 19650-1: Organization and digitization of information about buildings and civil engineering works, including building information modelling (BIM) – Information management using building information modelling – Part 1: Concepts and principles*. Geneva, 2018. Citado na página 18.

JIANG, A. Q. et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023. Citado na página 48.

Jurafsky, D.; Martin, J. H. *Speech and Language Processing (2nd Edition)*. Usa: Prentice-Hall, Inc., 2009. ISBN 0131873210. Citado 2 vezes nas páginas 25 e 29.

KUSNER, M. J. et al. From word embeddings to document distances. In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. [S.l.: s.n.], 2015. p. 957–966. Citado 2 vezes nas páginas 29 e 60.

- Lecun, Y. et al. Gradient-based learning applied to document recognition. In: *Gradient-based learning applied to document recognition*. [S.l.]: Institute of Electrical and Electronics Engineers Inc., 1998. v. 86, n. 11, p. 2278–2324. ISSN 0018-9219. Citado na página 39.
- Liddy, E. D. *Enhanced text retrieval using natural language processing*. [S.l.]: American Society for Information Science and Technology, 1998. 14–16 p. Citado na página 25.
- LIN, C.-Y. ROUGE: A package for automatic evaluation of summaries. In: *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*. [S.l.]: Association for Computational Linguistics, 2004. p. 74–81. Citado 2 vezes nas páginas 29 e 60.
- LIN, W. Y. Prototyping a chatbot for site managers using building information modeling (bim) and natural language understanding (nlu) techniques. *Sensors*, v. 23, n. 6, 2023. ISSN 1424-8220. Disponível em: <<https://www.mdpi.com/1424-8220/23/6/2942>>. Citado 2 vezes nas páginas 49 e 50.
- LIU, Y.; ZHANG, X.; LI, H. Bim machine learning and design rules to improve the assembly process. *Sustainability*, v. 14, n. 1, p. 288, 2022. Disponível em: <<https://www.mdpi.com/2071-1050/14/1/288>>. Citado na página 12.
- Manning, C.; Raghavan, P.; Schütze, H. *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008. ISBN 978-0-521-86571-5. Citado 2 vezes nas páginas 29 e 60.
- Martin, J. H.; Jurafsky, D. *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition*. [S.l.]: Pearson/Prentice Hall Upper Saddle River, 2009. Citado na página 28.
- McCaffrey, J. *Why You Should Use Cross-Entropy Error Instead Of Classification Error Or Mean Squared Error For Neural Network Classifier Training*. 2013. Acesso em: 12/02/2021. Citado na página 34.
- Mcculloch, W.; Pitts, W. A logical calculus of ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, p. 127–147, 1943. 5. Citado na página 30.
- MENDONÇA, E. A. d.; MANZIONE, L. *Conversão de regras de acessibilidade pela metodologia Rase para verificação automática em modelo BIM*. 2020. Citado na página 56.
- Mikolov, T. et al. Distributed representations of words and phrases and their compositionality. In: *Advances in neural information processing systems*. [S.l.: s.n.], 2013. p. 3111–3119. Citado na página 28.
- Minsky, M.; Papert, S. *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA, USA: MIT Press, 1969. 355–368 p. Citado na página 30.
- Mitchell, T. M. *Machine Learning*. 1. ed. Nova York: McGraw-Hill International Editions, 1997. 14 p. ISBN 978-00-7115-467-3. Citado na página 22.
- Mun, J.; Cho, M.; Han, B. *Text-guided Attention Model for Image Captioning*. 2016. Citado na página 39.
- NASCIMENTO, J. R. *Exploração de técnicas de engenharia de prompt para aprimorar os resultados do uso de LLM no TCMRio*. Natal: [s.n.], 2024. Acesso em: 25 maio 2024. Disponível em: <<https://repositorio.ufrn.br/handle/123456789/58251>>. Citado na página 49.

- Nasr, G. E.; Badr, E. A.; Joun, C. Cross entropy error function in neural networks: Forecasting gasoline demand. In: *Proceedings of the Fifteenth International Florida Artificial Intelligence Research Society Conference*. [S.l.]: AAAI Press, 2002. p. 381–384. ISBN 157735141x. Citado na página 35.
- NAVEED, H. et al. *A Comprehensive Overview of Large Language Models*. 2024. Disponível em: <<https://arxiv.org/abs/2307.06435>>. Citado na página 45.
- Neves, R. de Cássia David das. *Pré-processamento no processo de descoberta de conhecimento em banco de dados*. [S.l.]: Dissertação de Mestrado - UFRGS Lume, 2003. Citado na página 23.
- Ollama. *Ollama: Get up and running with large language models locally*. 2024. <<https://ollama.com>>. Acesso em: 2026-05-18. Citado na página 55.
- OPENAI et al. *GPT-4 Technical Report*. 2024. Disponível em: <<https://arxiv.org/abs/2303.08774>>. Citado 2 vezes nas páginas 13 e 45.
- PARREIRA, J. P. de C. *Implementação BIM nos processos organizacionais em empresas de construção – um caso de estudo*. [S.l.]: Dissertação de Mestrado - Faculdade de Ciências e Tecnologia, 2013. Citado na página 16.
- Pennington, J.; Socher, R.; Manning, C. D. Glove: Global vectors for word representation. In: MOSCHITTI, A.; PANG, B.; DAELEMANS, W. (Ed.). *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*. Doha, Qatar: Association for Computational Linguistics, 2014. p. 1532–1543. Disponível em: <<https://aclanthology.org/D14-1162/>>. Citado na página 28.
- PENTTILÄ, H. Describing the changes in architectural information technology to understand design complexity and free-form architectural expression. *Journal of Information Technology in Construction*, v. 11, n. 4, 02 2006. Citado na página 16.
- Popel, M.; Bojar, O. Training tips for the transformer model. *The Prague Bulletin of Mathematical Linguistics*, Charles University in Prague, Karolinum Press, v. 110, n. 1, p. 43–70, Apr 2018. ISSN 1804-0462. Citado 2 vezes nas páginas 42 e 44.
- RAFAILOV, R. et al. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. 2024. Disponível em: <<https://arxiv.org/abs/2305.18290>>. Citado na página 46.
- RANE, N.; CHOUDHARY, S.; RANE, J. Integrating building information modelling (bim) with chatgpt, bard, and similar generative artificial intelligence in the architecture, engineering, and construction industry: applications, a novel framework, challenges, and future scope. *SSRN Electronic Journal*, 01 2023. Citado 3 vezes nas páginas 12, 49 e 50.
- REIMERS, N.; GUREVYCH, I. Sentence-BERT: Sentence embeddings using Siamese BERT-Networks. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2019. p. 3982–3992. Disponível em: <<https://arxiv.org/abs/1908.10084>>. Citado 3 vezes nas páginas 29, 30 e 60.
- Rezende, S. O.; Marcacini, R. M.; Moura, M. F. O uso da mineração de textos para extração e organização não supervisionada de conhecimento. *Revista de Sistemas de Informação da FSMA*, v. 7, p. 7–21, 2011. ISSN 19835604. Citado na página 26.
- Rojas, R. *Neural Networks - A Systematic Introduction*. Berlin: Springer-Verlag, 1996. 154–155 p. Citado na página 31.

- Rosenblatt, F. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, v. 65, n. 6, p. 386–408, 1958. Citado na página 30.
- Rumelhart, D. E.; Hinton, G. E.; Williams, R. J. Learning Representations by Back-propagating Errors. *Nature*, v. 323, n. 6088, p. 533–536, 1986. Citado 2 vezes nas páginas 30 e 35.
- RUSSELL, S.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. 3. ed. [S.l.]: Prentice Hall, 2010. Citado 4 vezes nas páginas 23, 25, 34 e 35.
- SAKA, A. et al. Integrated bim and machine learning system for circularity prediction of construction demolition waste. *arXiv preprint arXiv:2407.14847*, 2024. Disponível em: <<https://arxiv.org/abs/2407.14847>>. Citado na página 13.
- SAMMOUR, F. et al. *Responsible AI in Construction Safety: Systematic Evaluation of Large Language Models and Prompt Engineering*. 2024. Disponível em: <<https://arxiv.org/abs/2411.08320>>. Citado 2 vezes nas páginas 50 e 51.
- SAMSAMI, R. Optimizing the utilization of generative artificial intelligence (ai) in the aec industry: Chatgpt prompt engineering and design. *CivilEng*, v. 5, n. 4, p. 971–1010, 2024. ISSN 2673-4109. Disponível em: <<https://www.mdpi.com/2673-4109/5/4/49>>. Citado 2 vezes nas páginas 50 e 51.
- SANTOS, K. T.; SAMPAIO, M. A. B. Aplicação da metodologia rase na norma de acessibilidade associada ao bim. *SIMPÓSIO BRASILEIRO DE GESTÃO E ECONOMIA DA CONSTRUÇÃO*, v. 12, n. 00, p. 1–8, out. 2021. Disponível em: <<https://eventos.antac.org.br/index.php/sibragec/article/view/500>>. Citado na página 13.
- Santos, R. et al. Técnicas de processamento de linguagem natural aplicadas ao processo de mineração de textos: Resultados preliminares de um mapeamento sistemático. In: *Técnicas de processamento de linguagem natural aplicadas ao processo de mineração de textos: Resultados preliminares de um mapeamento sistemático*. [S.l.: s.n.], 2015. Citado na página 26.
- Schmidhuber, J. Deep learning in neural networks: An overview. *Neural Networks*, Elsevier BV, v. 61, p. 85–117, Jan 2015. ISSN 0893-6080. Citado 2 vezes nas páginas 25 e 35.
- Shao, L.; Zhu, F.; Li, X. Transfer learning for visual categorization: a survey. *IEEE transactions on neural networks and learning systems*, v. 26, n. 5, p. 1019–1034, May 2015. ISSN 2162-237x. Citado na página 25.
- Soares, D.; Teive, R. Estudo comparativo entre as redes neurais artificiais mlp e rbf para previsão de cheias em curto prazo. *Revista de Informática Teórica e Aplicada*, v. 22, n. 2, p. 67–86, 2015. ISSN 21752745. Citado na página 30.
- SOLIHIN, W.; EASTMAN, C. Classification of rules for automated BIM rule checking development. *Automation in Construction*, v. 53, p. 69–82, 2015. Citado 2 vezes nas páginas 13 e 19.
- Souza, F.; Nogueira, R.; Lotufo, R. BERTimbau: pretrained BERT models for Brazilian Portuguese. In: *9th Brazilian Conference on Intelligent Systems, BRACIS, Rio Grande do Sul, Brazil, October 20-23 (to appear)*. [S.l.: s.n.], 2020. Citado 2 vezes nas páginas 29 e 60.
- SOUZA, L. L. A. de. *Diagnóstico do uso do BIM em empresas de projeto de Arquitetura*. [S.l.]: Dissertação de Mestrado - Universidade Federal Fluminense, 2009. Citado na página 17.

- STRUBELL, E.; GANESH, A.; MCCALLUM, A. *Energy and Policy Considerations for Deep Learning in NLP*. 2019. Disponível em: <<https://arxiv.org/abs/1906.02243>>. Citado na página 45.
- Sutskever, I.; Vinyals, O.; Le, Q. V. Sequence to sequence learning with neural networks. In: Ghahramani, Z. et al. (Ed.). *Advances in Neural Information Processing Systems*. [S.l.]: Curran Associates, Inc., 2014. v. 27. Citado na página 38.
- Tafner, M. A.; Xerez, M. de; Filho, E. R. *Redes neurais artificiais: introdução e princípios de neurocomputação*. Blumenau: Eko, 1995. ISBN 8571140502. Citado na página 30.
- Tai, K. S.; Socher, R.; Manning, C. Improved semantic representations from tree-structured long short-term memory networks. In: *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Beijing, China: Association for Computational Linguistics, 2015. p. 1556–1566. Citado 2 vezes nas páginas 37 e 38.
- TAN, P.-N.; STEINBACH, M.; KUMAR, V. *Introdução a DATAMINING Mineração de Dados*. 2. ed. Rio de Janeiro: Person Education, Inc., 2006. ISBN 978-85-7393-761-9. Citado 3 vezes nas páginas 12, 22 e 23.
- TAORI, R. et al. *Stanford Alpaca: An Instruction-following LLaMA model*. 2023. <https://github.com/tatsu-lab/stanford_alpaca>. Citado na página 48.
- TOUVRON, H. et al. *LLaMA: Open and Efficient Foundation Language Models*. 2023. Disponível em: <<https://arxiv.org/abs/2302.13971>>. Citado 2 vezes nas páginas 44 e 45.
- Vaswani, A. et al. *Attention Is All You Need*. 2017. Disponível em: <<https://arxiv.org/abs/1706.03762>>. Citado 4 vezes nas páginas 41, 44, 46 e 47.
- Vieira, R.; Lima, V. L. S. de. *Linguística computacional: princípios e aplicações*. Jaia, SBC, Fortaleza, Brasil, 2001. Citado na página 26.
- Willmott, C. J. On the validation of models. *Physical Geography*, Taylor and Francis, v. 2, n. 2, p. 184–194, 1981. Citado 2 vezes nas páginas 34 e 35.
- ZHANG, T. et al. BERTScore: Evaluating text generation with BERT. In: *Proceedings of the 8th International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2020. Citado 2 vezes nas páginas 29 e 60.
- ZHAO, W. X. et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023. Disponível em: <<https://arxiv.org/abs/2303.18223>>. Citado na página 13.

Apêndices

APÊNDICE



Prompts utilizados na Engenharia de Prompt

Este apêndice reproduz, na íntegra e exatamente como foram enviados aos modelos, os *prompts* de cada nível de normalização (N1, N2 e N3). Cada *prompt* reside em um arquivo de texto próprio no diretório `prompts/` do repositório. Os textos são apresentados sem acentuação nas instruções, tal como nos arquivos-fonte; a acentuação aparece apenas nos exemplos de saída esperada. Os marcadores entre chaves — `{text}`, `{text_n1}`, `{aplicabilidade}`, `{requisito}`, `{selecao}` e `{execao}` — são substituídos, em tempo de execução, pelo texto bruto da norma, pela regra atômica do N1 e pelo operador N2 correspondente.

A.1 N1 — *Prompt Direct*

Código A.1 – *Prompt* do nível N1 (*Prompt Direct*) — `prompts/n1.txt`.

```

1 Voce e um reescritor. Transforme o texto em sentencas RASE N1.
2
3 Regras:
4 1) Quebre em sentencas curtas e diretas.
5 2) Cada sentenca deve ter uma unica regra computavel.
6 3) Nao invente elementos (aplicabilidade, selecao, requisito,
   excecao). Preserve apenas o que existir.
7 4) Nao adicione explicacoes, titulos, bullets, numeracao ou o texto
   original.
8 5) Saidas: apenas as sentencas, uma por linha, terminadas com ponto
   final.
9
10 Exemplo:
11 Entrada:
12 A inclinacao transversal da superficie deve ser de ate 2 % para
   pisos internos e de ate 3 % para pisos externos.
```

```
13 Saida:
14 Pisos internos devem ter inclinacao transversal de no maximo 2%.
15 Pisos externos devem ter inclinacao transversal de no maximo 3%.
16
17 TEXTO_INICIO
18 {text}
19 TEXTO_FIM
```

A.2 N2 — *Chain-of-Thought*

Código A.2 – Prompt do nível N2 (*Chain-of-Thought*) — prompts/n2.txt.

```
1 Extrator RASE N2.
2 Use APENAS o Texto N1 para extrair os elementos. O Texto completo e
  apenas referencia.
3
4 Regras (ordem fixa):
5 1) aplicabilidade (opcional): onde/quando se aplica, sem verbos.
6 2) selecao (opcional): subconjunto da aplicabilidade, sem verbos.
7 3) execucao (opcional): casos que NAO seguem a regra.
8 4) requisito (obrigatorio): acao/condicao principal, comece com
  verbo.
9
10 Regras de saida:
11 - Retorne exatamente 4 linhas no formato abaixo.
12 - Cada campo deve aparecer no maximo uma vez.
13 - Se nao existir, use "" (string vazia).
14 - Nao adicione explicacoes, listas ou texto extra.
15
16 Exemplo (formato):
17 Texto completo:
18 "As areas ... Norma."
19 Texto N1:
20 "As areas de qualquer espaco ou edificacao de uso publico ou
   coletivo devem ser servidas de uma ou mais rotas acessiveis."
21 Resposta (4 linhas):
22 aplicabilidade: As areas de qualquer espaco ou edificacao
23 selecao: uso publico ou coletivo
24 execucao: ""
25 requisito: devem ser servidas de uma ou mais rotas acessiveis
26
27 Agora processe:
```

```

28 Texto completo:
29 "{text}"
30 Texto N1:
31 "{text_n1}"
32
33 Resposta (4 linhas):
34 aplicabilidade:
35 selecao:
36 execucao:
37 requisito:

```

A.3 N3 — *Few-Shot*

O nível N3 emprega um *prompt* por operador RASE. Os quatro compartilham a mesma estrutura — esquema JSON de saída, heurísticas de mapeamento e exemplos *few-shot* —, variando apenas as heurísticas e os exemplos próprios de cada operador.

A.3.1 Aplicabilidade

Código A.3 – *Prompt* do operador Aplicabilidade (N3) —
prompts/n3_aplicabilidade.txt.

```

1 Extrator RASE N3 (aplicabilidade).
2 Use APENAS o operador N2 de aplicabilidade para gerar as
  propriedades N3. O texto completo/N1 e apenas referencia, se
  existir.
3
4 Regras:
5 1) Gere um JSON com os campos: type, object, property, comparation,
  target, unit.
6 2) type deve ser "aplicabilidade". Se o operador estiver vazio,
  todos os campos devem ser "".
7 3) Use o operador como fonte principal; use Texto completo/N1 apenas
  para desambiguar.
8 4) object: substantivo(s) principal(is) do trecho; se houver mais de
  um, separe por "; ".
9 5) property/comparation/target:
10 - Quando o trecho indicar uso (ex: "edificacoes residenciais"),
  use property "uso".
11 - comparation "=" quando o texto definir o uso.
12 - comparation "contém" quando for subconjunto/parte do uso.

```

```
13 - Quando o trecho indicar "acessos", use object "componente",
    property "tipo",
14    comparation "inclui", target "porta; passagem".
15 - Quando o trecho indicar "edificacoes e equipamentos urbanos",
    use
16    object "edificação; equipamento urbano" e target "obra nova".
17 - Quando o trecho indicar tipo do componente (ex: entradas,
    rotas), use property "tipo", comparation "inclui".
18 - Quando nao houver propriedade clara, deixe
    property/comparation/target vazios.
19 6) unit: use "" (vazio) a menos que exista unidade explicita.
20
21 Regras de saida:
22 - Retorne somente um JSON valido.
23 - Nao adicione explicacoes ou texto extra.
24
25 Exemplo:
26 Operador N2 (aplicabilidade):
27 As edificacoes residenciais multifamiliares, condominios e conjuntos
    habitacionais
28
29 Resposta (JSON):
30 {
31   "type": "aplicabilidade",
32   "object": "edificação",
33   "property": "uso",
34   "comparation": "=",
35   "target": "residenciais multifamiliares, condomínios e conjuntos
    habitacionais",
36   "unit": ""
37 }
38
39 Exemplo:
40 Operador N2 (aplicabilidade):
41 Os acessos
42
43 Resposta (JSON):
44 {
45   "type": "aplicabilidade",
46   "object": "componente",
47   "property": "tipo",
48   "comparation": "inclui",
```

```
49   "target": "porta; passagem",
50   "unit": ""
51 }
52
53 Exemplo:
54 Operador N2 (aplicabilidade):
55 edificacoes e equipamentos urbanos
56
57 Resposta (JSON):
58 {
59   "type": "aplicabilidade",
60   "object": "edificação; equipamento urbano",
61   "property": "",
62   "comparation": "",
63   "target": "obra nova",
64   "unit": ""
65 }
66
67 Exemplo:
68 Operador N2 (aplicabilidade):
69 As unidades autonomas acessiveis
70
71 Resposta (JSON):
72 {
73   "type": "aplicabilidade",
74   "object": "espaço",
75   "property": "uso",
76   "comparation": "contém",
77   "target": "comum",
78   "unit": ""
79 }
80
81 Agora processe:
82 Texto completo:
83 "{text}"
84 Texto N1:
85 "{text_n1}"
86 Operador N2 (aplicabilidade):
87 "{aplicabilidade}"
88
89 Resposta (JSON):
```

A.3.2 Requisito

Código A.4 – *Prompt* do operador Requisito (N3) — `prompts/n3_requisito.txt`.

```
1 Extrator RASE N3 (requisito).
2 Use APENAS o operador N2 de requisito para gerar as propriedades N3.
  O texto completo/N1 e apenas referencia, se existir.
3
4 Regras:
5 1) Gere um JSON com os campos: type, object, property, comparison,
  target, unit.
6 2) type deve ser "requisito". Se o operador estiver vazio, todos os
  campos devem ser "".
7 3) Use o operador como fonte principal; use Texto completo/N1 apenas
  para desambiguar.
8 4) object: substantivo(s) principal(is) do trecho; se houver mais de
  um, separe por "; ".
9 5) property/comparison/target:
10 - "acessível" -> property "acessível", comparison "=", target
  "VERDADEIRO"
11 - "conectadas as rotas acessíveis" / "vinculados através de rota
  acessível" ->
12   property "conectado a rota acessível" (ou "conectado à rota
  acessível" se houver crase),
13   comparison "=", target "VERDADEIRO"
14 - "livres de obstáculos" -> property "possui obstáculo",
  comparison "=", target "FALSO"
15 - "devem ser servidas de rota(s) acessíveis" -> property "uso",
  comparison "=", target "VERDADEIRO"
16 - Quando não houver propriedade clara, deixe
  property/comparison/target vazios.
17 6) unit: use "" (vazio) a menos que exista unidade explícita.
18
19 Regras de saída:
20 - Retorne somente um JSON válido.
21 - Não adicione explicações ou texto extra.
22
23 Exemplo:
24 Operador N2 (requisito):
25 devem permanecer livres de quaisquer obstáculos de forma permanente
26
27 Resposta (JSON):
28 {
```

```
29   "type": "requisito",
30   "object": "porta; passagem",
31   "property": "possui obstáculo",
32   "comparation": "=",
33   "target": "FALSO",
34   "unit": ""
35 }
36
37 Exemplo:
38 Operador N2 (requisito):
39 devem ser acessíveis
40
41 Resposta (JSON):
42 {
43   "type": "requisito",
44   "object": "porta; espaço",
45   "property": "acessível",
46   "comparation": "=",
47   "target": "VERDADEIRO",
48   "unit": ""
49 }
50
51 Exemplo:
52 Operador N2 (requisito):
53 devem ser servidas de uma ou mais rotas acessíveis
54
55 Resposta (JSON):
56 {
57   "type": "requisito",
58   "object": "espaço ou edificação",
59   "property": "uso",
60   "comparation": "=",
61   "target": "VERDADEIRO",
62   "unit": ""
63 }
64
65 Agora processe:
66 Texto completo:
67 "{text}"
68 Texto N1:
69 "{text_n1}"
70 Operador N2 (requisito):
```

```
71 "{requisito}"
72
73 Resposta (JSON):
```

A.3.3 Seleção

Código A.5 – Prompt do operador Seleção (N3) — prompts/n3_selecao.txt.

```
1 Extrator RASE N3 (selecao).
2 Use APENAS o operador N2 de selecao para gerar as propriedades N3. O
  texto completo/N1 e apenas referencia, se existir.
3
4 Regras:
5 1) Gere um JSON com os campos: type, object, property, comparation,
  target, unit.
6 2) type deve ser "selecao". Se o operador estiver vazio, todos os
  campos devem ser "".
7 3) Use o operador como fonte principal; use Texto completo/N1 apenas
  para desambiguar.
8 4) object: substantivo(s) principal(is) do trecho; se houver mais de
  um, separe por "; ".
9 5) property/comparation/target:
10 - "uso publico/coletivo/comum/restrito" -> property "uso".
11 - comparation "=" quando o texto definir o uso.
12 - comparation "contém" quando o texto indicar subconjunto (ex:
  "uso comum", "uso restrito").
13 - listagem de tipos (entradas, rotas, circulacoes) -> property
  "tipo", comparation "inclui",
14 target com tipos normalizados (ex: "porta; passagem;
  circulação").
15 - Quando a selecao for apenas um qualificador (ex: "internos",
  "externos", "eventuais"),
16 deixe todos os campos vazios.
17 - Quando nao houver propriedade clara, deixe
  property/comparation/target vazios.
18 6) unit: use "" (vazio) a menos que exista unidade explicita.
19
20 Regras de saida:
21 - Retorne somente um JSON valido.
22 - Nao adicione explicacoes ou texto extra.
23
24 Exemplo:
```



```
25 Operador N2 (selecao):
26 uso publico ou coletivo
27
28 Resposta (JSON):
29 {
30     "type": "selecao",
31     "object": "espaço",
32     "property": "uso",
33     "comparation": "=",
34     "target": "público ou coletivo",
35     "unit": ""
36 }
37
38 Exemplo:
39 Operador N2 (selecao):
40 todas as entradas, bem como as rotas de interligacao as funcoes do
    edificio
41
42 Resposta (JSON):
43 {
44     "type": "selecao",
45     "object": "componente; espaço",
46     "property": "tipo",
47     "comparation": "inclui",
48     "target": "porta; passagem; circulação",
49     "unit": ""
50 }
51
52 Agora processe:
53 Texto completo:
54 "{text}"
55 Texto N1:
56 "{text_n1}"
57 Operador N2 (selecao):
58 "{selecao}"
59
60 Resposta (JSON):
```

A.3.4 Exceção

```
1 Extrator RASE N3 (execao).
2 Use APENAS o operador N2 de execao para gerar as propriedades N3. O
  texto completo/N1 e apenas referencia, se existir.
3
4 Regras:
5 1) Gere um JSON com os campos: type, object, property, comparation,
  target, unit.
6 2) type deve ser "excecao". Se o operador estiver vazio, todos os
  campos devem ser "".
7 3) Use o operador como fonte principal; use Texto completo/N1 apenas
  para desambiguar.
8 4) object: substantivo(s) principal(is) do trecho; se houver mais de
  um, separe por "; ".
9 5) property/comparation/target:
10 - "uso restrito" ou listagem de areas excluidas -> property
   "uso", comparation "contém",
11   target com lista normalizada (ex: "casa de máquinas, barrilete
   e técnico").
12 - Quando for apenas condicao/tempo ("em reformas", "se ..."),
   deixe todos os campos vazios.
13 - Quando nao houver propriedade clara, deixe
   property/comparation/target vazios.
14 6) unit: use "" (vazio) a menos que exista unidade explicita.
15
16 Regras de saida:
17 - Retorne somente um JSON valido.
18 - Nao adicione explicacoes ou texto extra.
19
20 Exemplo:
21 Operador N2 (execao):
22 Areas de uso restrito, como casas de maquinas, barriletes e passagem
  de uso tecnico
23
24 Resposta (JSON):
25 {
26   "type": "excecao",
27   "object": "espaço",
28   "property": "uso",
29   "comparation": "contém",
30   "target": "casa de máquinas, barrilete e técnico",
31   "unit": ""
32 }
```

```
33
34 Exemplo:
35 Operador N2 (execao):
36 Em reformas
37
38 Resposta (JSON):
39 {
40     "type": "",
41     "object": "",
42     "property": "",
43     "comparation": "",
44     "target": "",
45     "unit": ""
46 }
47
48 Agora processe:
49 Texto completo:
50 "{text}"
51 Texto N1:
52 "{text_n1}"
53 Operador N2 (execao):
54 "{execao}"
55
56 Resposta (JSON):
```